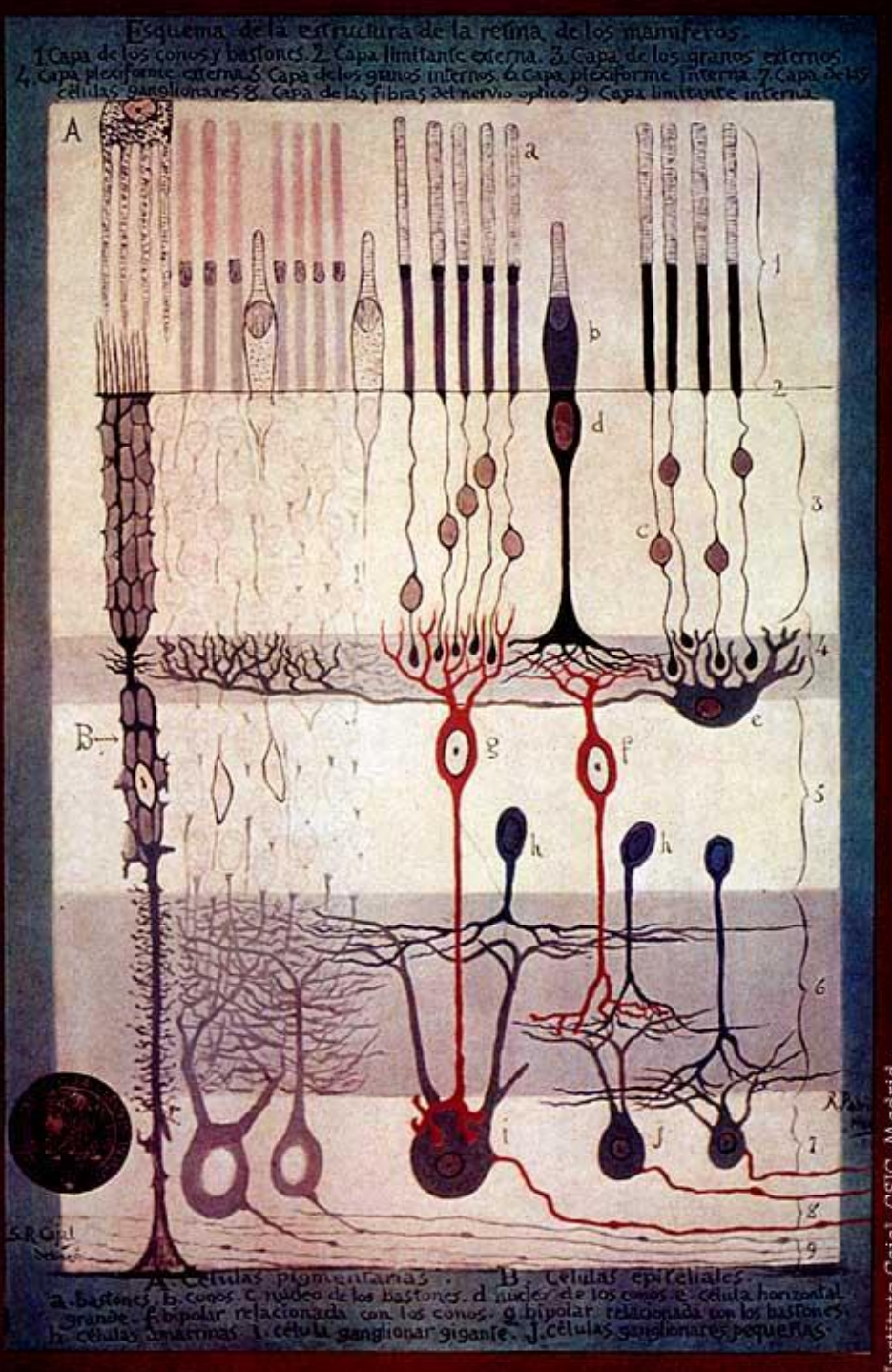




Convolutional Neural Network

Computer Vision

Dr Varun Ojha

varun.ojha@newcastle.ac.uk

ojhavk.github.io

School of Computing
Newcastle University

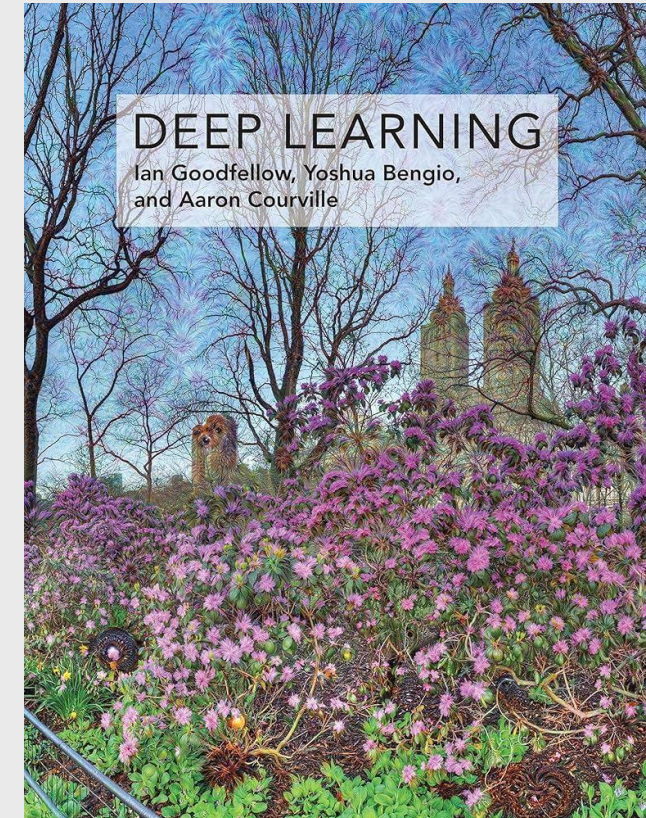
*Drawing of mammalian retina by
Santiago Ramón y Cajal*

$$2^{3^x} - 3 = 5$$

what is x ?

Content

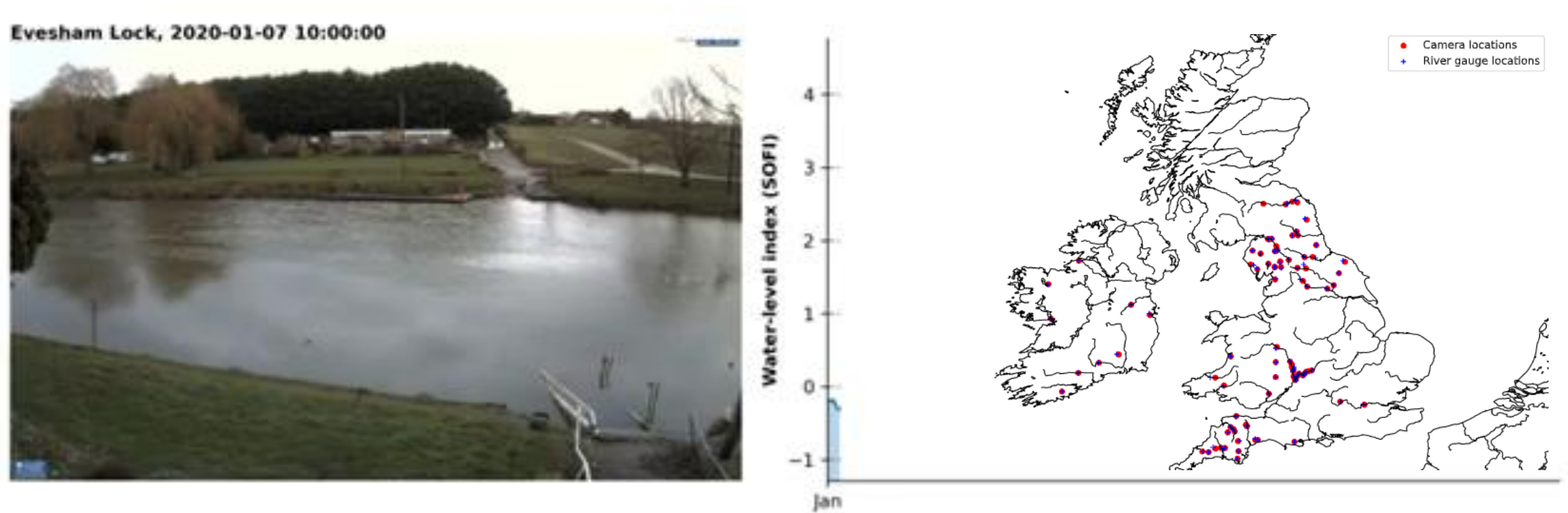
- **Part 1: Introduction / Why CNNs**
- **Part 2: Design of Convolutional Nets**
 - Image Data
 - Components of ConvNets
 - ConvNet Architectures and Challenges
 - Regularisation in ConvNets / DNNs
- **Part 3: Convolutional Neural Network Models**
 - Image Segmentation Models
 - Object Detection Models
 - Generative Models Concept



Goodfellow et al (2017) Deep Learning, MIT Press
<https://www.deeplearningbook.org/>

CNNs for Flood Monitoring

AI and CCTV Cameras across UK help automated tracking and monitoring of flood saturation



AI and CCTV camera help monitor water pollution

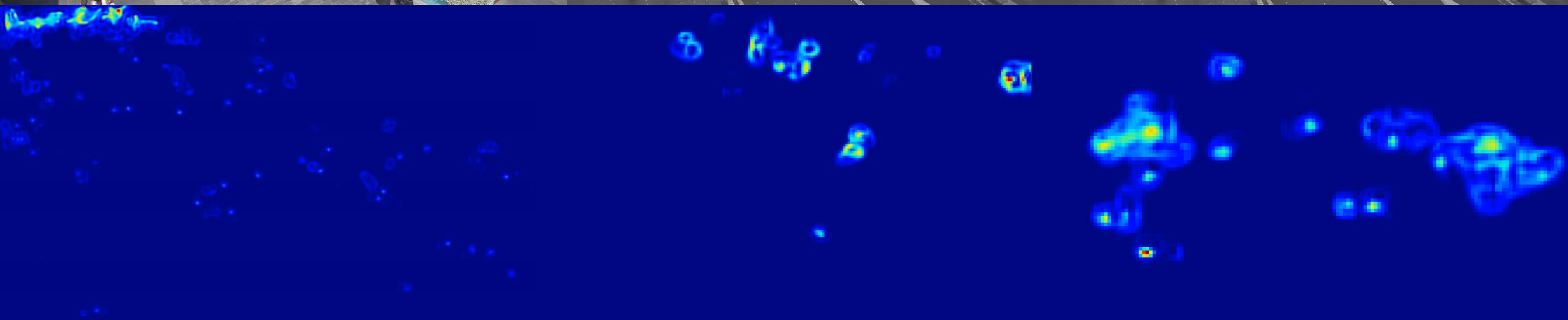
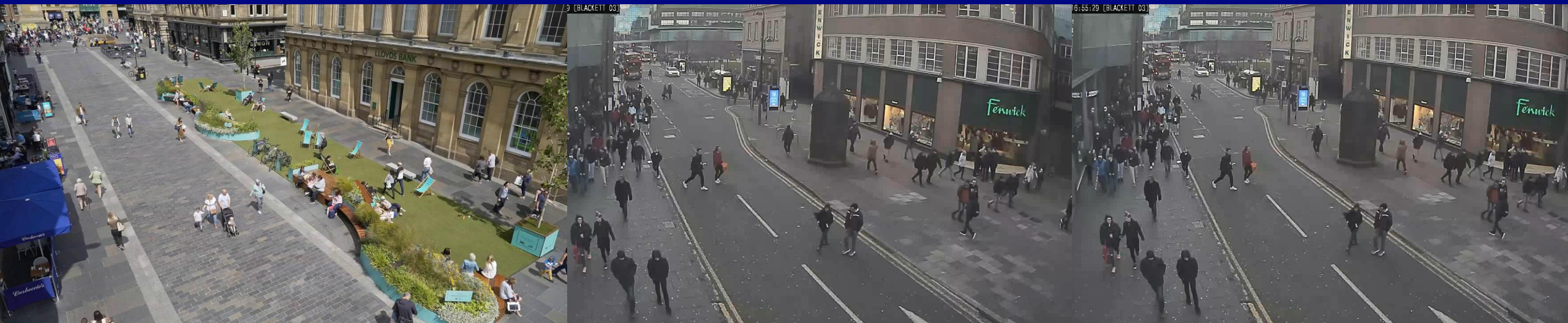
Input Video



Output Video



CNN for Crowd Management in Cities



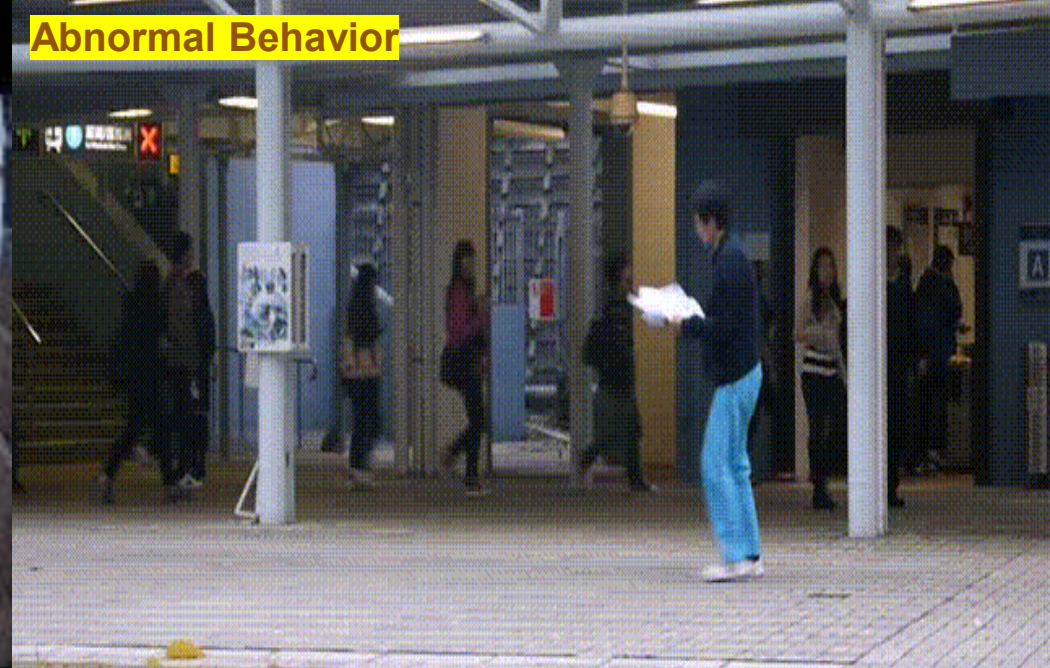
CNN for Crime Monitoring

Assault

Frame No: = 008



Abnormal Behavior



Fighting

Frame No: = 008

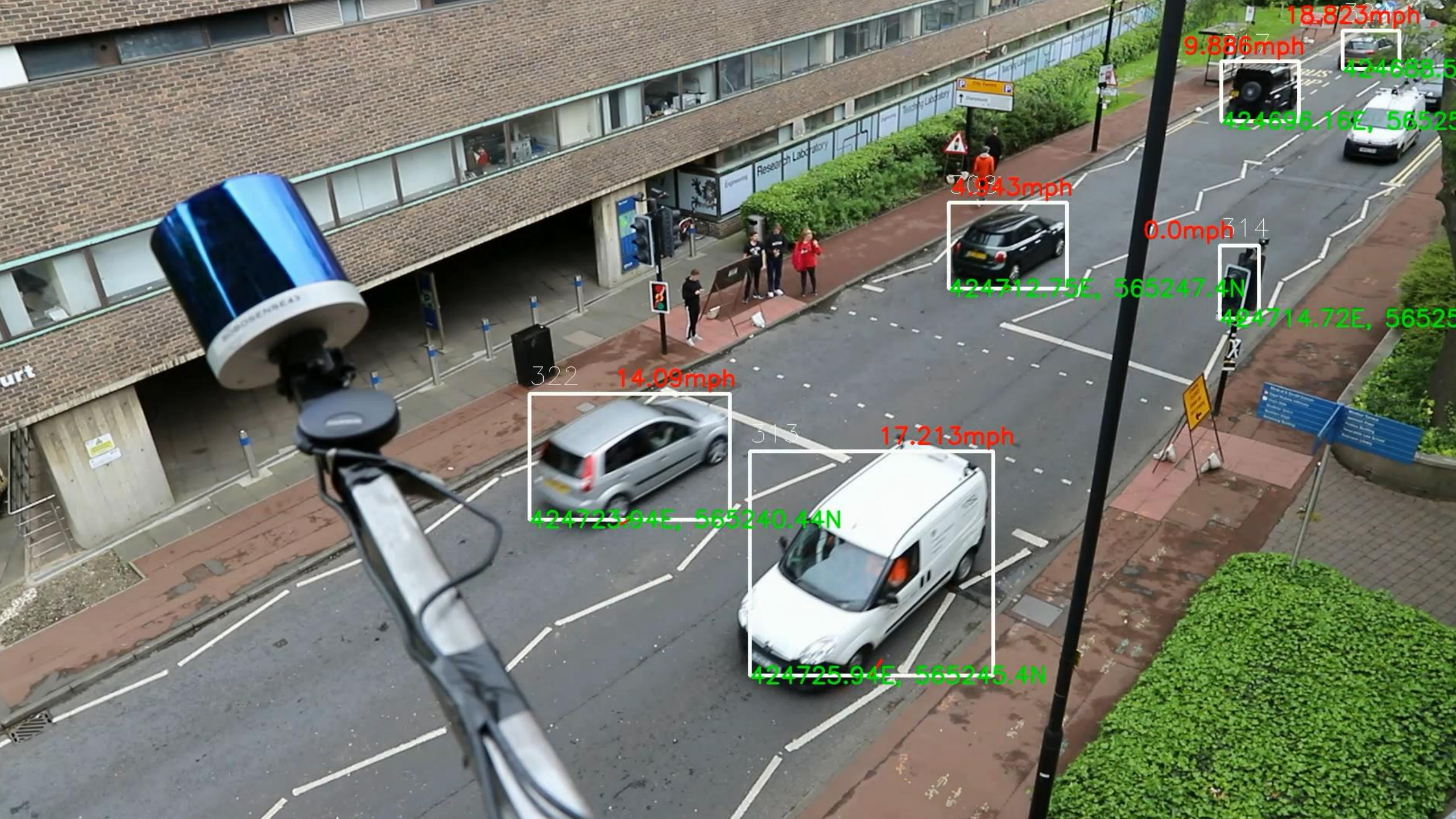


Explosion

Frame No: = 008



ADDR: 1 - V1.64



18.823mph

9.886mph

424688.5

424696.16E, 56525

40.43mph

0.0mph

424712.75E, 565247.4N

424714.72E, 56525

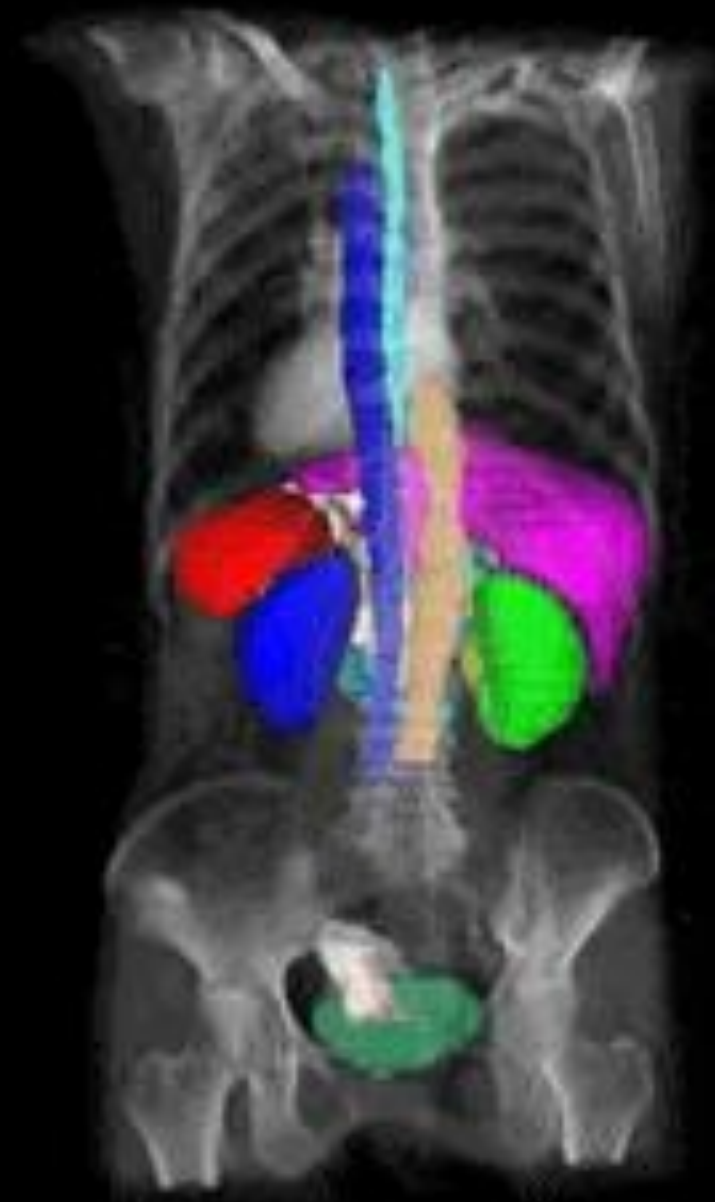
322 14.09mph

313

17.213mph

424723.94E, 565240.44N

424725.94E, 565245.4N



Part 2

Design of Convolutional Nets

Image is only for motivation; it does have some relation to motivating CNN development but not necessary to this lecture and module

9:45 AM

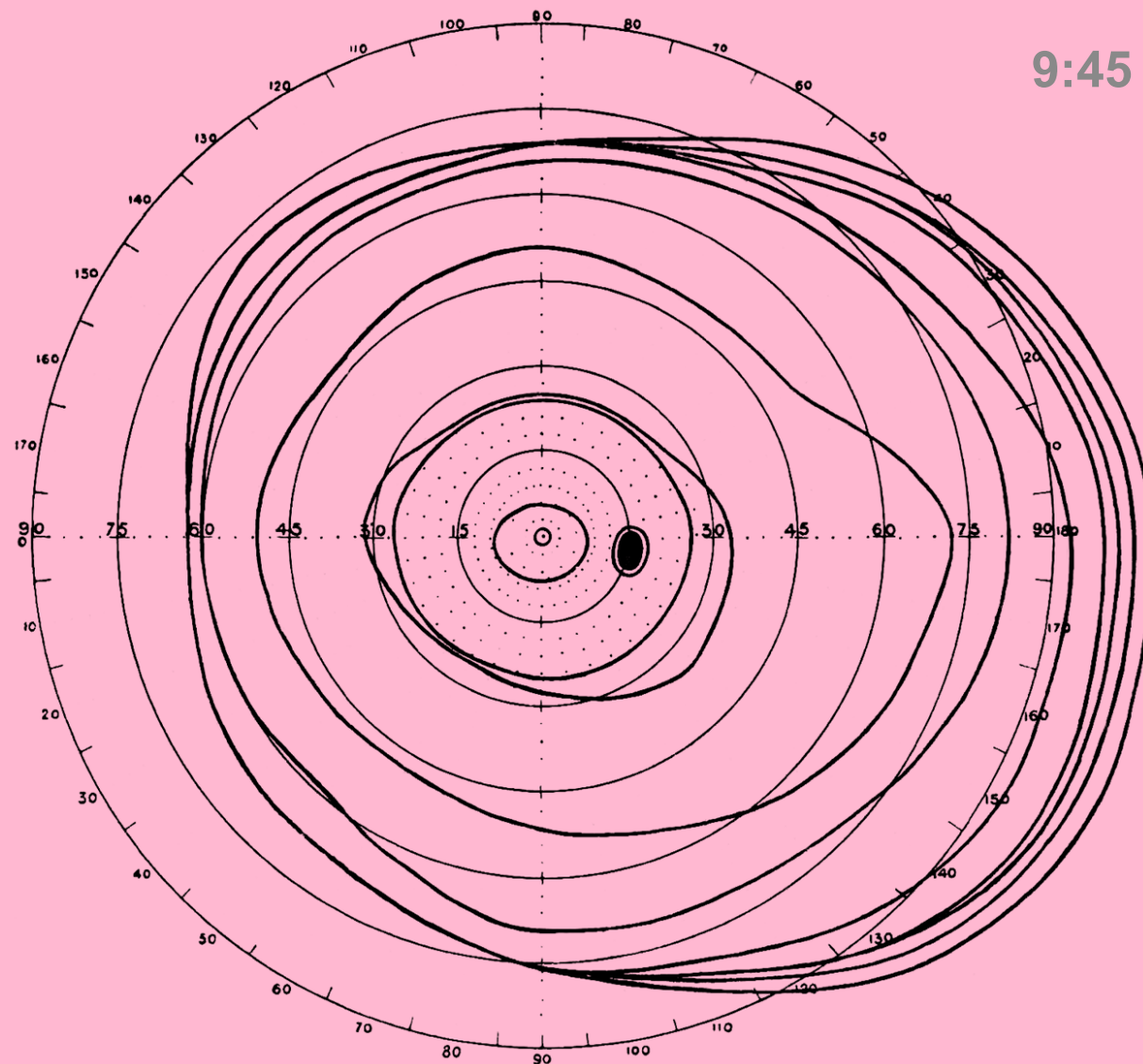


FIG. 1.—CHART OF THE VISUAL FIELD OF THE RIGHT EYE, SHOWING THE ISOPTERS FROM THE PERIPHERY INWARDS TO THE CENTRE FOR $\frac{160}{1000}$, $\frac{80}{1000}$, $\frac{40}{1000}$, $\frac{20}{1000}$, $\frac{10}{1000}$, $\frac{5}{1000}$, $\frac{2.5}{1000}$, $\frac{1.25}{1000}$, $\frac{0.63}{1000}$, AND $\frac{0.31}{1000}$.

The numerator of each fraction represents the diameter of the test object in millimetres, the denominator the distance from the patient. It will be seen that as the test object is reduced in size the field becomes smaller, slowly at the periphery, but rapidly towards the centre. On the nasal side the larger test objects all give the same extent of field, so that the nasal edge of the field is perpendicular, the temporal edge being steeply sloping. The geometrical centre of the field is about 20° to the outer side of the visual axis or physiological centre.

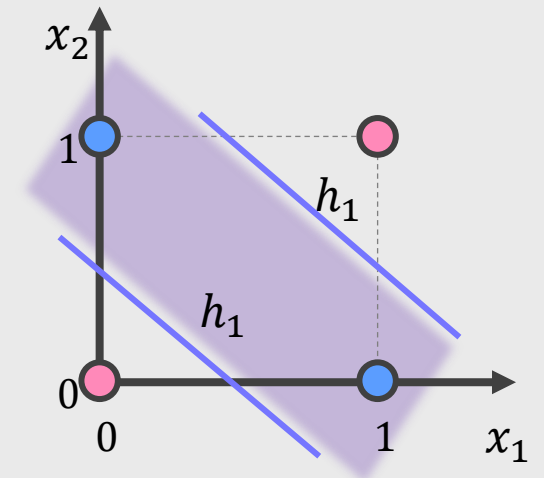
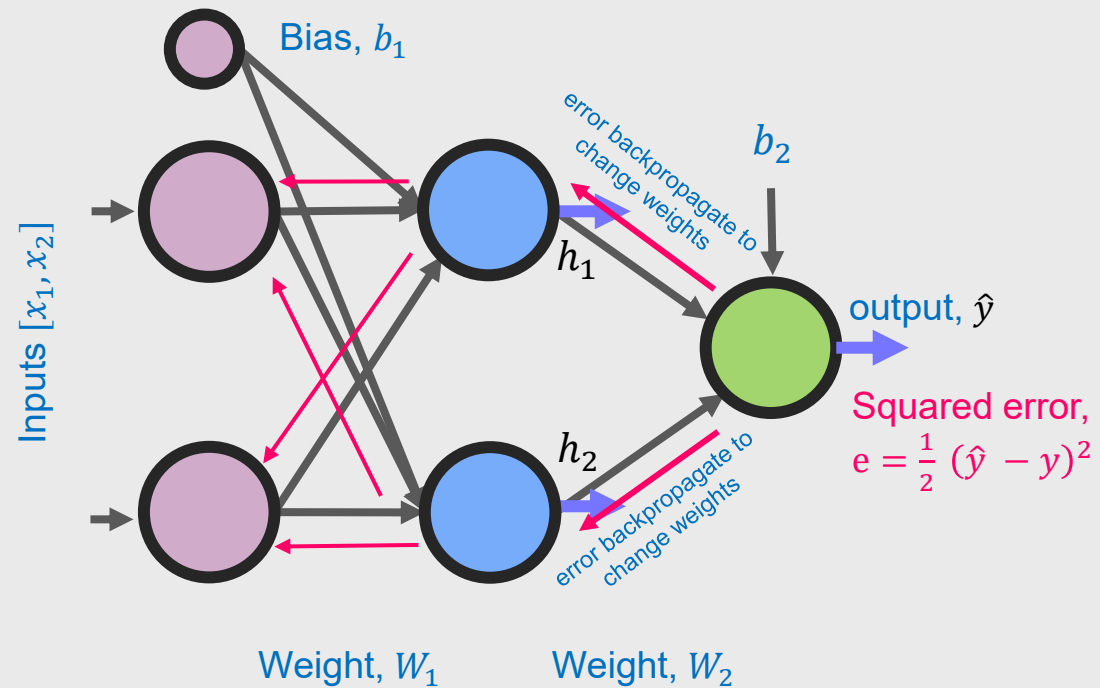
Deep Neural Networks

Error backpropagation can solve non-linearly separable patterns

Full connections between individual nodes/neurons forms matrix multiplication between input/activation and weight matrices

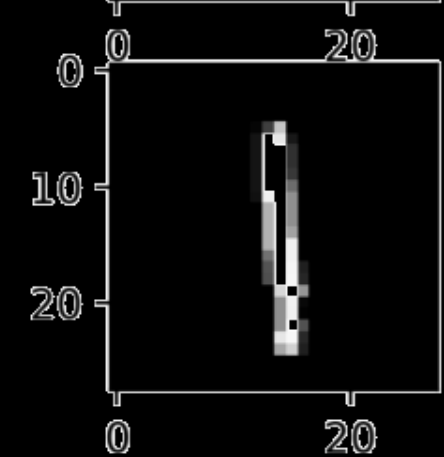
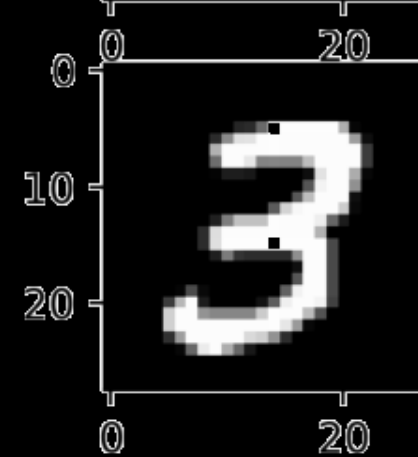
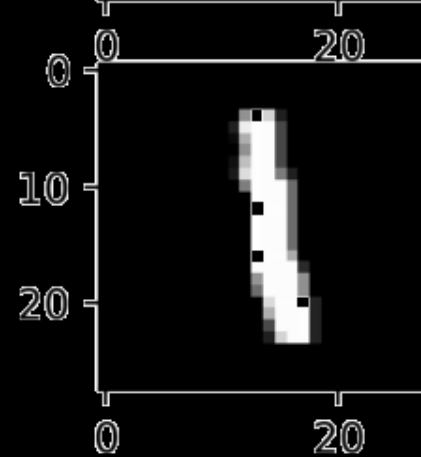
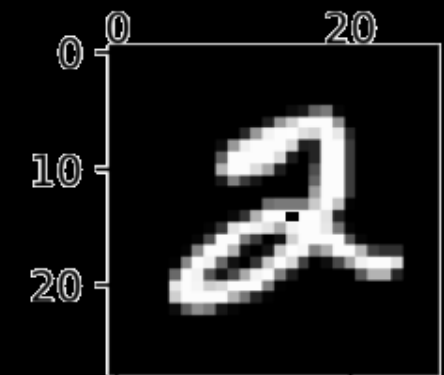
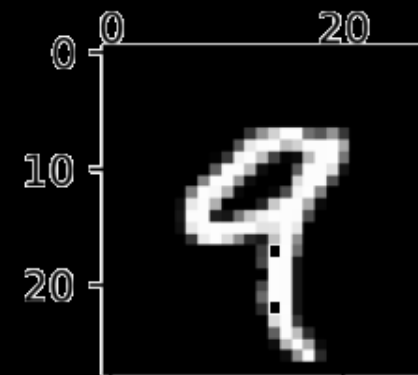
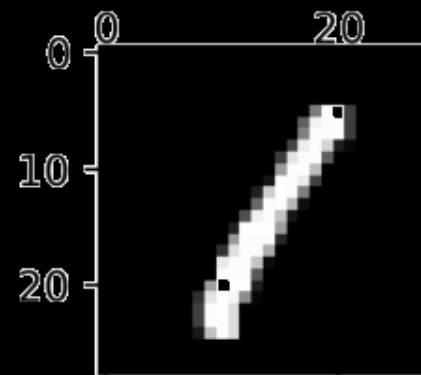
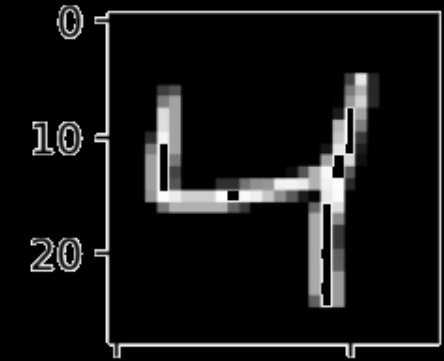
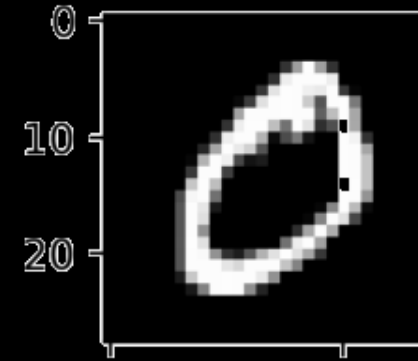
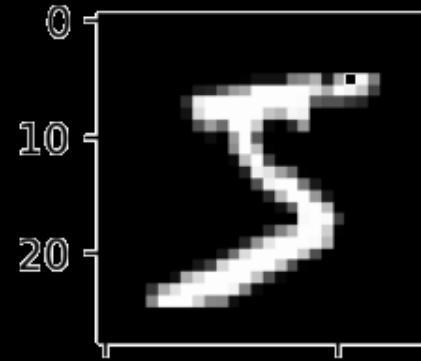
XOR Problem

	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	1
3:	1	0	1
4:	1	1	0



Computer Vision and Pattern Recognition

Dataset is a set of labelled images



Computer Vision

Dataset is a set of labelled images.

Images can more complex images than just character recognition and the task can vary e.g.

classifying one image against other or

recognising objects such rock, water, statue, cloud, sun etc.



Data: 2D

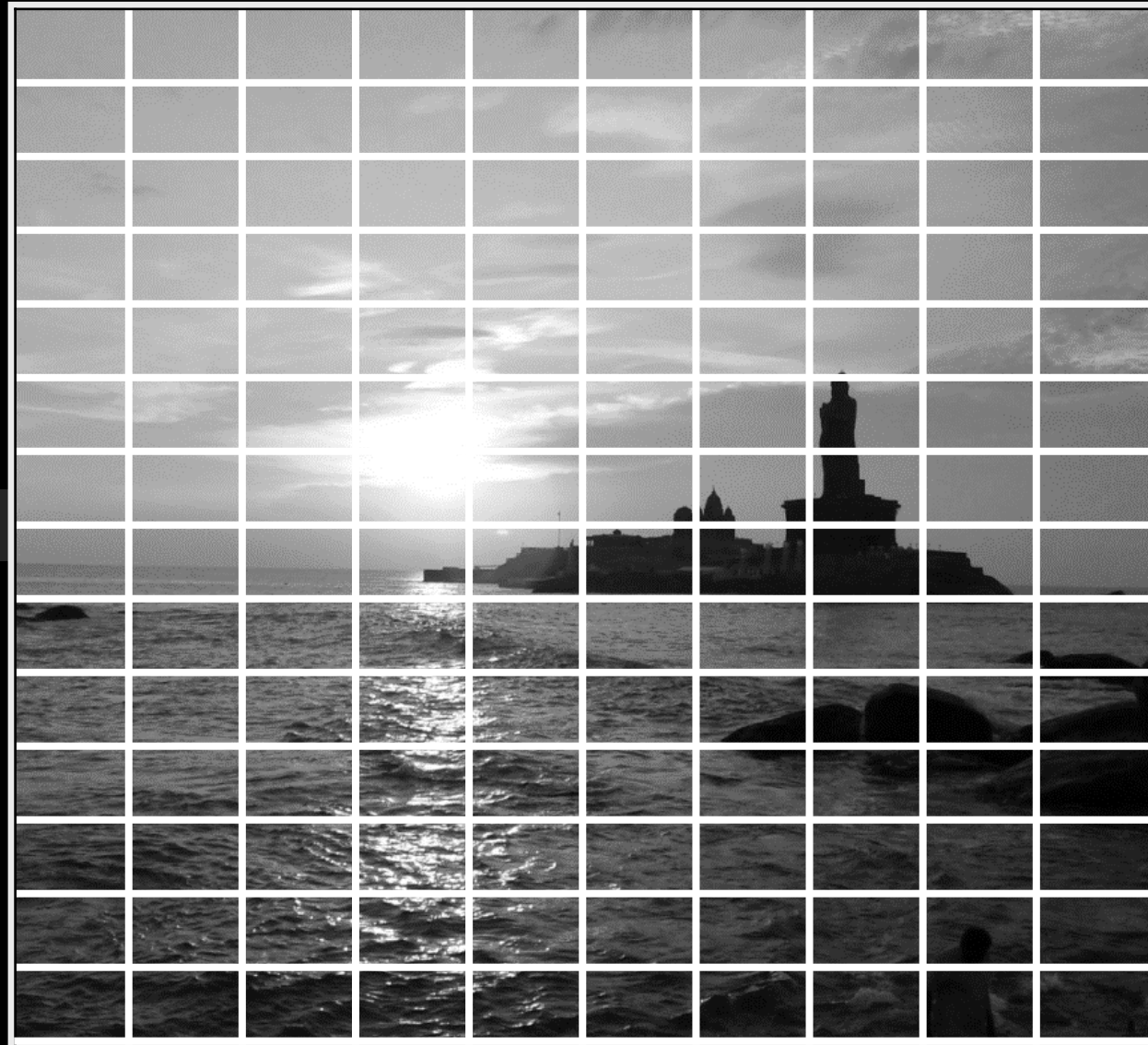
Image can be gray scale

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

For $Height = 256, Width = 256$

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,256} \\ \vdots & \ddots & \vdots \\ p_{256,1} & \cdots & p_{256,256} \end{bmatrix}$$

Height (H)



Width (W)

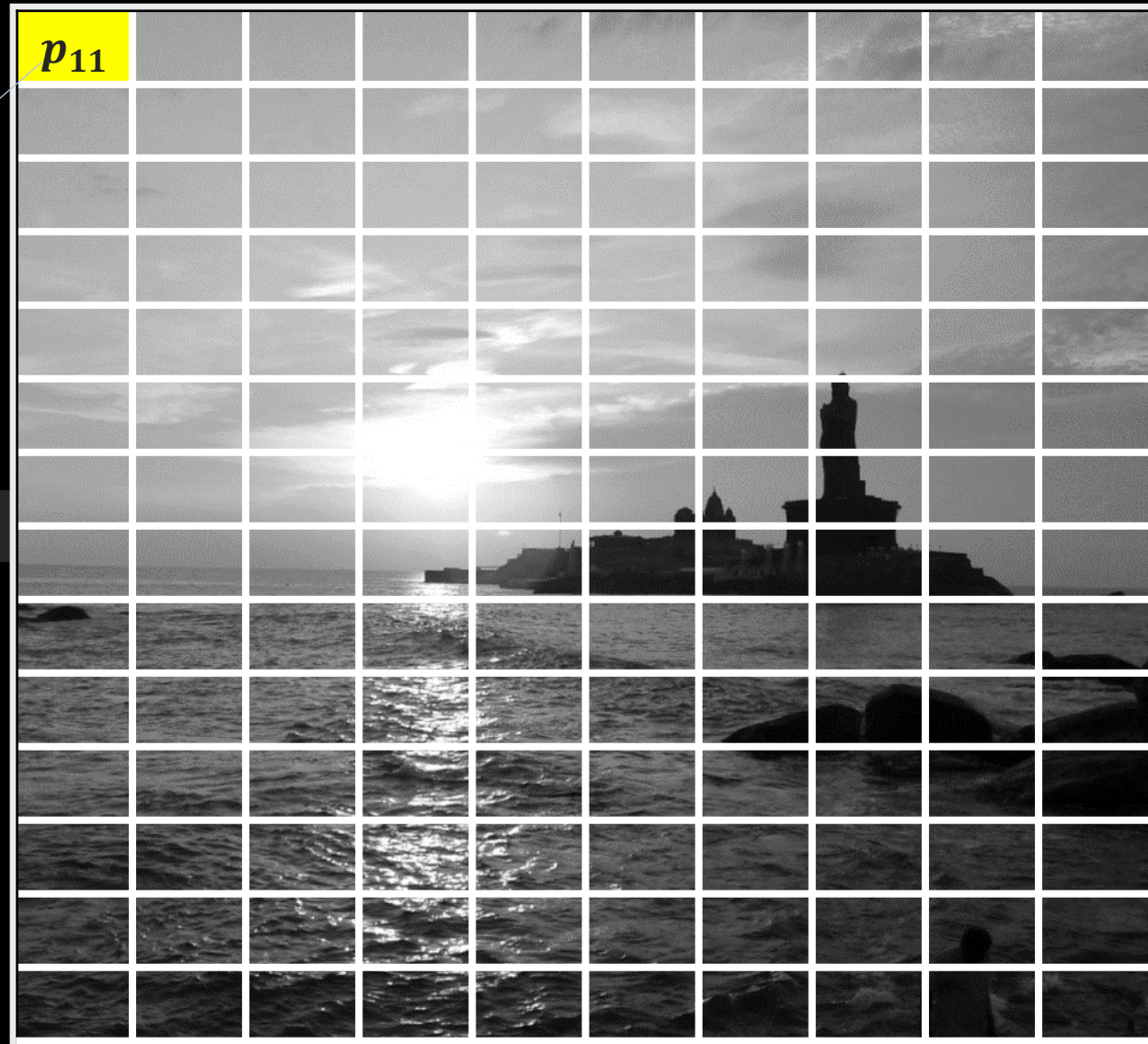
Data: 2D

Image can be gray scale

$$p_{ij} \in \{0, 1, 2, \dots, 255\}$$

For $Height = 256, Width = 256$

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,256} \\ \vdots & \ddots & \vdots \\ p_{256,1} & \cdots & p_{256,256} \end{bmatrix}$$



Height (H)

Width (W)

Data

Image can be color scale

9:45 AM



Data: 3D

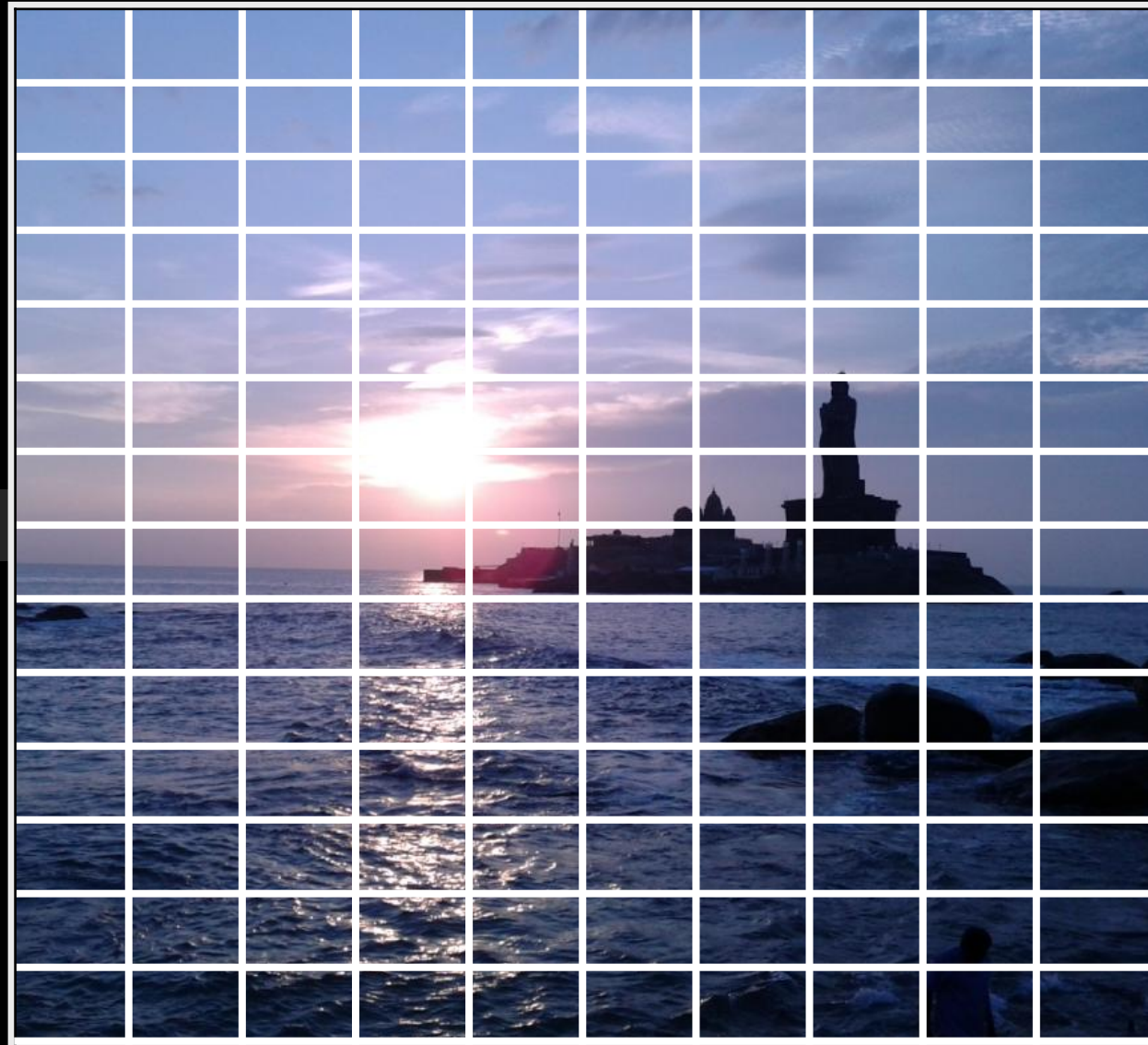
Image can be color scale

$$I_{RED} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

Height (H)

$$I_{Green} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

$$I_{Blue} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$



Width (W)

Data

Image can be color scale

Channel /Depth (D)

9:45 AM

$$I_{RED} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

Height (H)

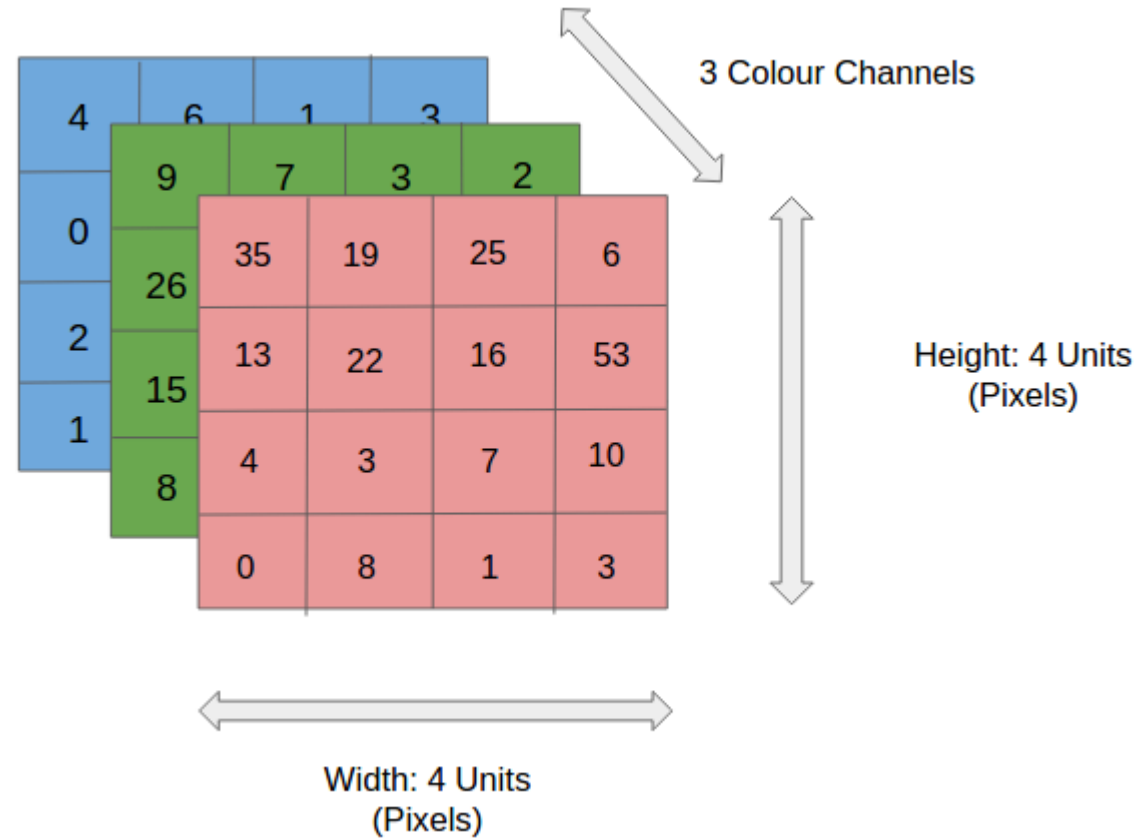
$$I_{Green} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

$$I_{Blue} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

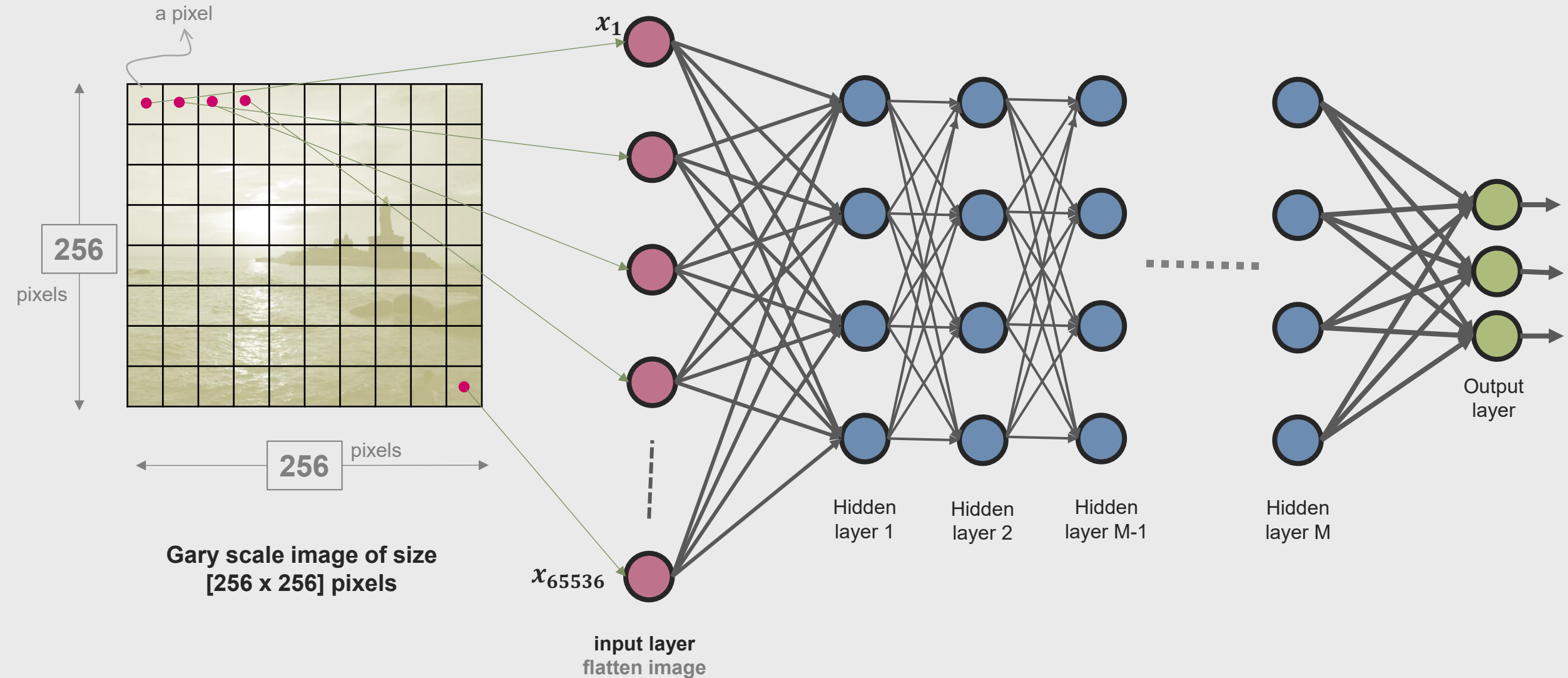


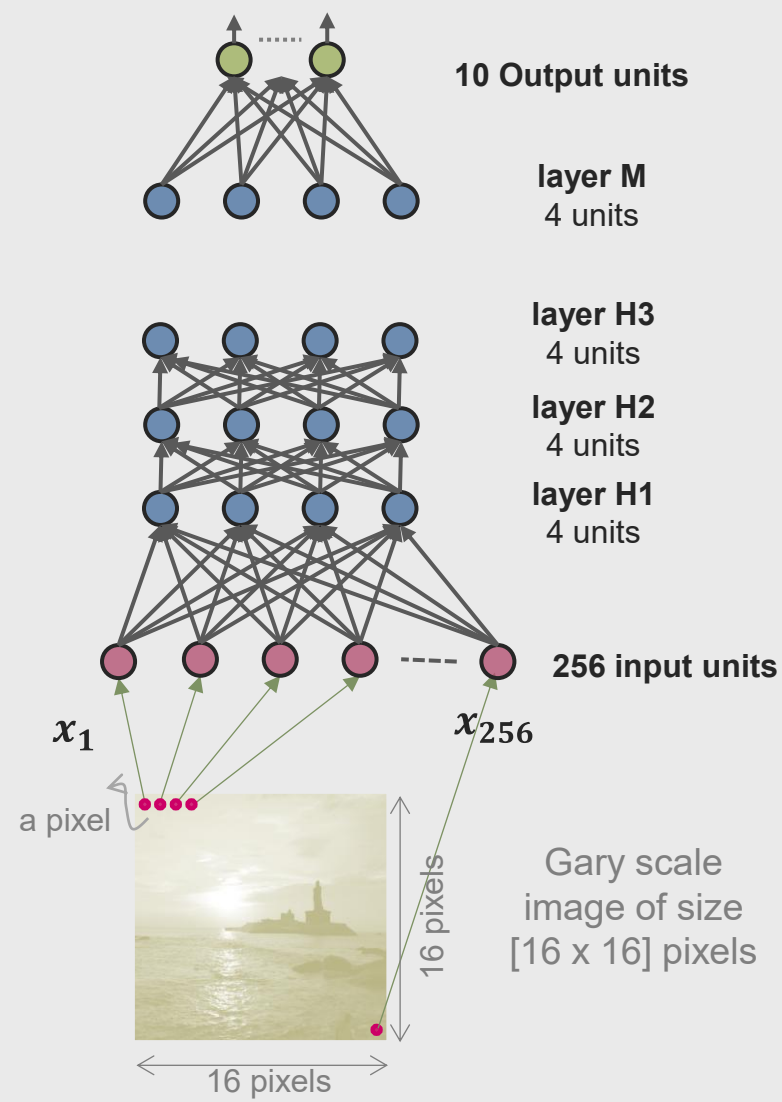
Width (W)

Computer Vision Data

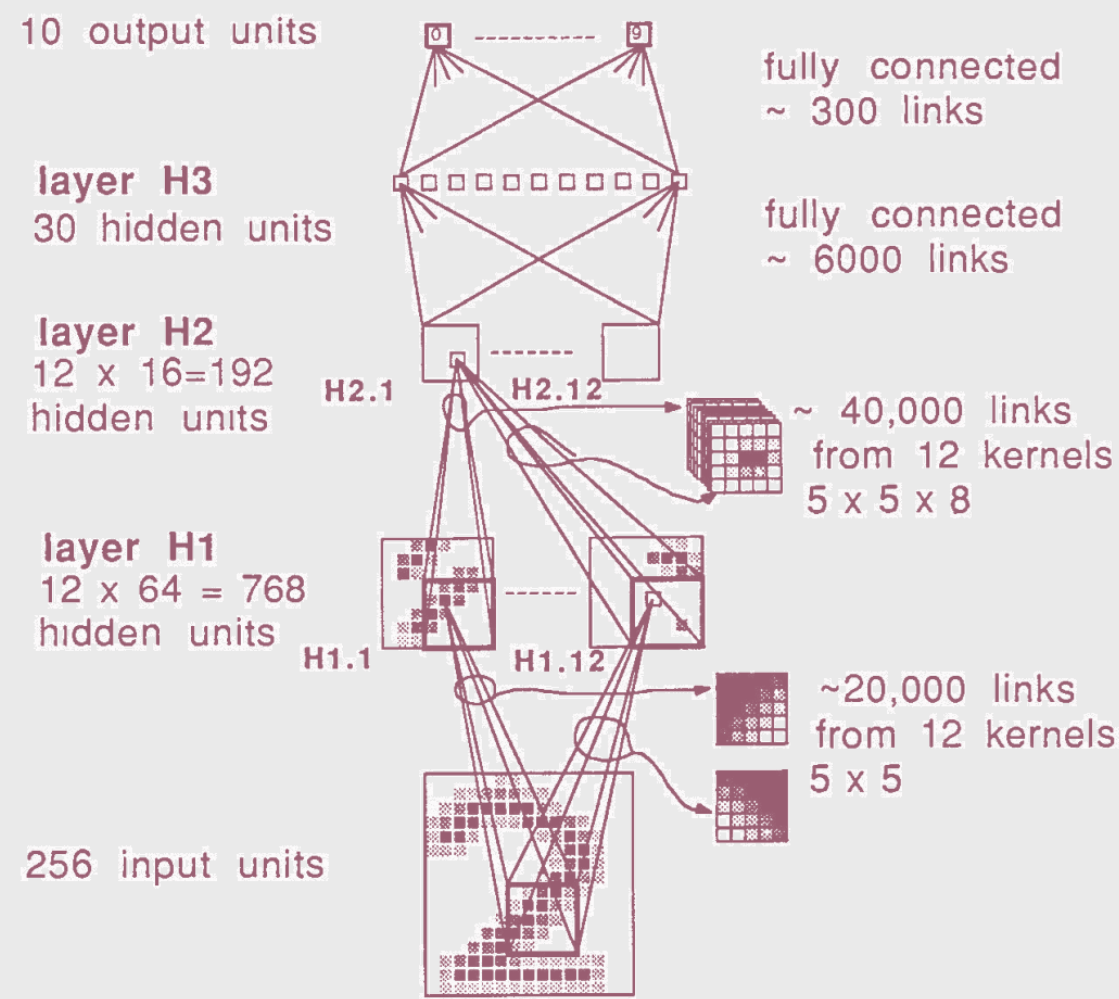


Computer Vision using Deep Neural Networks





Deep Neural Network (DNN)

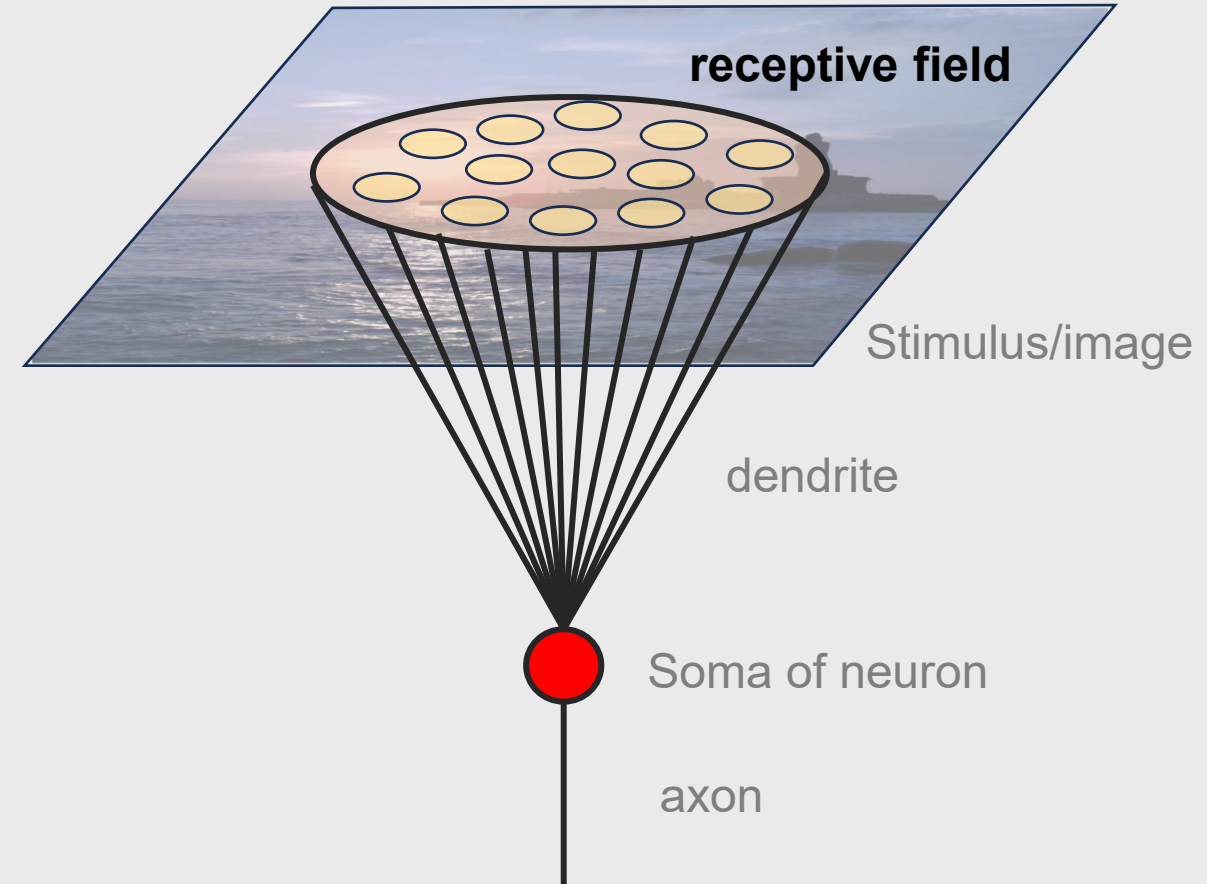
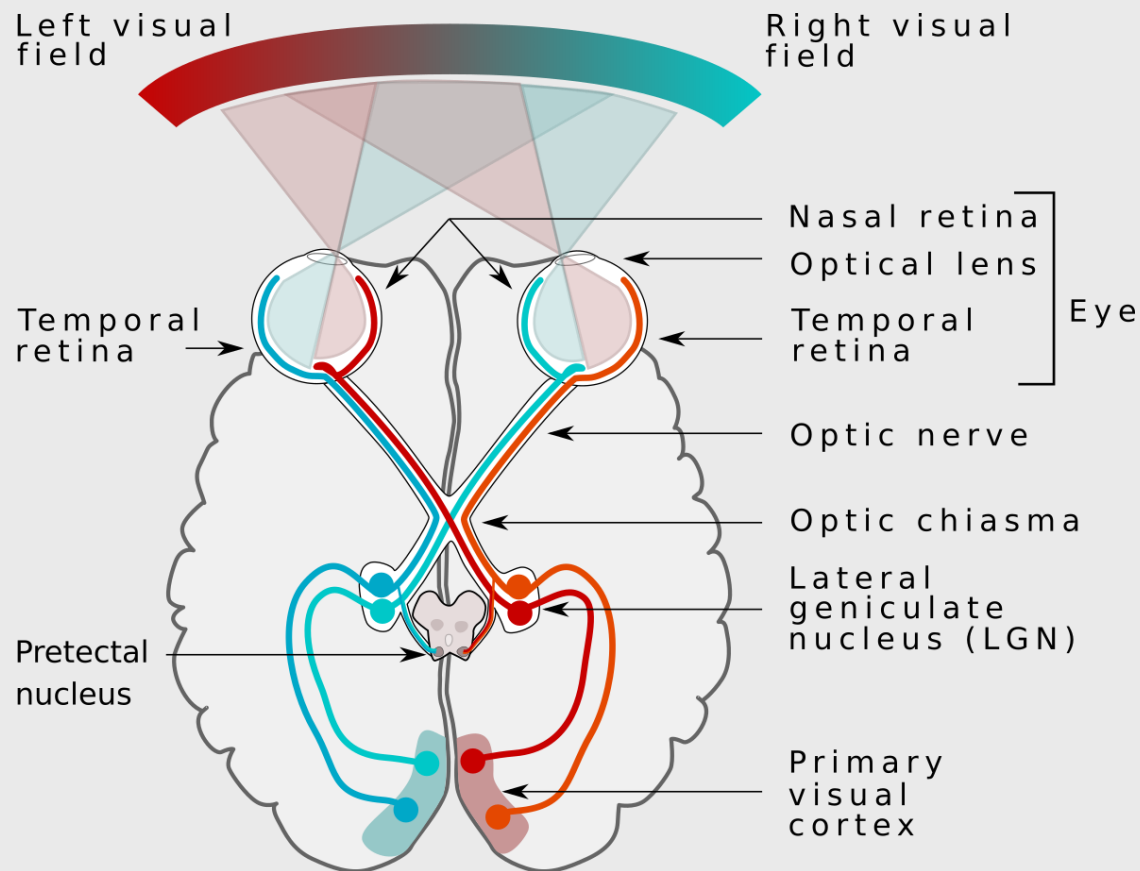


Convolutional Neural Network (CNN)

Receptive Field

9:45 AM

The receptive field (RF) of a neuron (biological neuron) refers to the attributes of a visual stimulus that generates a response in that cell. Thompson et al (2017). Activity-dependent development of visual receptive fields. Current Opinion Neurobiology



Receptive Field Analogy in Convolutional Nets 9:45 AM

Convolutional operation with depth, stride, and padding is an operation to processes the visual stimuli. Each convolutional neuron processes data only for its receptive field.

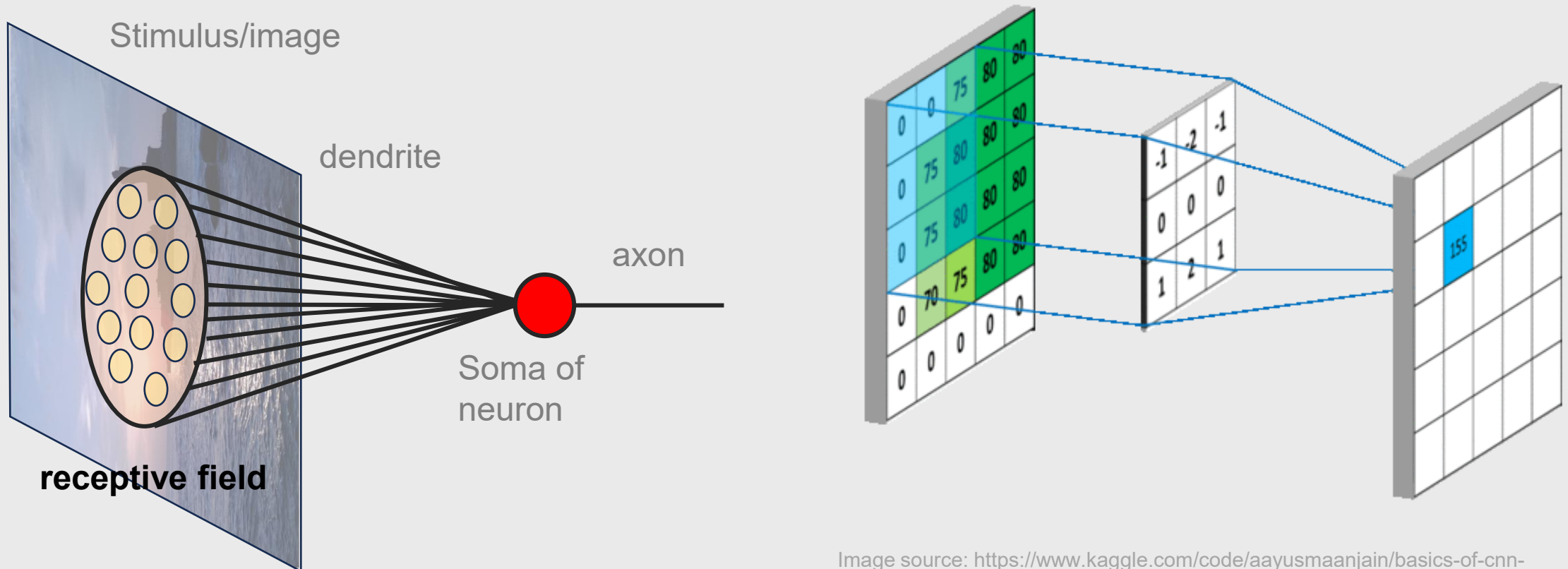
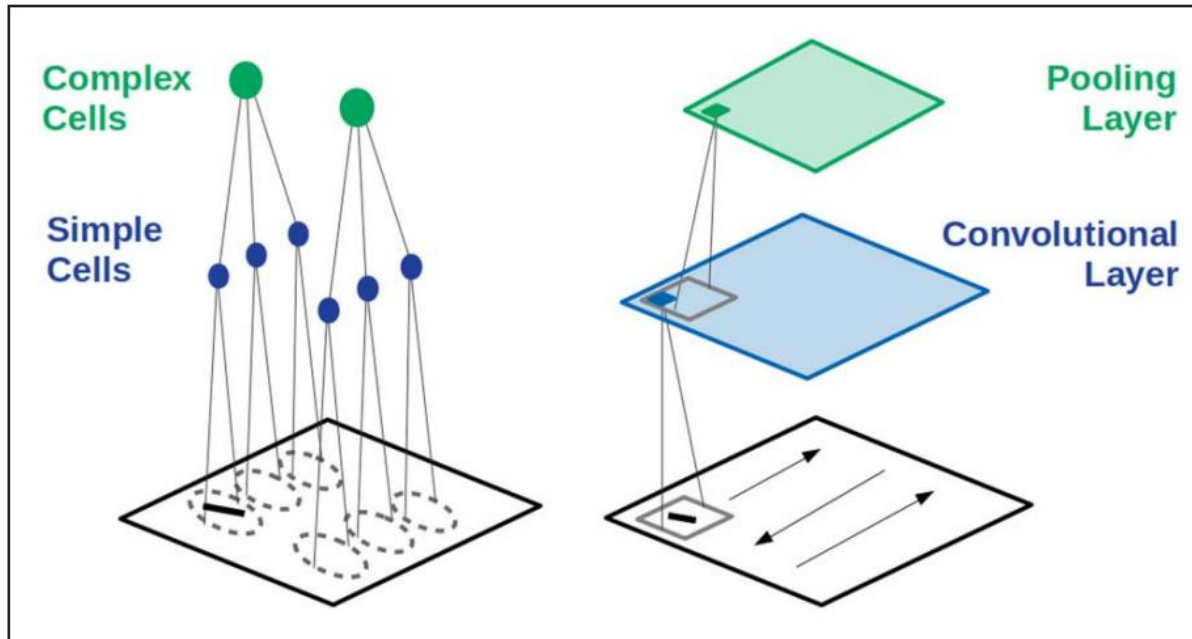


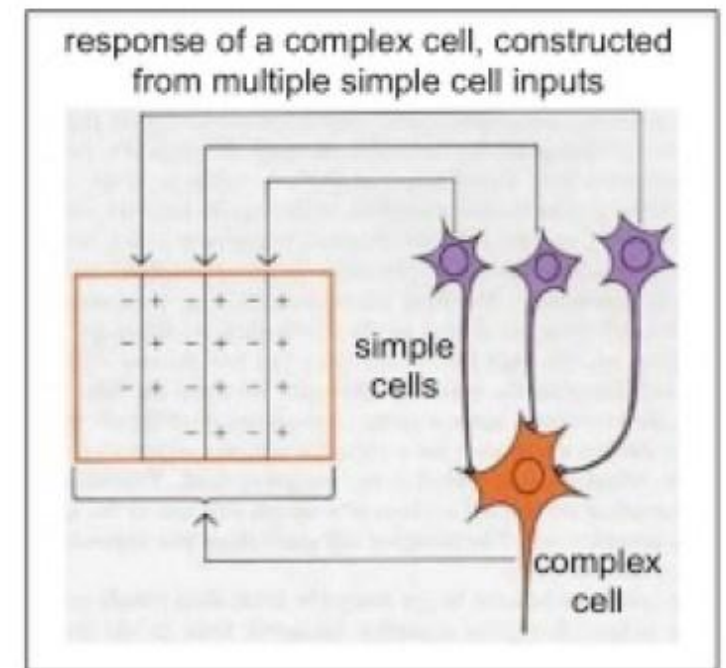
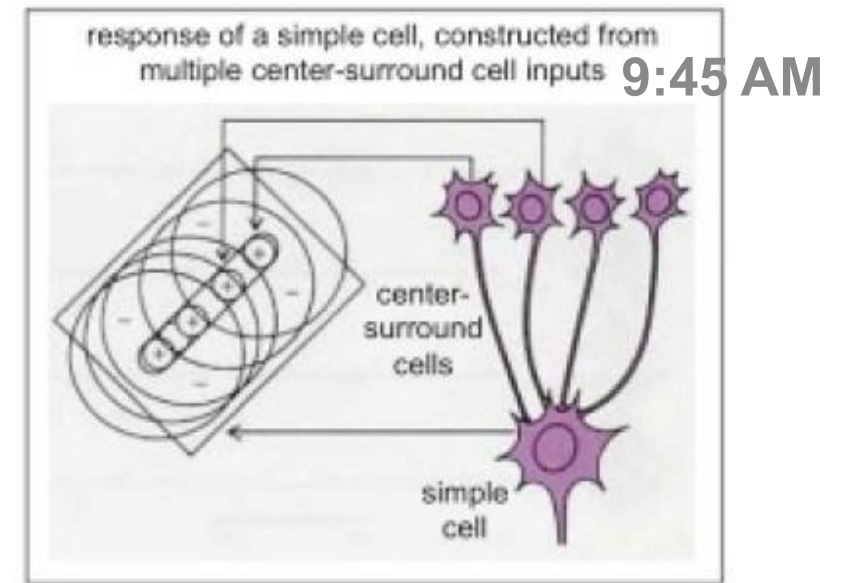
Image source: <https://www.kaggle.com/code/aayusmaanjan/basics-of-cnn-with-pytorch-lightning>

Biological Inspiration of Convolutional Neural Network



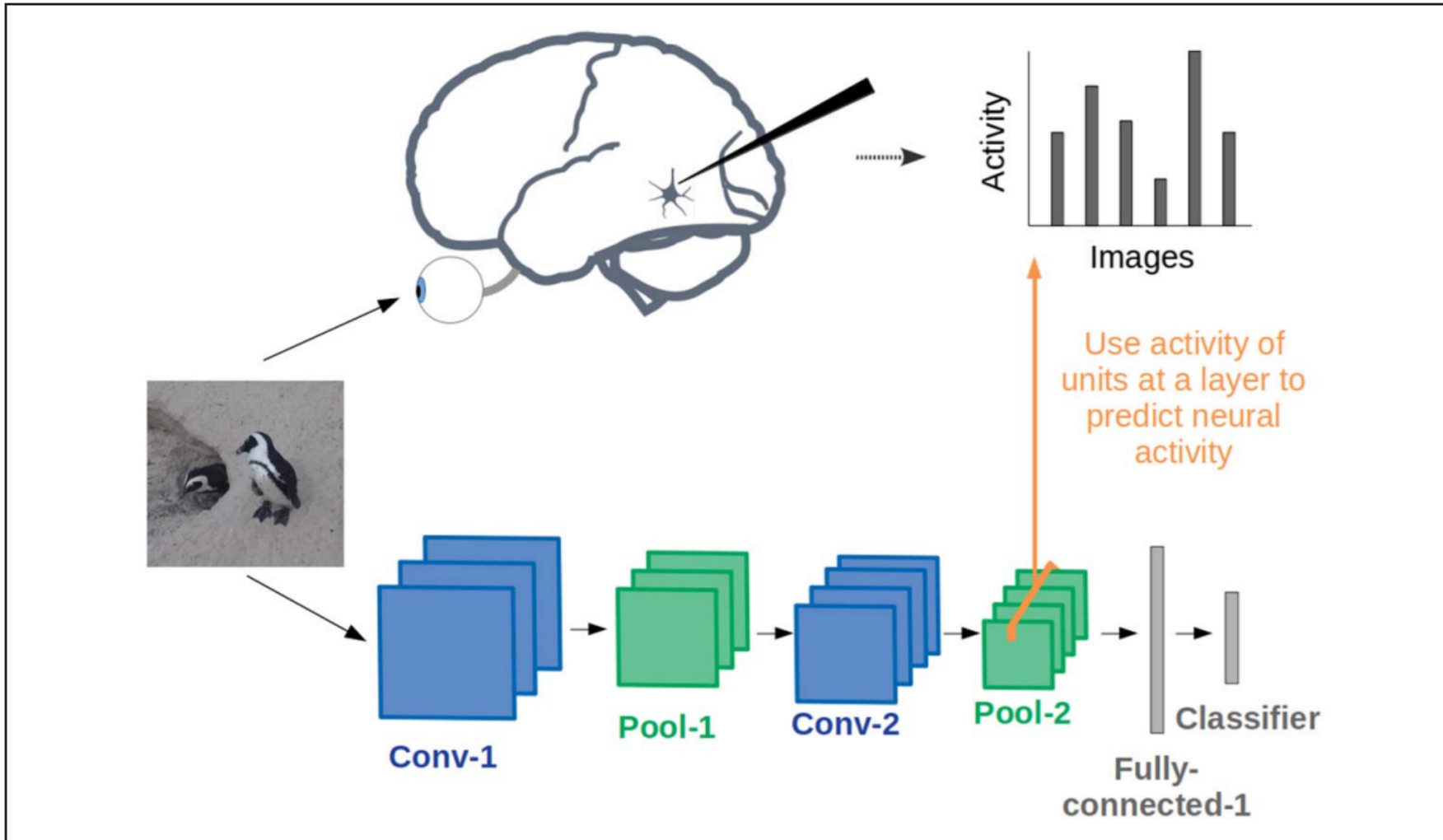
Lindsay (2019) Journal of Cognitive Neuroscience

Hubel and Wiesel (1962) discovered that **Simple cells (blue)** have preferred locations in the image (dashed ovals) wherein they respond most strongly to bars of particular orientation. **Complex cells (green)** receive input from many simple cells and thus have more **spatially invariant responses**. These operations are replicated in a CNN (right).



CNN and an Animal response to an Image

9:45 AM



Modern CNNs start by convolving a set of filters with an input image and rectifying the outputs, leading to **“feature maps” akin to the planes of S-cells** in the Noncognition. Max pooling is then applied, creating complex cell-like responses. After several iterations of this pattern, non-convolutional fully connected layers are added, and the last layer contains as many units as number of categories in the task to output a category label for the image

Convolutional Nets are Spatial Invariant

9:45 AM

Spatial invariance is the property of a system to produce the same output even if the input is shifted, moved, rotated, or translated.



(a) translation



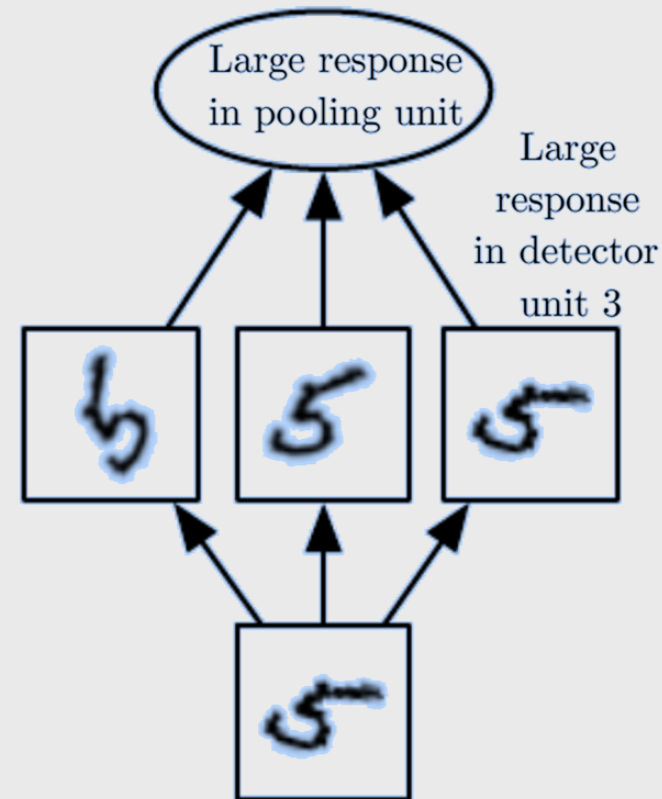
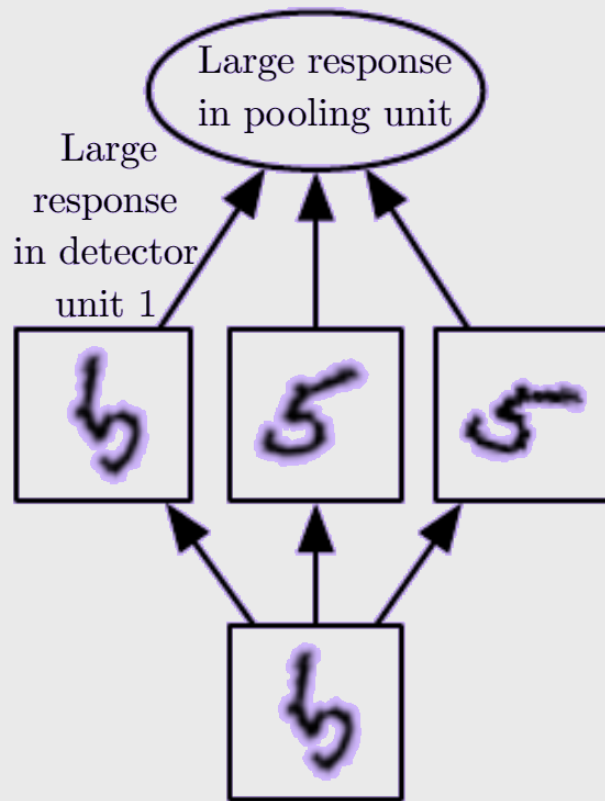
(b) scale/aspect ratio



(c) scale/rotation

Convolutional Nets are Translation Invariant

CNNs can recognize a feature even if it appears in different locations in an image or images are rotated, translated, shifted.

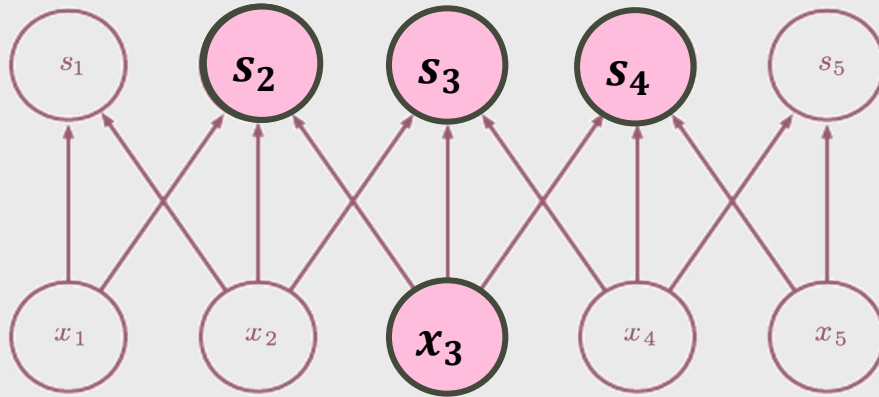


Sparsity

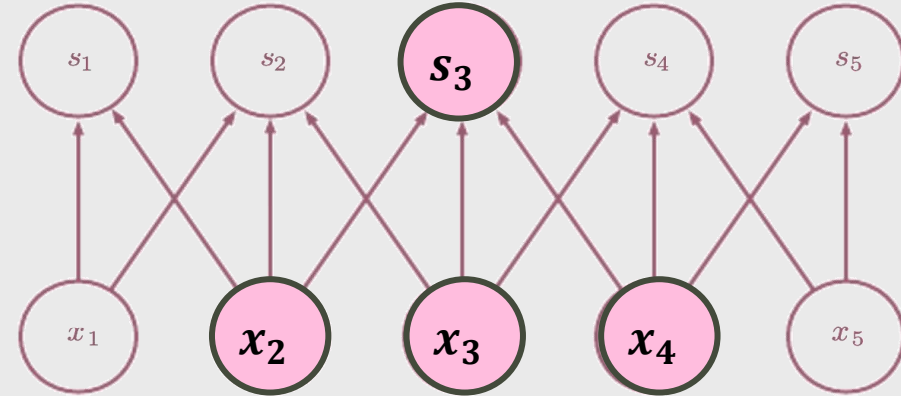
9:45 AM

Only three outputs are affected by x
 S is formed by convolution with a
kernel of width 3

Convolution

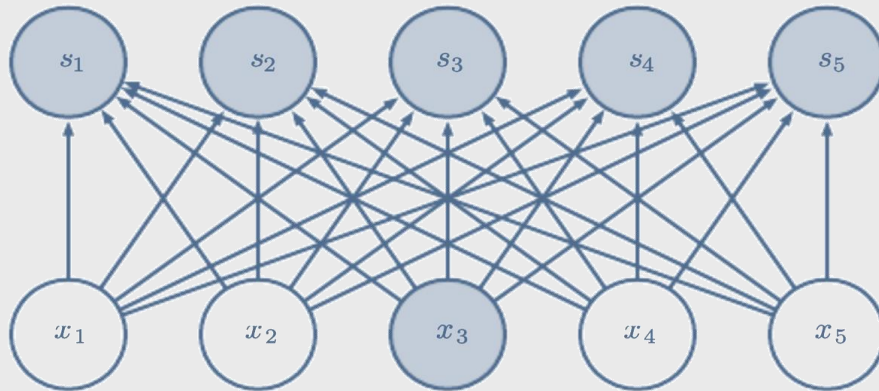


Only three input affect s_3
 S is formed by convolution with a kernel
of width 3 (receptive field of s_3)

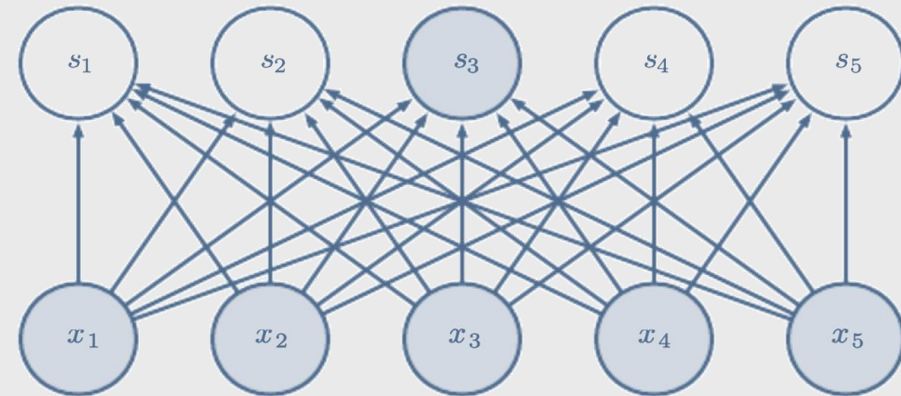


Memory and
Time efficiency

Fully Connected



All the output affect s_3



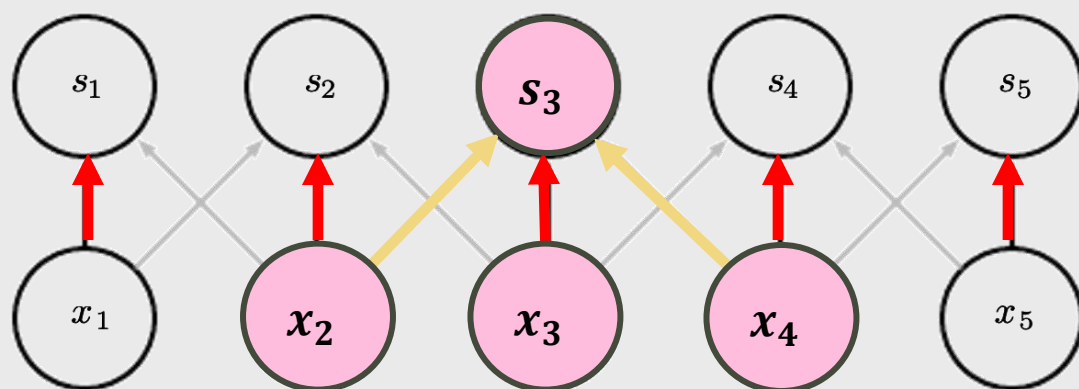
All the inputs affect s_3

Memory and
Time intensive

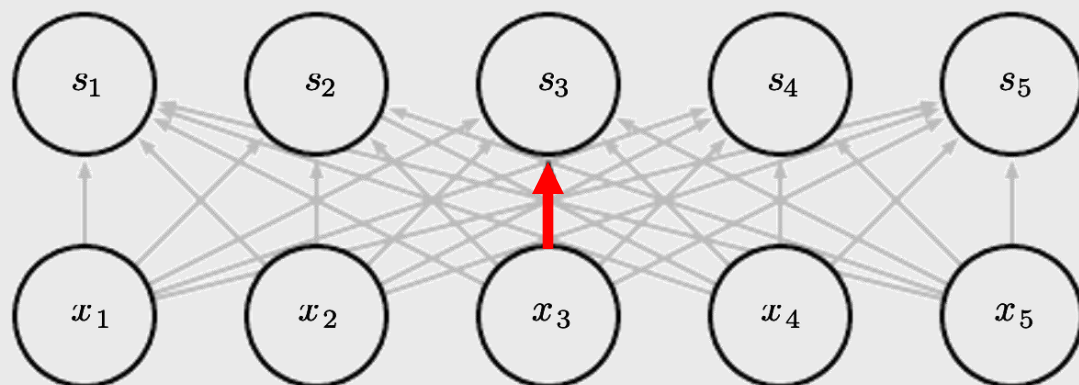
Parameter Sharing (↗) and Tied weights (↑)

9:45 AM

Parameter sharing refers to using the same parameter for more than one function in a model. In CNNs, each filter (a small weight matrix) has a set of weights (parameters) associated with it, and these weights are shared across all the neurons that use that filter

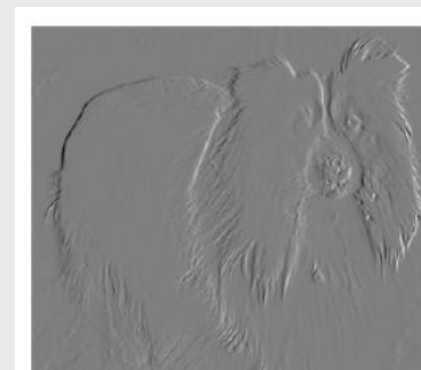


The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model. Because of parameter sharing, this single parameter is used at all input locations



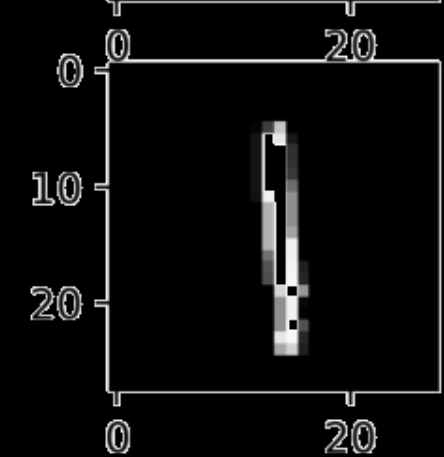
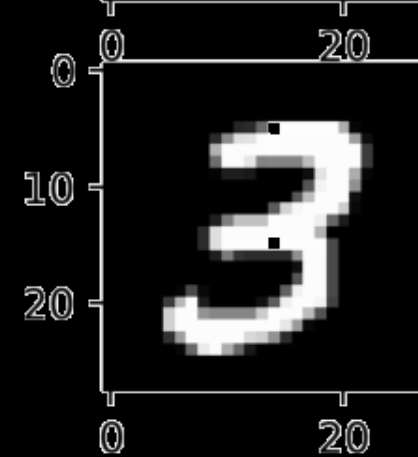
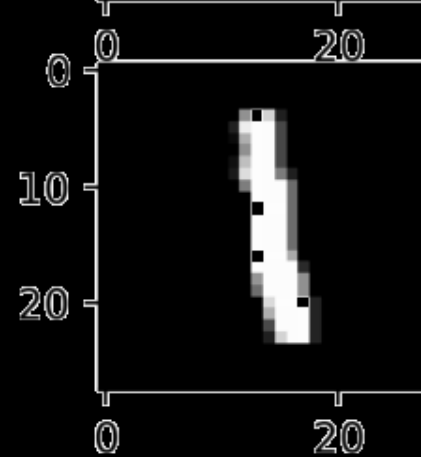
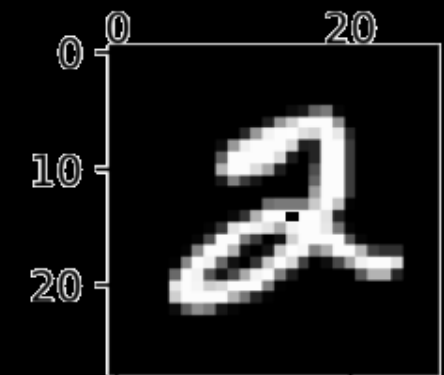
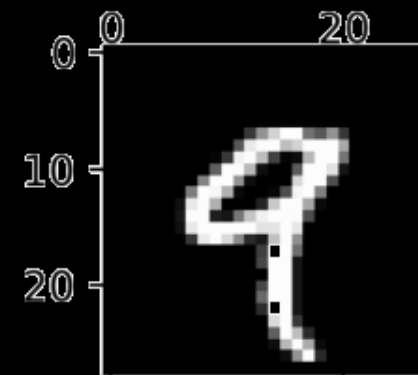
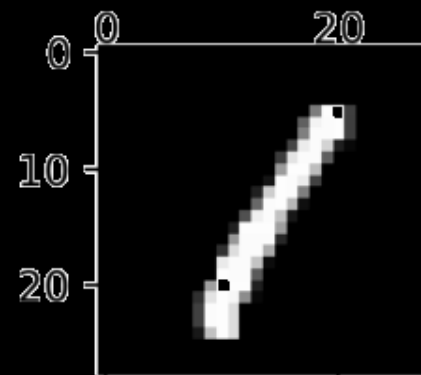
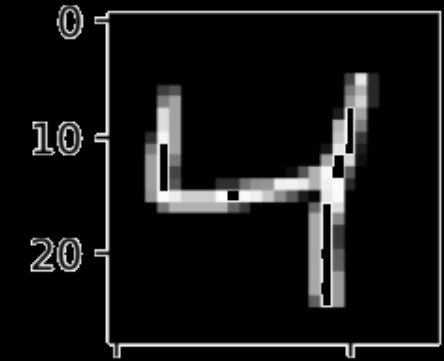
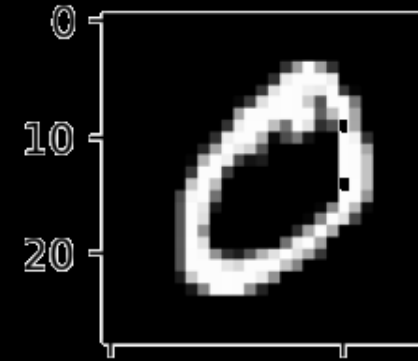
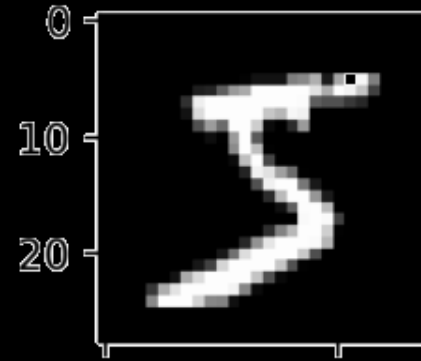
The single black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing, so the parameter is used only once

Edge Detection



Computer Vision and Pattern Recognition

Dataset is a set of labelled images



Communicated by Dana Ballard

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel

AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

Neural Computation 1, 541-551 (1989) © 1989 Massachusetts Institute of Technology

80322-4129 80206

40004 14310

37872 05453

3302 75216

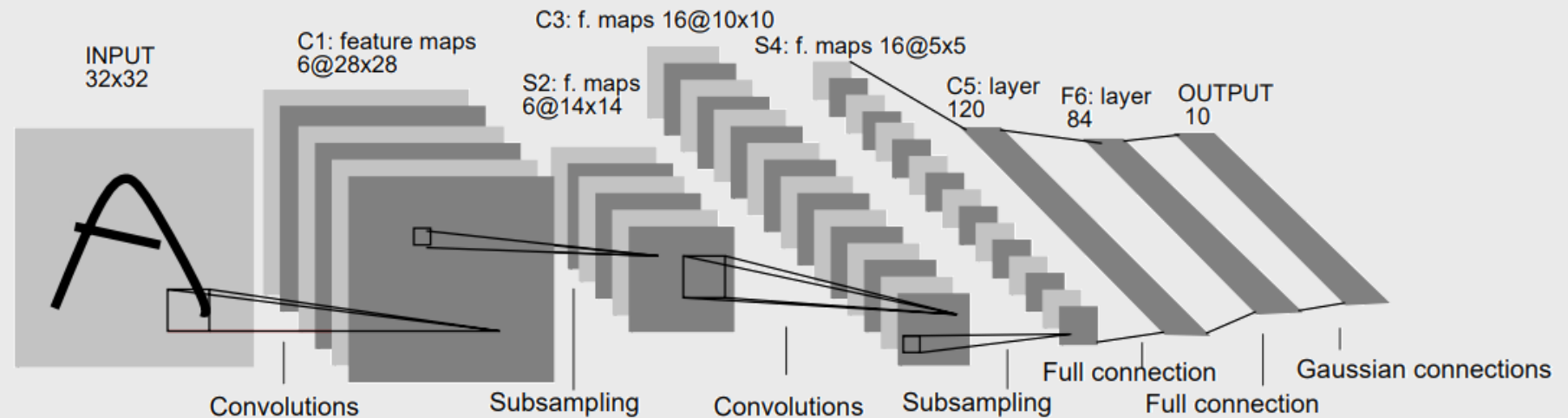
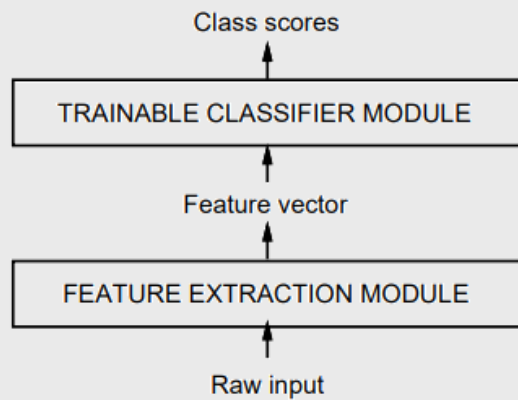
35460 44209

1011915485726803226414186
6359720299299722510046701
3084114591010615406103631
1064111030475262009979966
8912056708557131427955460
6018750187112993089970984
0109707597331972015519055
1075318255182814358090943
1787521655460354603546055
18255108503047520439401

Figure 1 Examples of original zip codes (top) and normalized digits from the testing set (bottom).

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner



Traditional pattern
recognition.

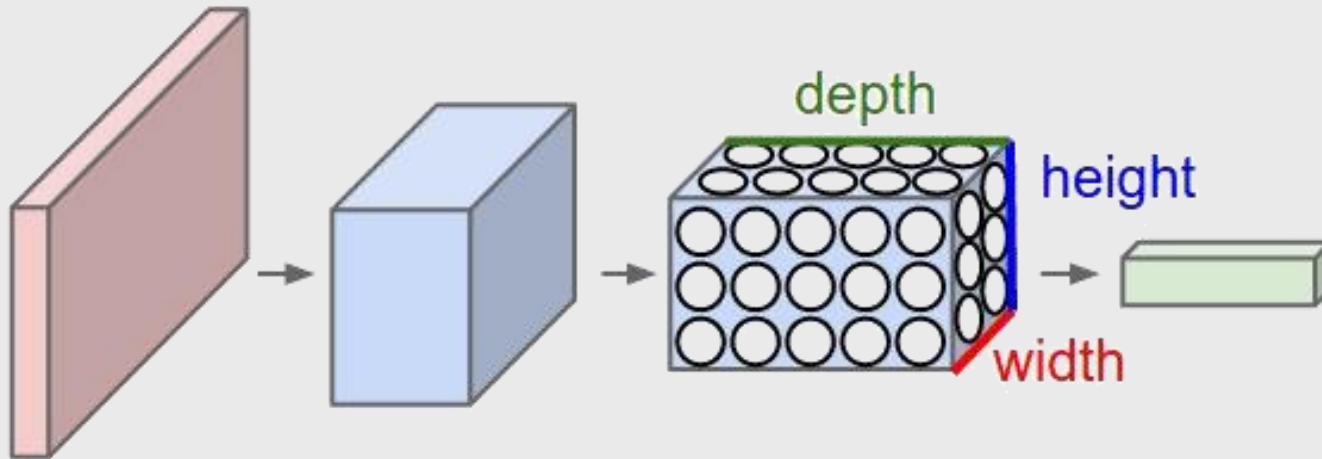
It requires manual feature
extraction steps

Architecture of **LeNet**

a Convolutional Neural Network

CNN are End-to-End model eliminating need for
manual feature extraction

Convolutional Neural Network (ConvNet) 9:45 AM



A **ConvNet** arranges its neurons in three dimensions (**width, height, depth**).

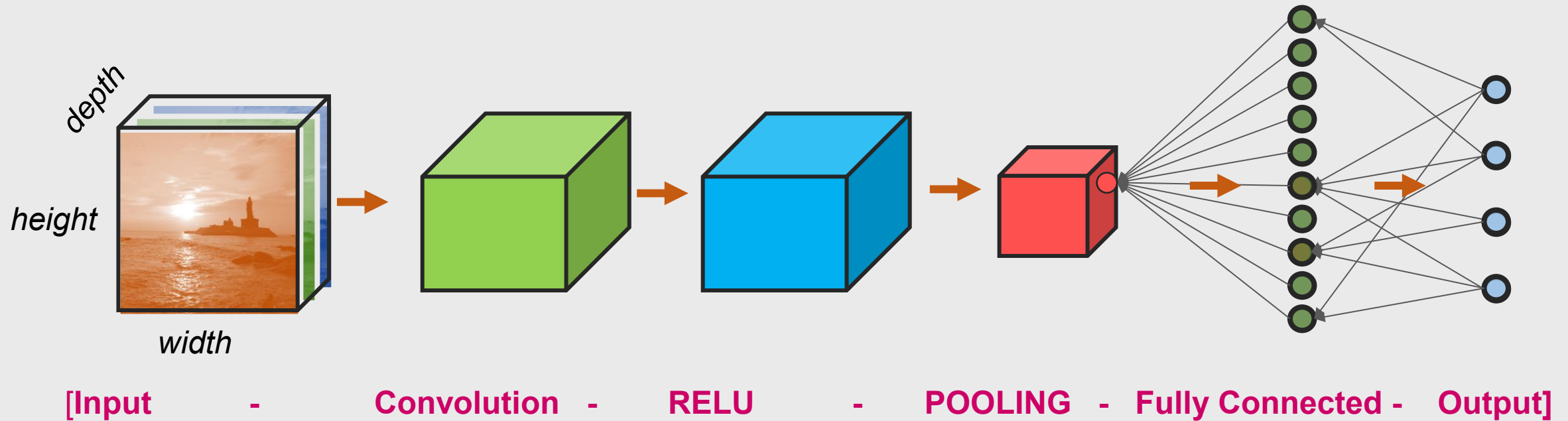
Every layer of a ConvNet transforms the **3D input volume** to a **3D output volume** of neuron activations.

In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels)

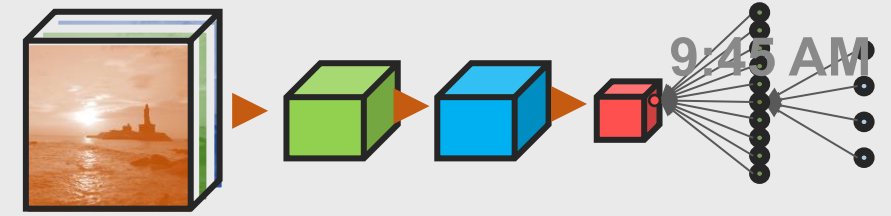
ConvNet/ CNN

Architecture: A Simple ConvNet / CNN

[INPUT - CONV - RELU - POOL - FC]



ConvNet/ CNN

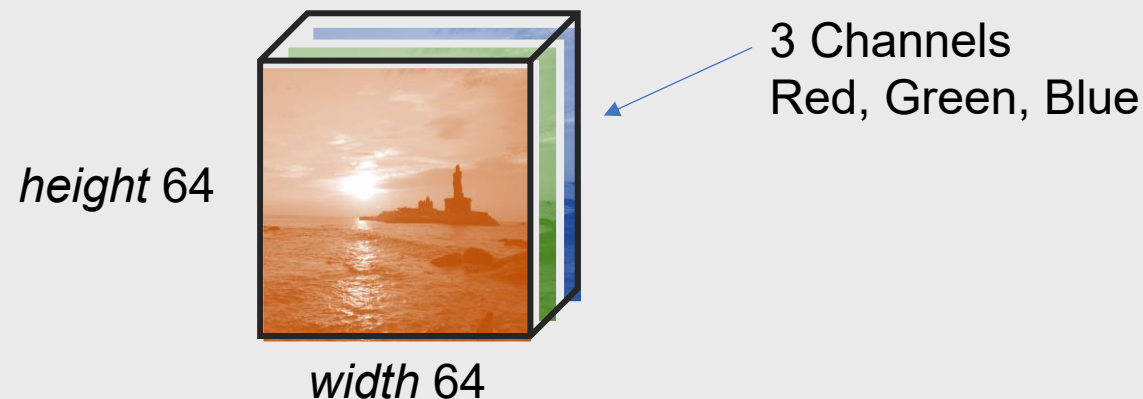


Architecture: A Simple ConvNet / CNN

[**INPUT** - CONV - RELU - POOL - FC]

INPUT [64x64x3] holds the raw pixel values of the image.

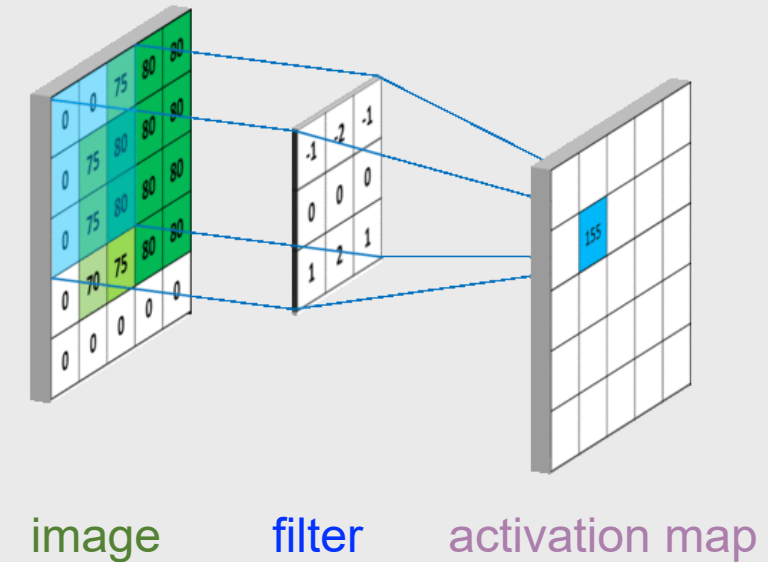
Image *width* 64, *height* 64, and with *three* colour channels R,G,B.



ConvNet (read)

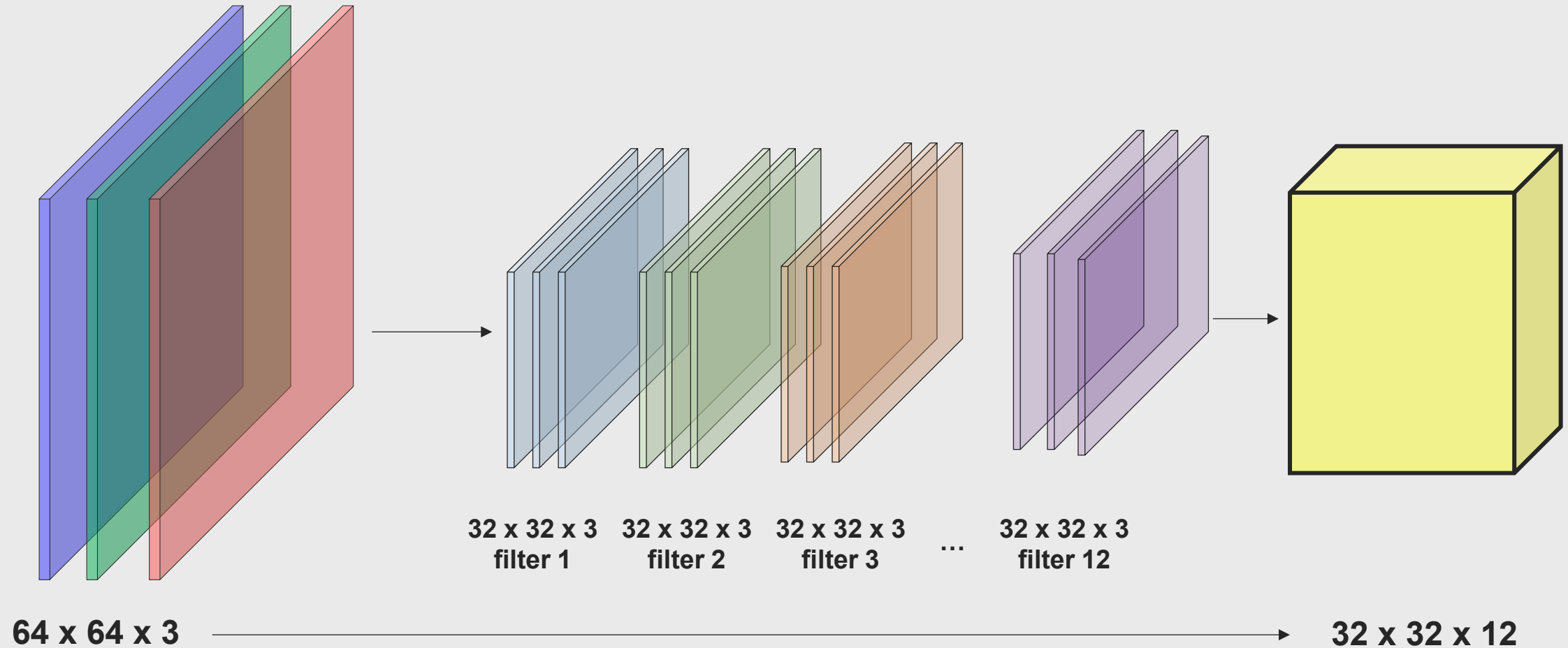
Convolution Layer

- **Forward pass:** we slide (**convolve**) each filter across the width and height of the input volume and compute Hadamard products between the entries of the filter and the input at any position.
- When we slide the **filter/kernel** over the width and height of the input volume, we will produce a **2-dimensional activation map** that gives the responses of that filter at every spatial position
- We can have a set of filters (e.g., 12)



ConvNet

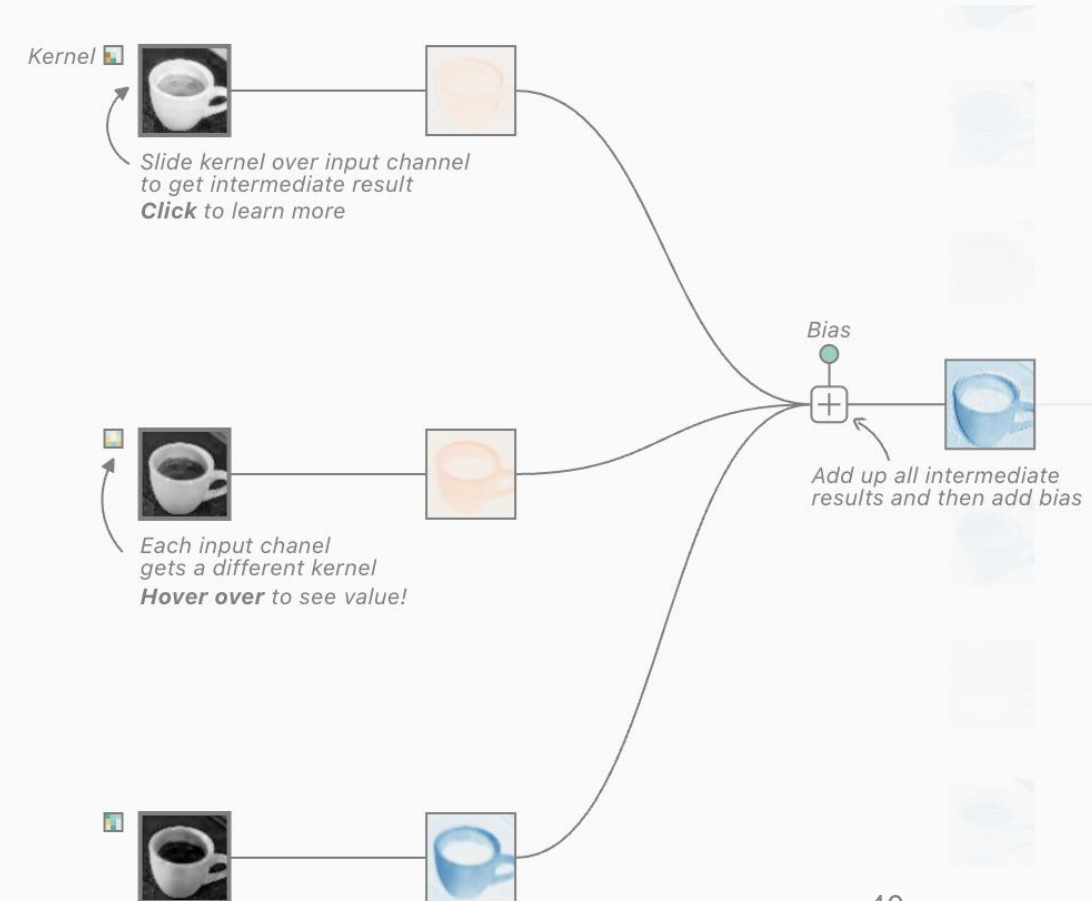
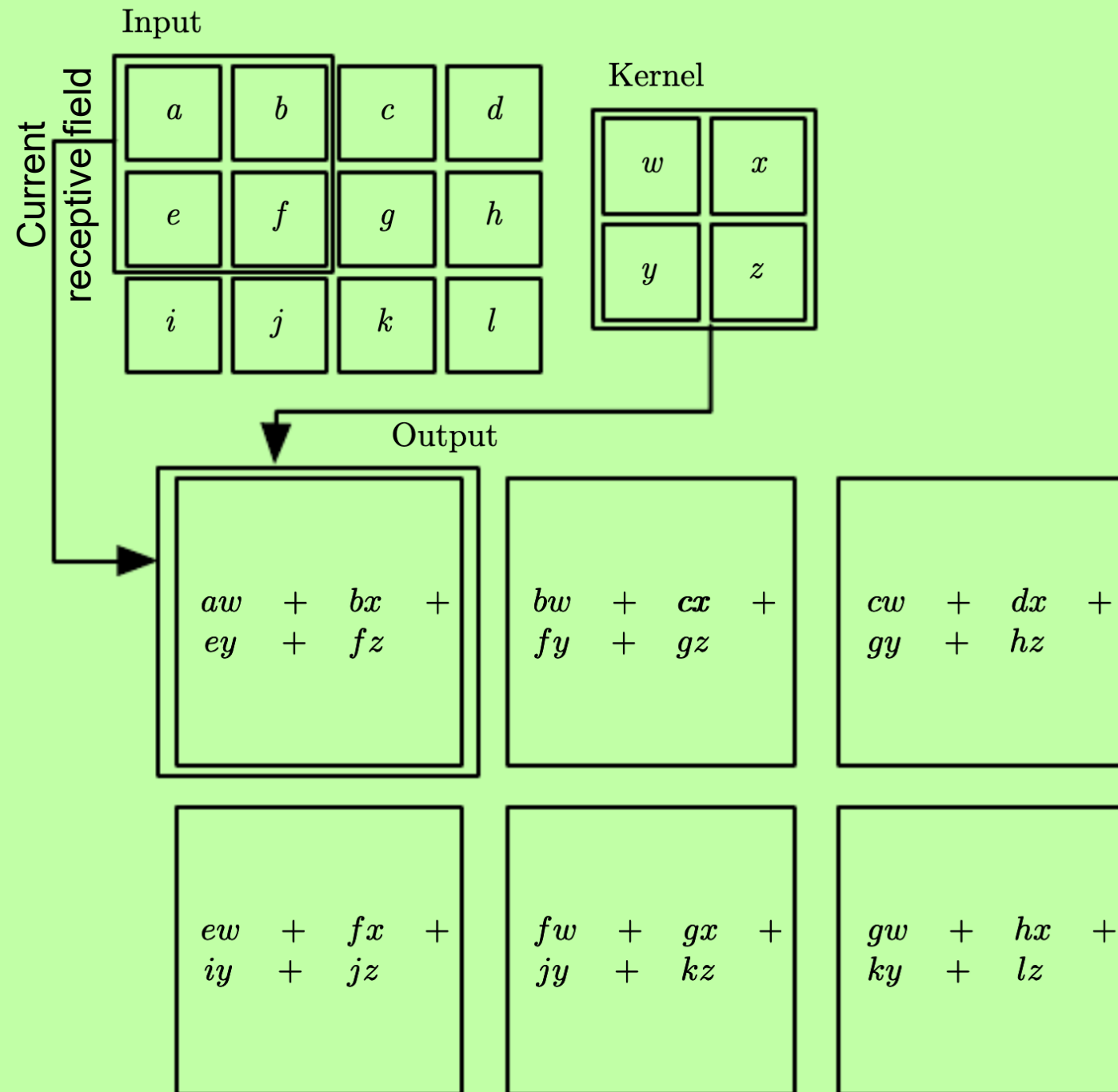
Convolution Layer



ConvNet

Convolution Layer

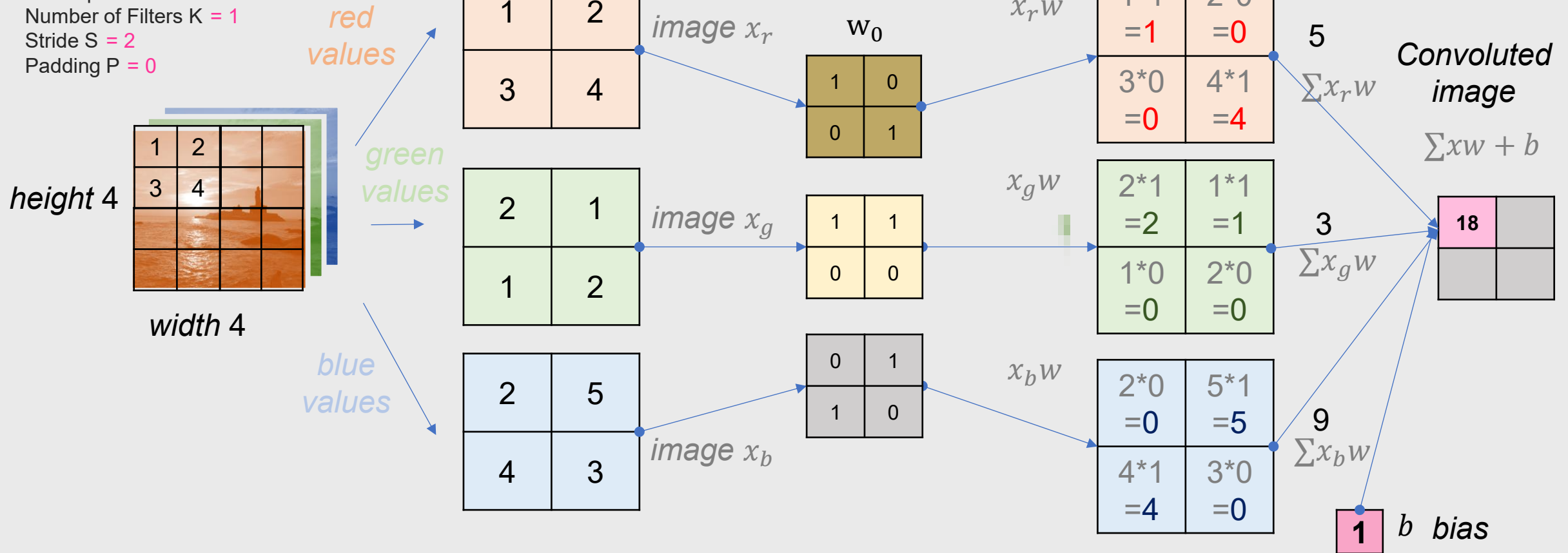
9:45 AM



Simple Convolution Example

Input Volume and Convolution

Image Width $W_1 = 4$
 Image Height $H_1 = 4$
 Image Depth $D_1 = 3$
 Filter spatial dimension $F = 2$
 Number of Filters $K = 1$
 Stride $S = 2$
 Padding $P = 0$



Note that each 2D map performs element-wise multiplication instead a dot product as may be advised in most text. **However**, if you flatten the tensors (i.e., make them vectors), you can do a dot product. Note that is only 1 filter but we may have many filters.

ConvNet

Convolution Layer:

Receptive field

Input volume of size $W_1 \times H_1 \times D_1$

Requires four hyperparameters:

- Number of filters K ,
- their spatial extent F ,
- the stride S ,
- the amount of zero padding P .

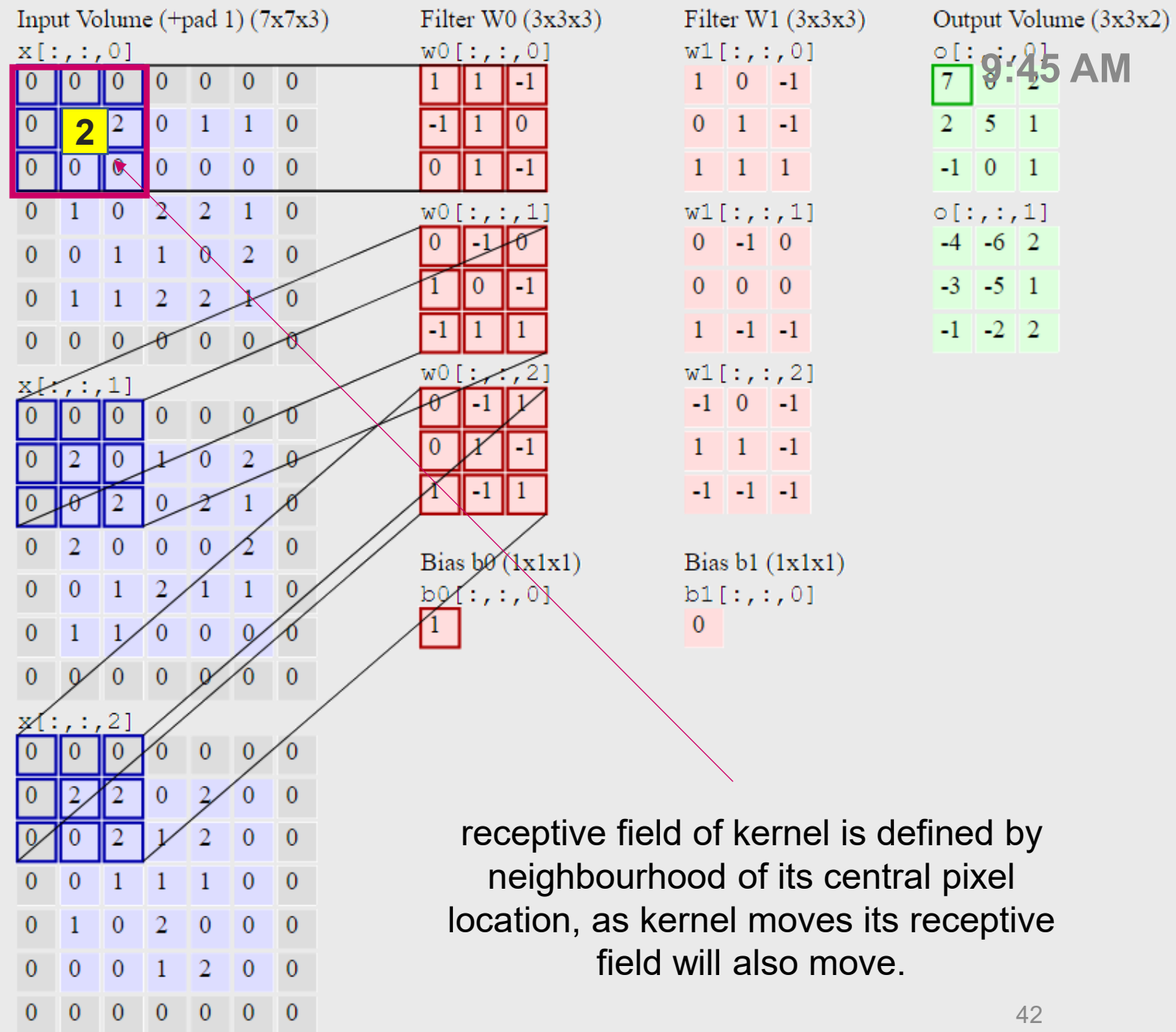
Output volume of size $W_2 \times H_2 \times D_2$

where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = K$

Source: <http://cs231n.github.io/convolutional-networks/>

receptive
field



ConvNet

Convolution

Layer: Padding

Input volume of size $5 \times 5 \times 3$

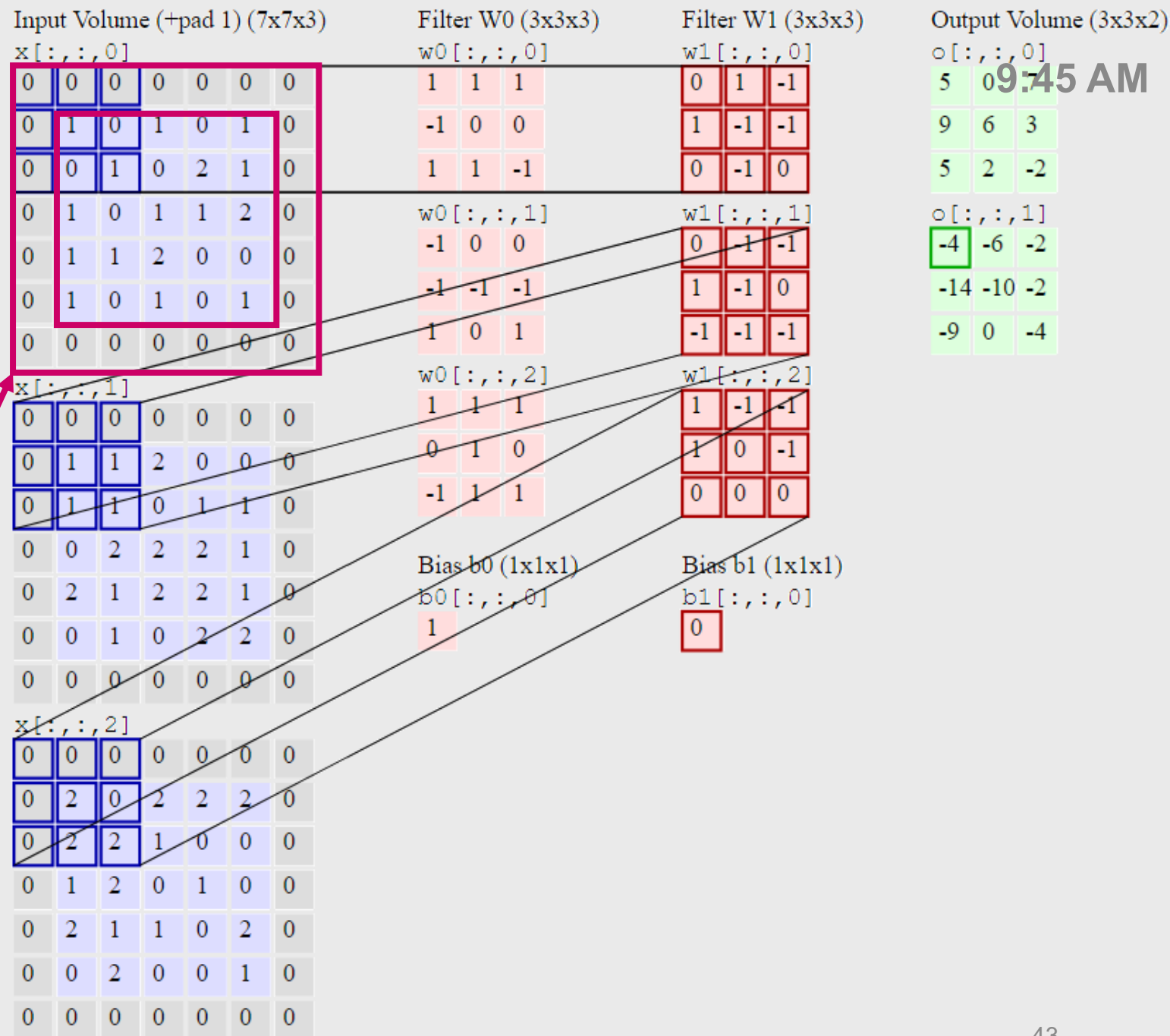
Requires four hyperparameters:

- Number of filters $K = 2$,
- their spatial extent $F = 3$,
- the stride $S = 2$,
- the amount of zero padding $P = 1$.

Output volume of size $W_2 \times H_2 \times D_2$

where:

- $W_2 = (5 - 3 + 2 \cdot 1) / 2 + 1$
- $H_2 = (5 - 3 + 2 \cdot 1) / 2 + 1$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = 2$



ConvNet Convolution Layer: Stride

Input volume of size $5 \times 5 \times 3$

Requires four hyperparameters:

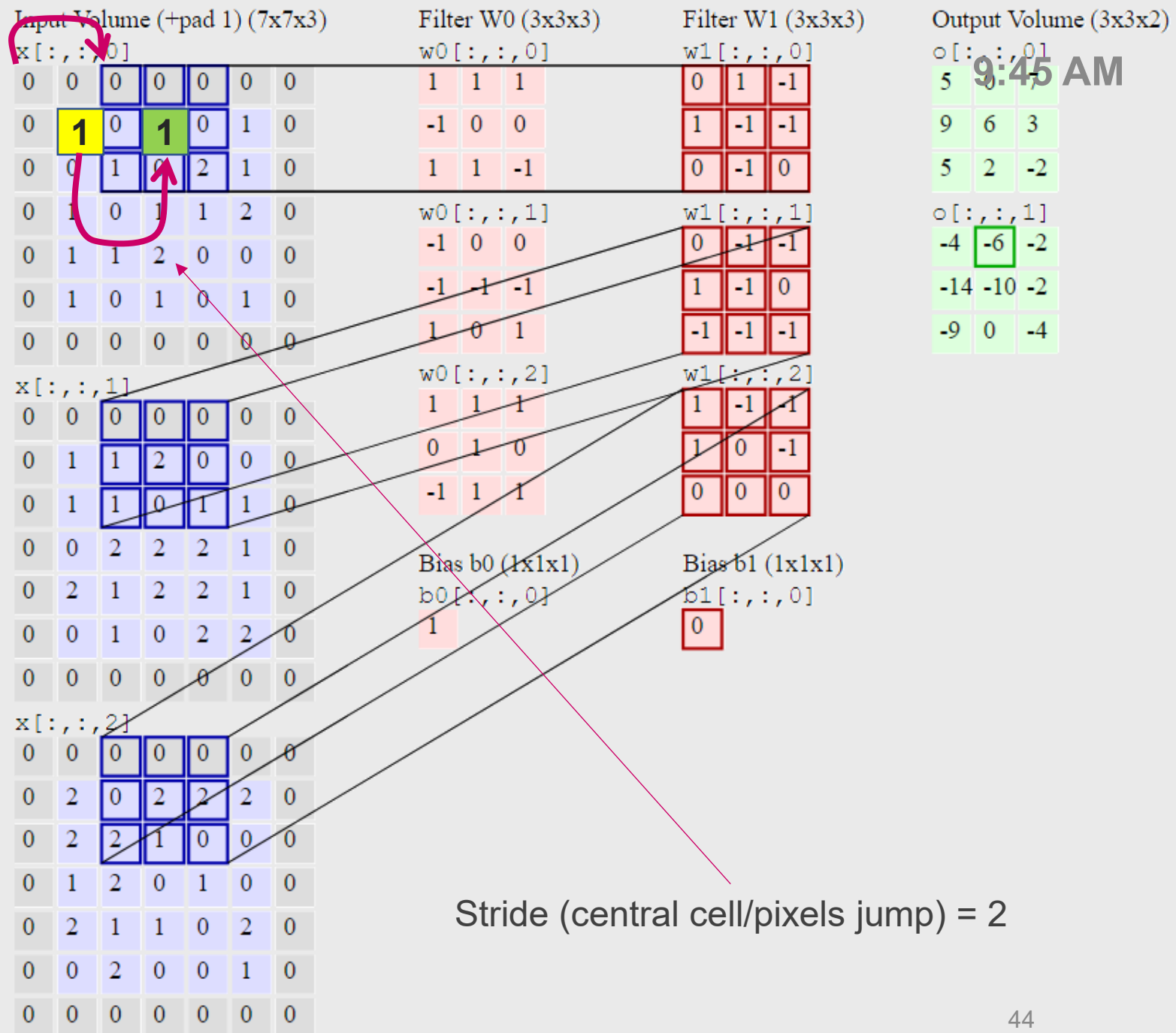
- Number of filters $K = 2$,
- their spatial extent $F = 3$,
- the stride $S = 2$, (moved 2 pixels)
- the amount of zero padding $P = 1$.

Output volume of size $W_2 \times H_2 \times D_2$

where:

- $W_2 = 3$
- $H_2 = 3$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = 2$

Source: <http://cs231n.github.io/convolutional-networks/>



Effect of Learned Convolutional Filter on Images 9:45 AM

This is most likely scenario as we do not know exactly what will be the filter weights after training. Hence CNN is a Blackbox

These weights/values of 3x3 kernels/filters are learned during training
(filter values and outputs are mere representative)

Identity

0	0	0
0	1	0
0	0	0

Sharpen

0	-1	0
-1	7	-1
0	-1	0

Blur

1/5	1/5	1/5
1/5	1/5	1/5
1/5	1/5	1/5

Laplacian

0	1	0
1	-5	1
0	1	0

Gaussian

1/8	2/8	1/8
2/8	4/8	2/8
1/8	2/8	2/8



Original



Identity



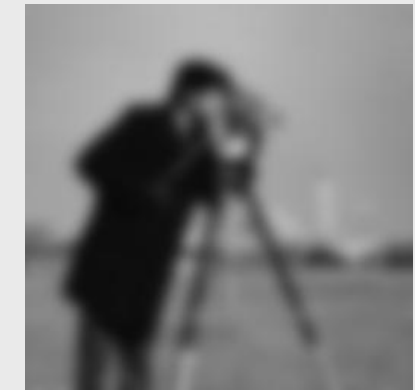
Sharpen



Blur



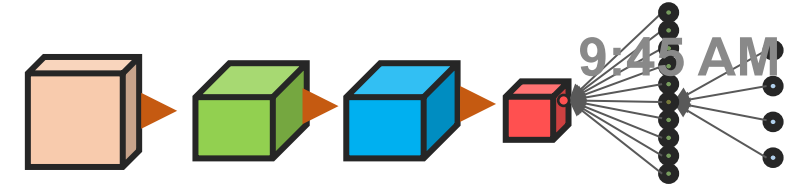
Laplacian



Gaussian

See this for more examples: <https://setosa.io/ev/image-kernels/>

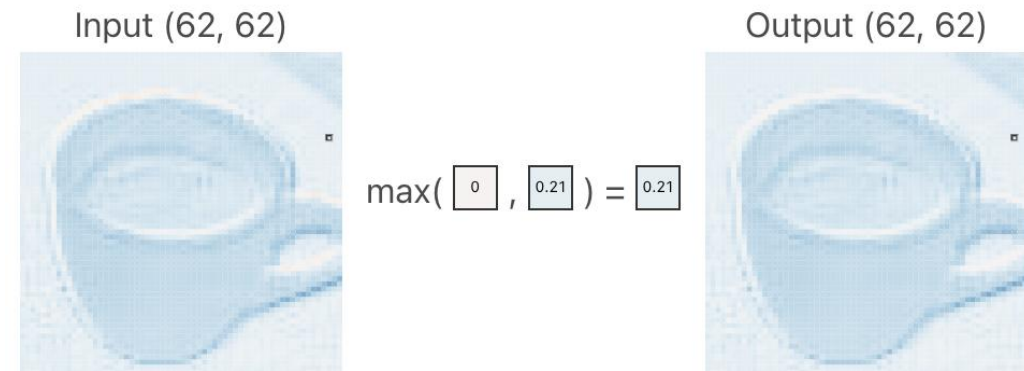
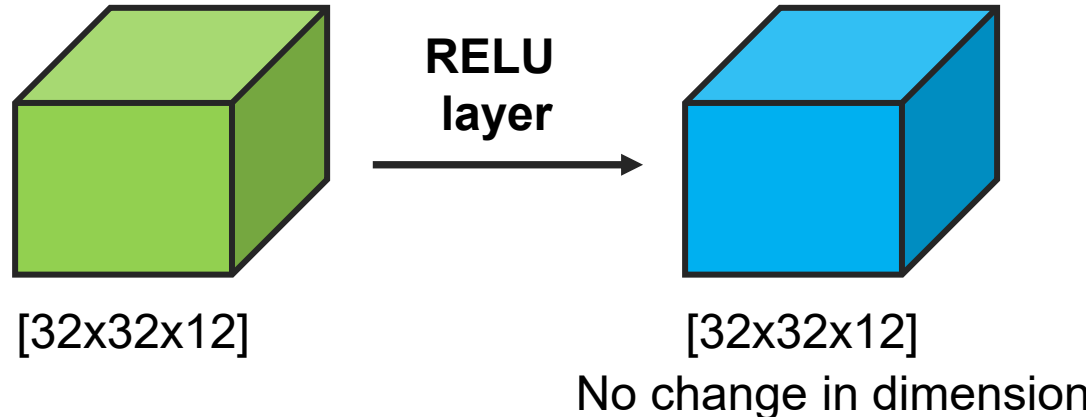
ConvNet/ CNN



Architecture: A Simple ConvNet / CNN

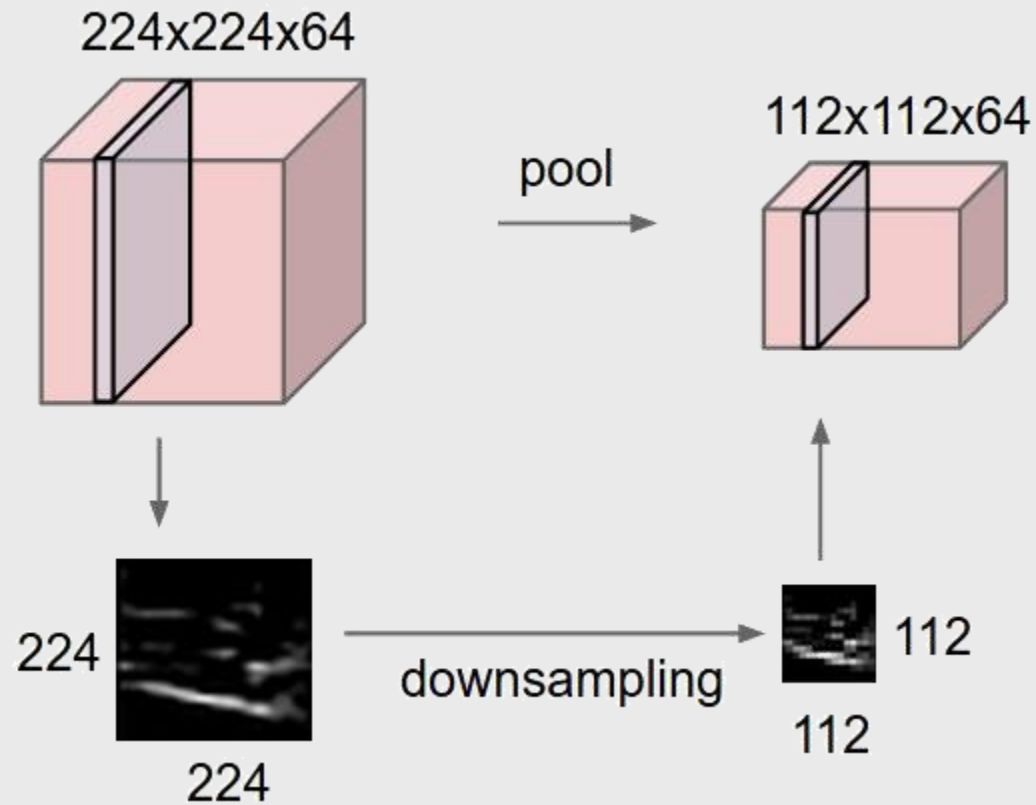
[INPUT - CONV - **RELU** - POOL - FC]

RELU layer will apply an elementwise activation function, such as the $\max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ($[32 \times 32 \times 12]$).



ConvNet

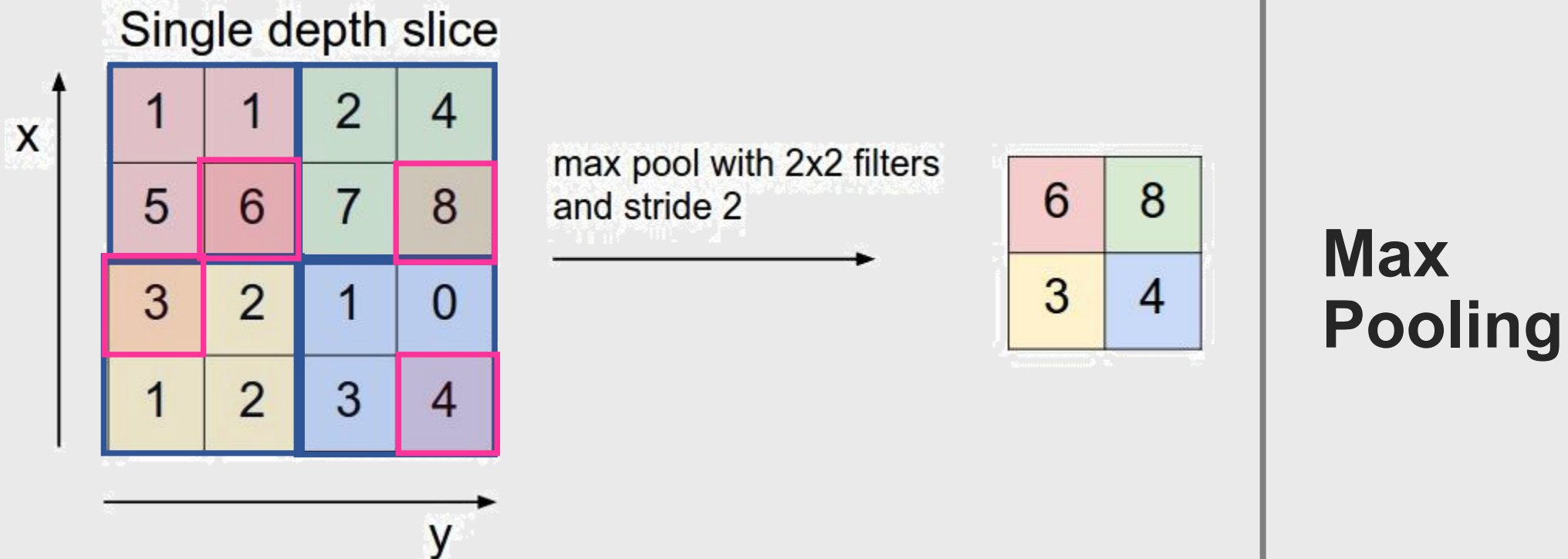
Pooling Layer



Pooling layer **down samples** the volume spatially, independently in each depth slice of the input volume

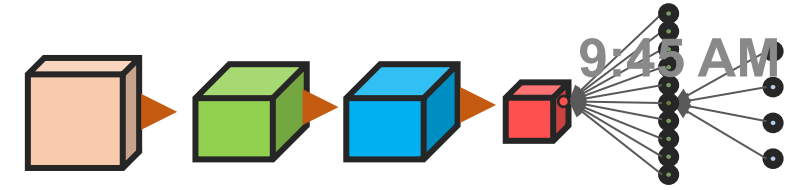
ConvNet

Pooling Layer



A very good source: <http://cs231n.github.io/convolutional-networks/>

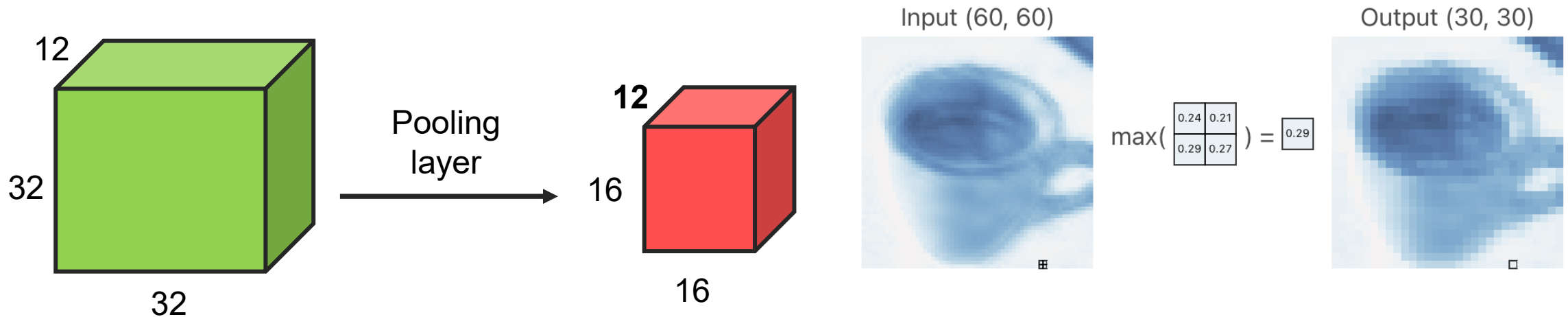
ConvNet/ CNN



Architecture: A Simple ConvNet / CNN

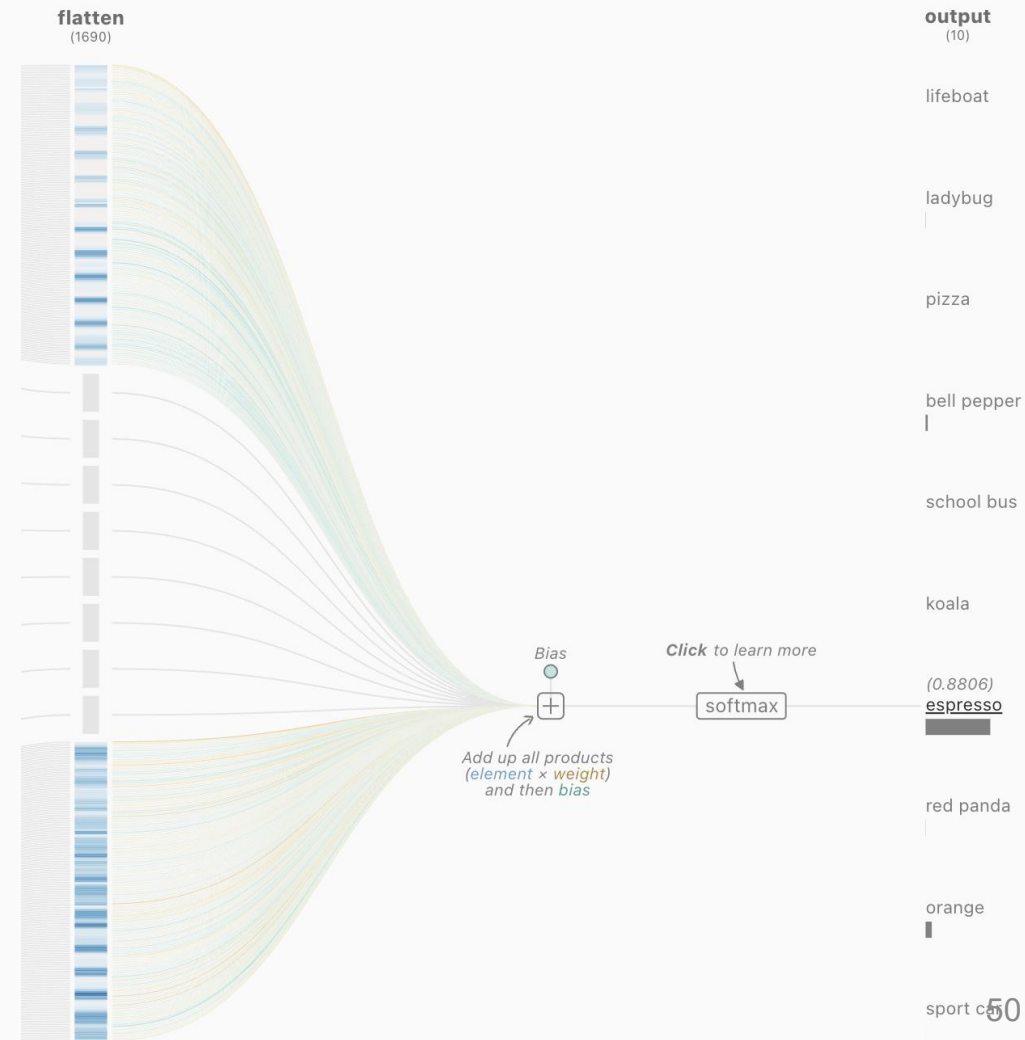
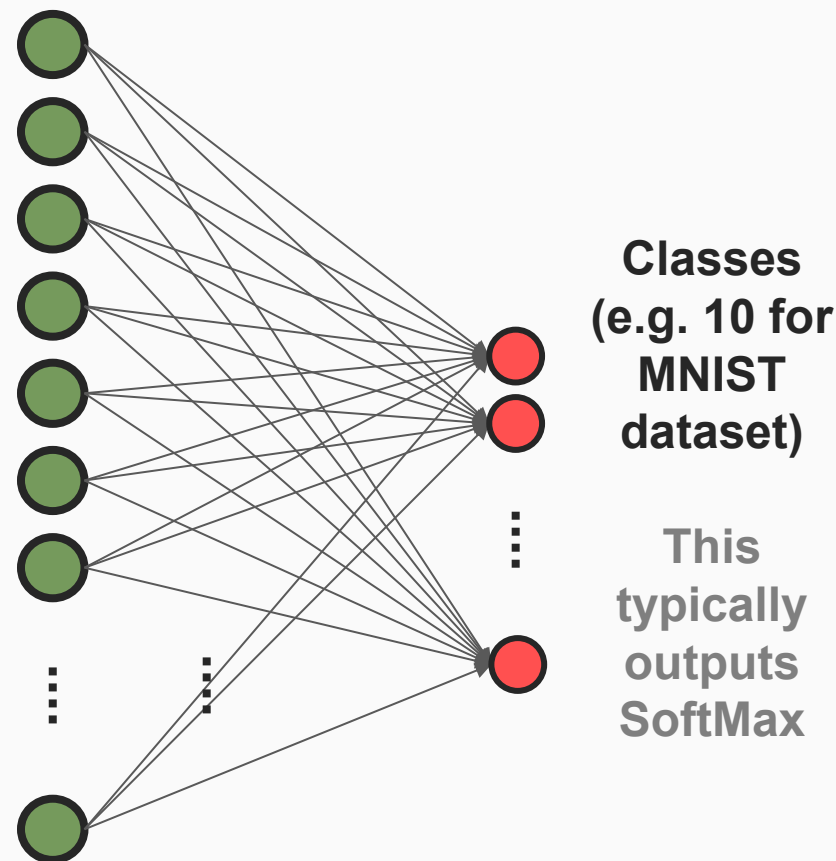
[INPUT - CONV - RELU - **POOL** - FC]

POOL layer will perform a **down sampling** operation along the spatial dimensions (width, height), resulting in volume such as [16x16x12]



ConvNet

Fully Connected (Dense) Layer



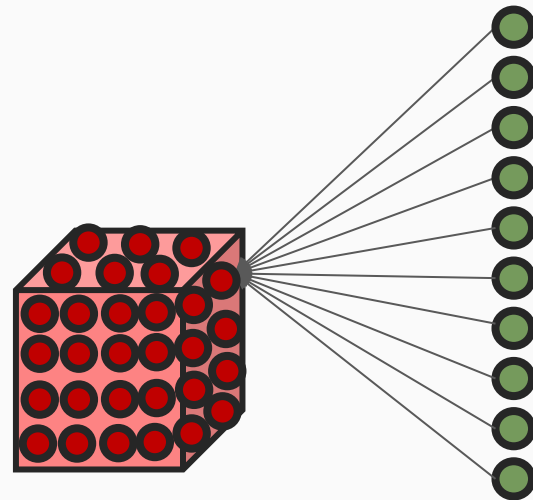
ConvNet/ CNN

Architecture: ConvNet / CNN

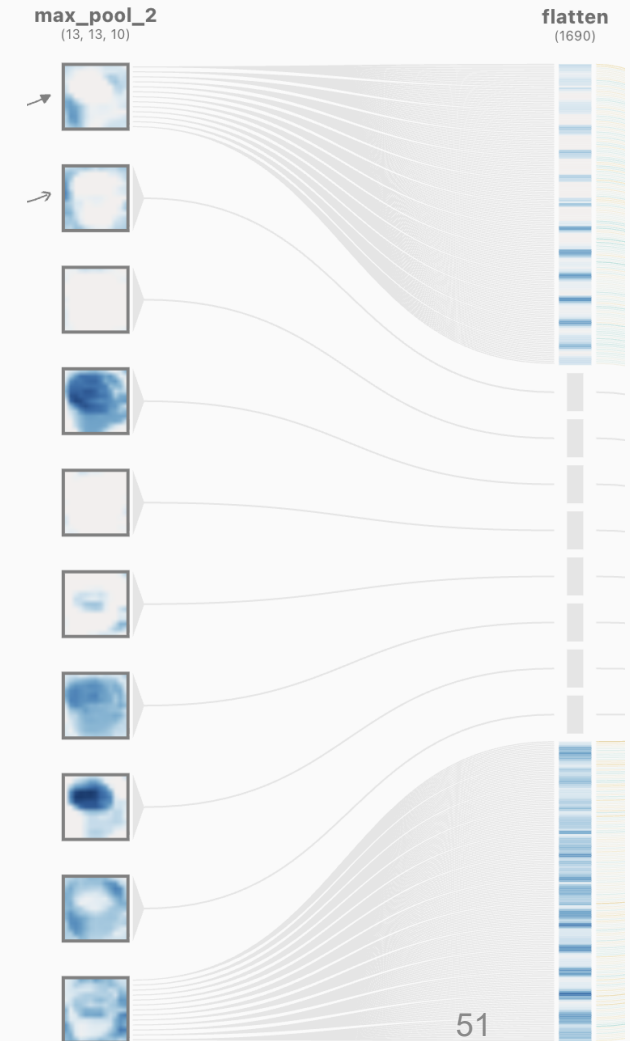
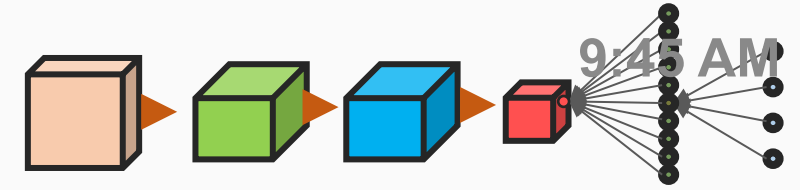
[INPUT - CONV - RELU - POOL - FC]

FC (i.e. fully-connected) layer may compute the class scores, resulting in volume of size $[1 \times 1 \times 10]$, where each of the 10 neurone correspond to a class score, such as among the 10 categories.

A single neuron of the volume. There are as many neuron as is the volume size



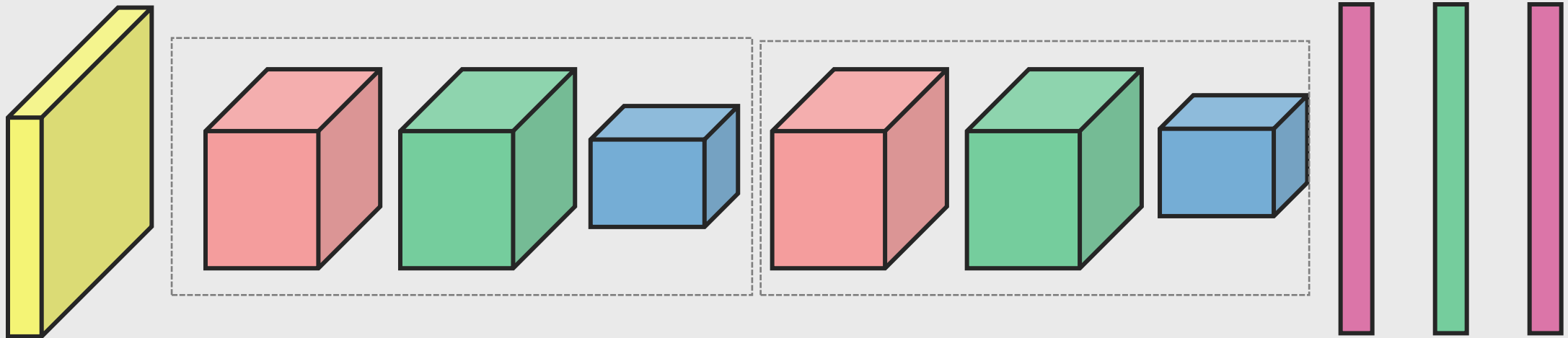
An FC layer is also a linear layer. It can have as many neurons/nodes as a user defines them to be



ConvNet

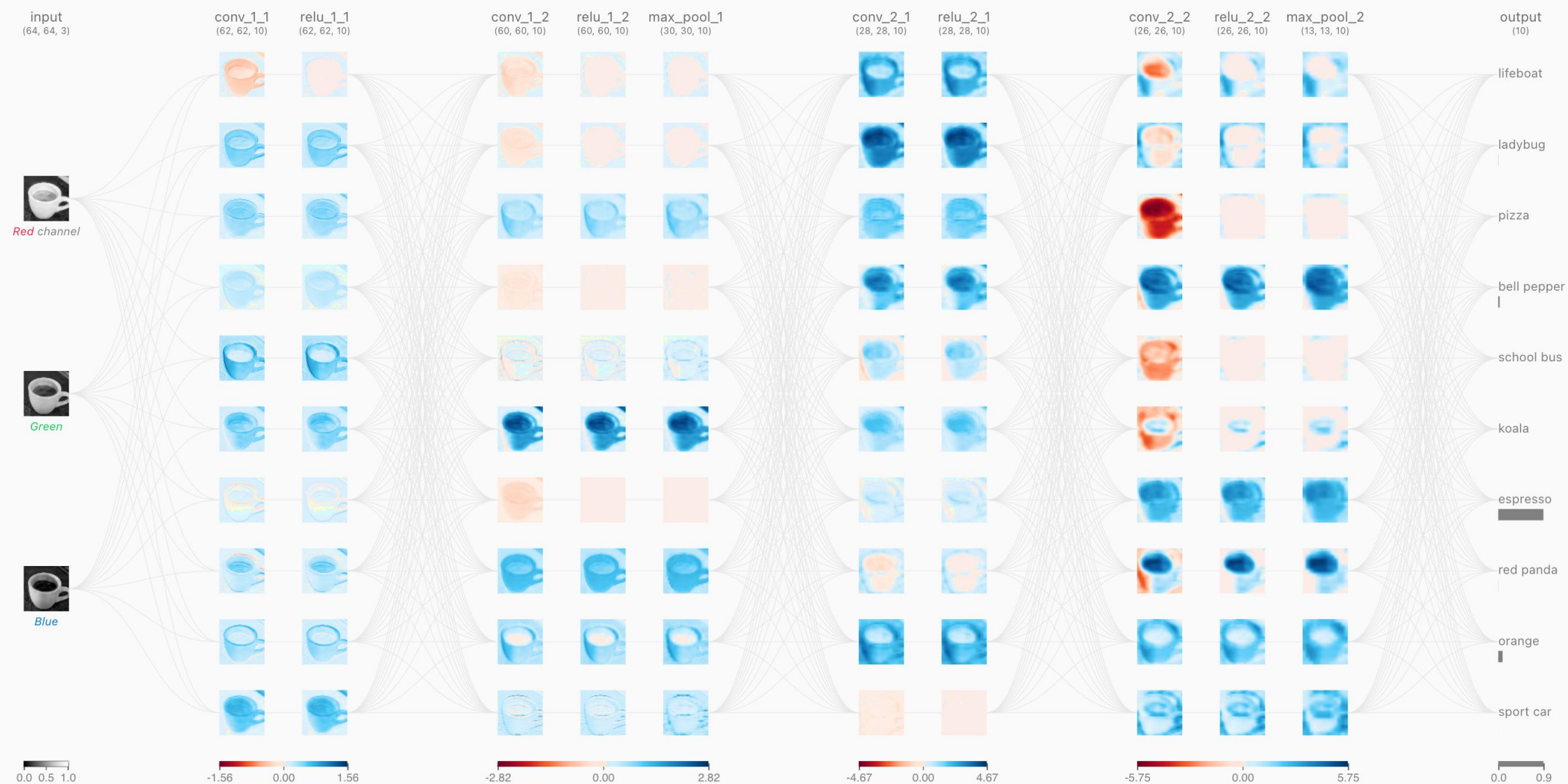
ConvNet Architecture

INPUT $\rightarrow$ [CONV $\rightarrow$ RELU $\rightarrow$ POOL] * 2 $\rightarrow$ FC $\rightarrow$ RELU $\rightarrow$ FC]



Visualizing CNN

9:45 AM



Try yourself: <https://poloclub.github.io/cnn-explainer/>

ConvNet on Image Classification

9:45 AM

Live demo <http://cs231n.stanford.edu/>



A very good source:
<http://cs231n.github.io/convolutional-networks/>

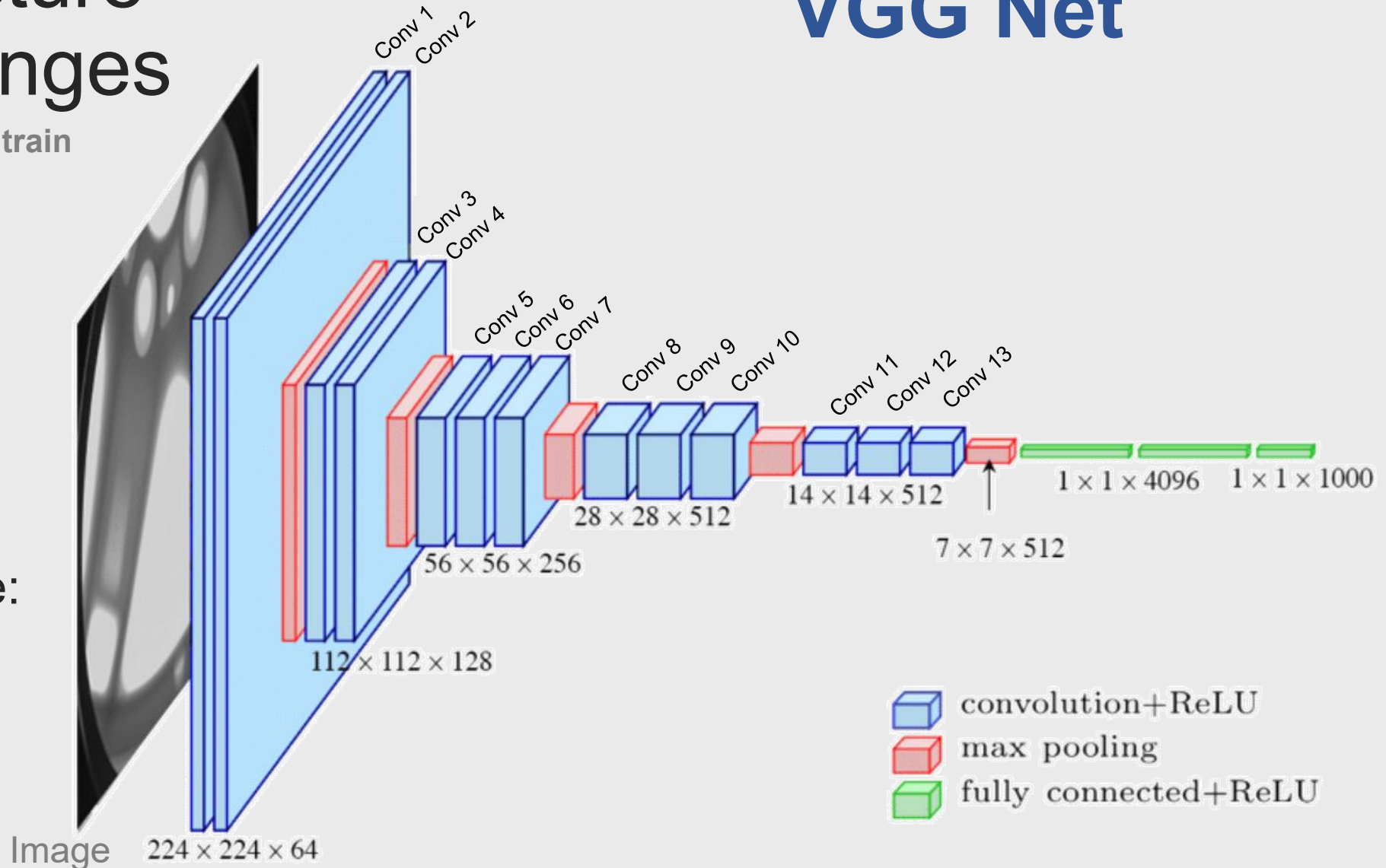
CNN architecture design challenges

Deeper the network, harder to train

- Visual Geometry Group (VGG) Network (VGG Net)
- VGG 16 - an example architecture: 13 Convolution layers 3 fc layer.

VGG Net

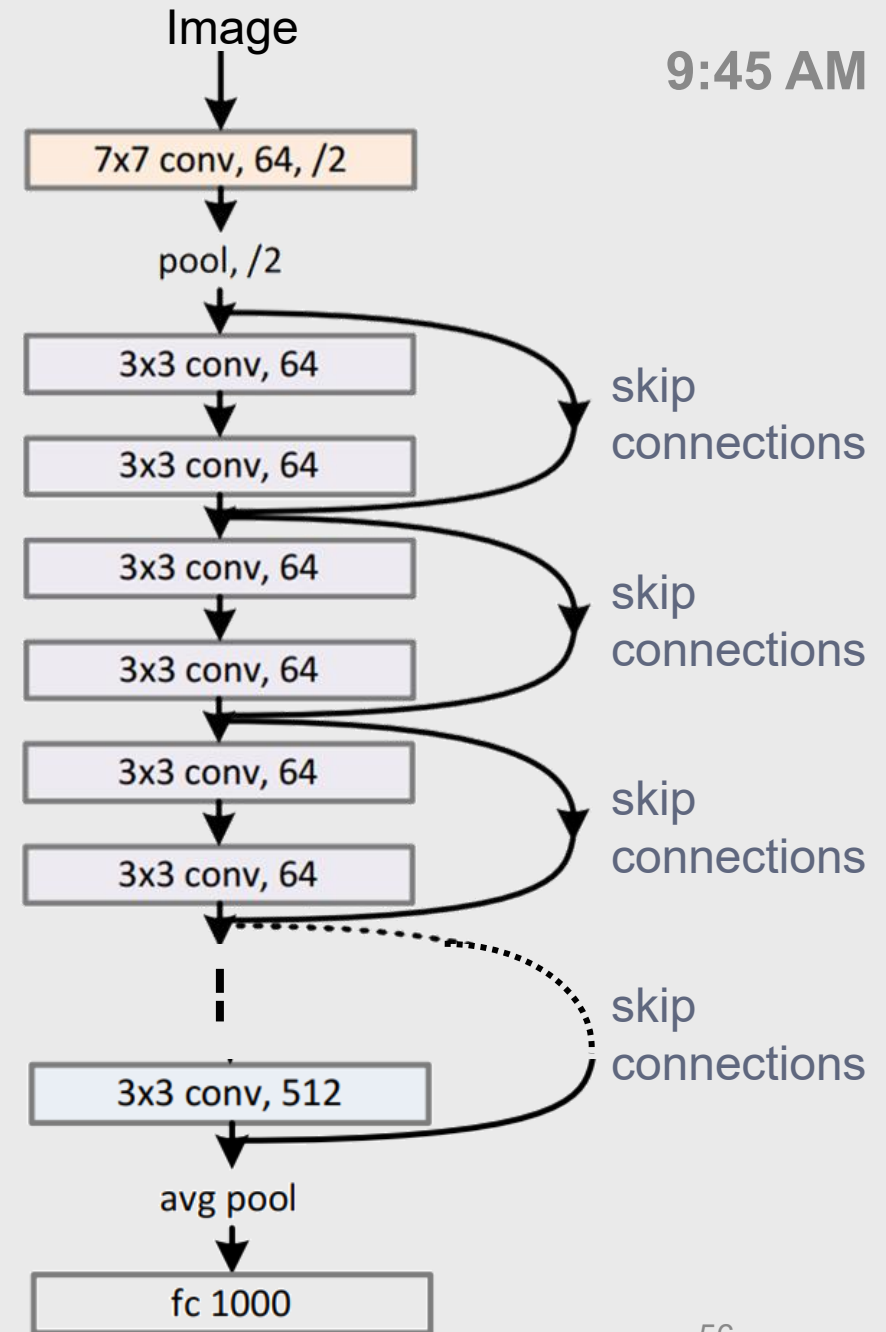
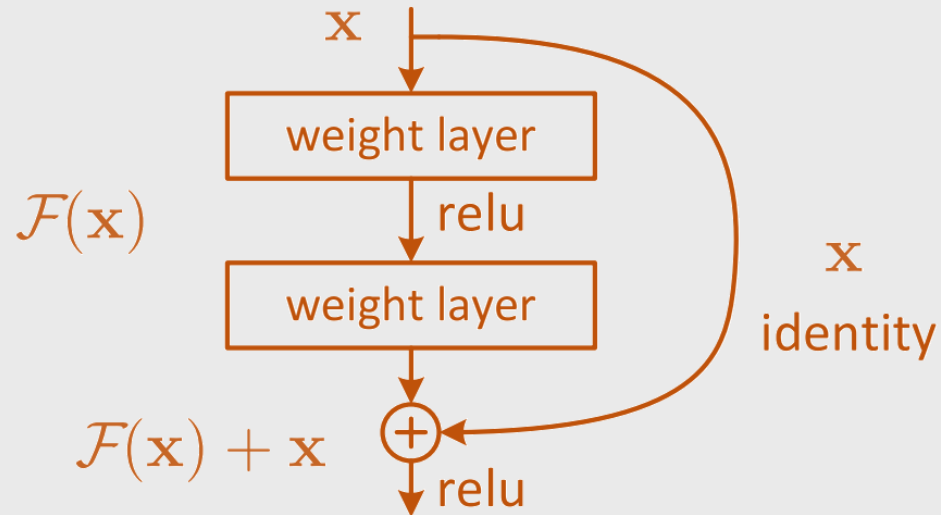
9:45 AM



Residual Network (ResNet)

It solves vanishing/exploding gradient problem. Skip connections allow you to take the activation value from an earlier layer and pass it to a much deeper layer in a network. This allows for smoother gradient flow, ensuring important features are preserved in the training process.

ResNet Block



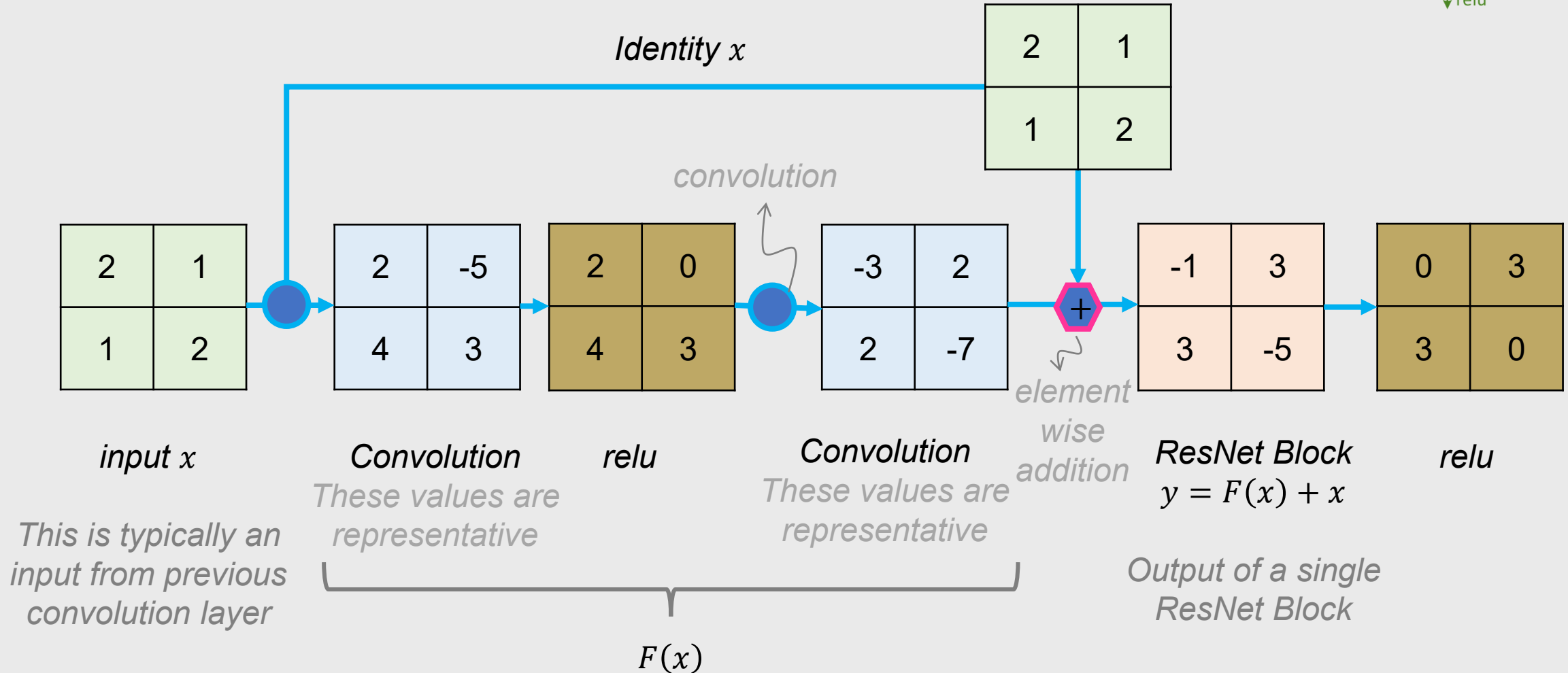
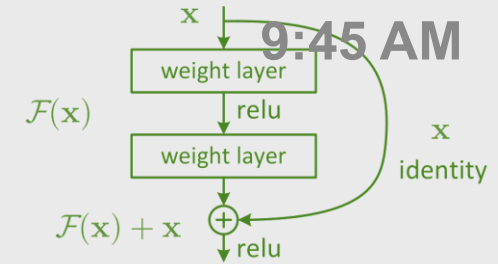
9:45 AM

He et al (2016) Deep Residual Learning for Image Recognition, CVPR

Image source: <https://arxiv.org/pdf/1512.03385>

Simple Example of ResNet Block

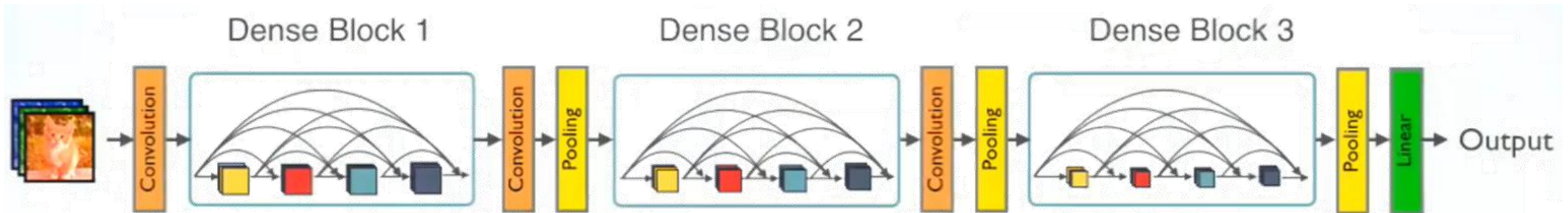
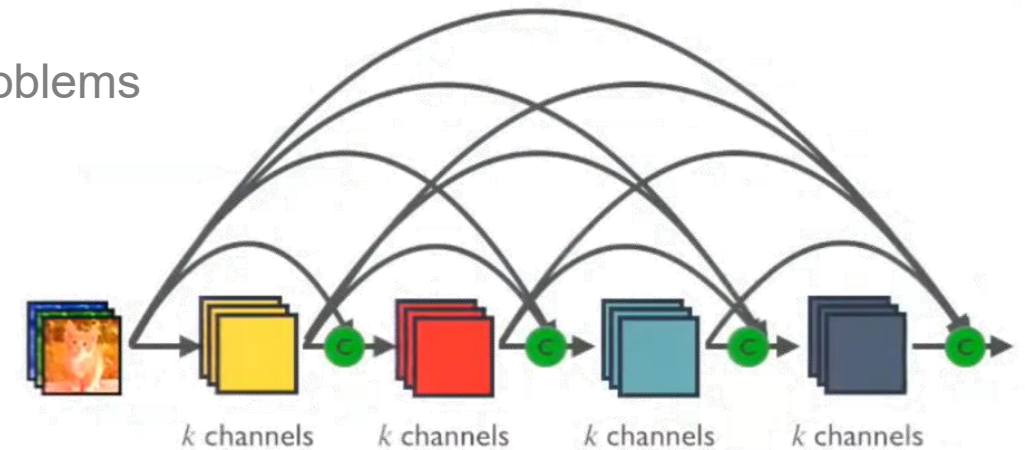
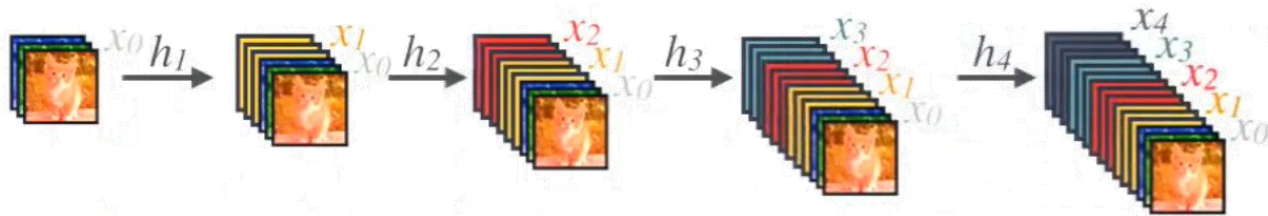
element-wise addition to pass the output to the next layer or block



Dense Net

9:45 AM

Dense Connection to solve vanishing or exploding gradient problems



ResNet performs an element-wise addition to pass the output to the next layer or block. **DenseNet** connects all layers directly to each other. It does this through concatenation (combine features by concatenating). Such connection provide stronger gradient flow, feature diversity form using simple features to complex features, as opposed to feature corelation in ResNet.

Image source: <https://www.youtube.com/watch?v=-W6y8xnd--U>

Huang et al (2017) Densely Connected Convolutional Networks

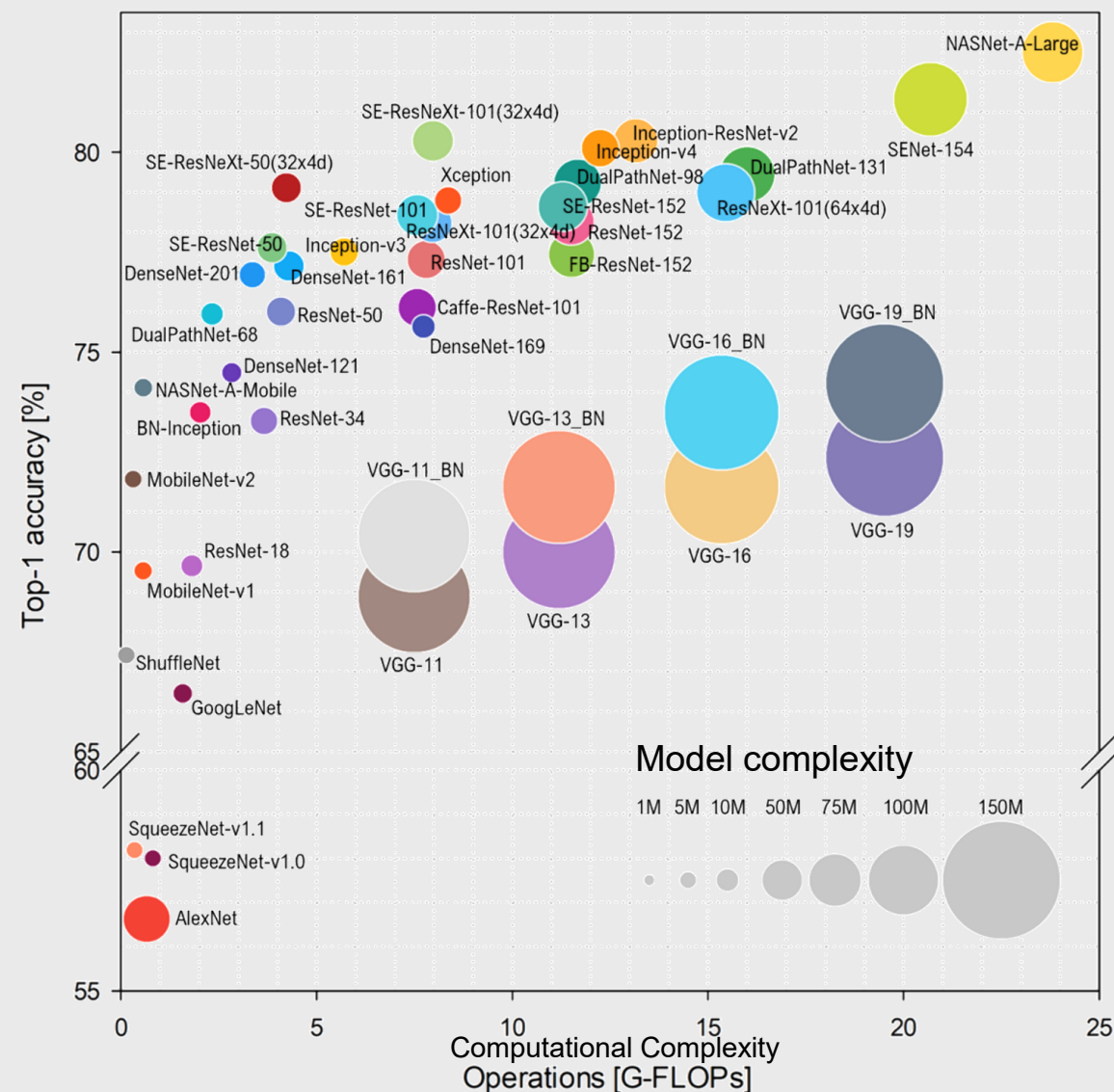
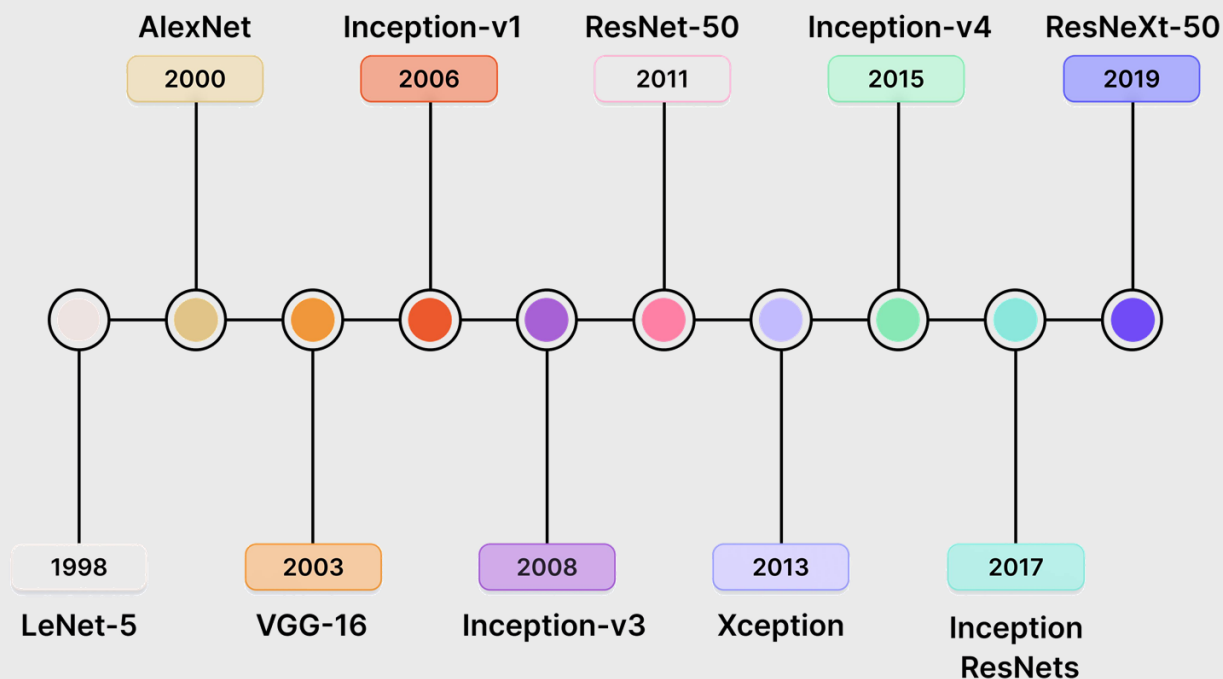
Evolution of CNN Architectures

9:45 AM



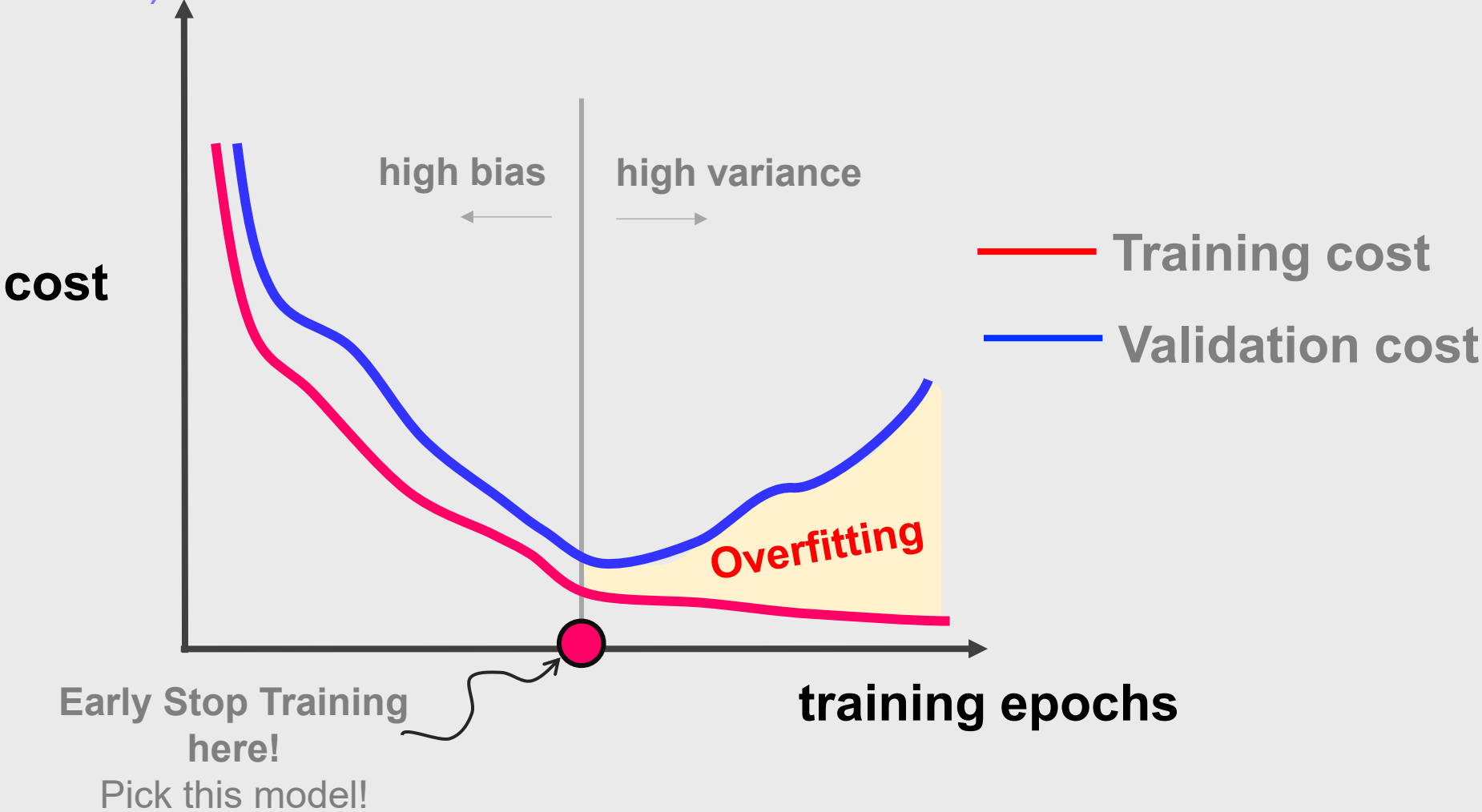
<https://www.image-net.org/challenges/LSVRC/index.php>

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) that changed AI Research



Overfitting Issues

(from previous lecturers)



Regularization

9:45 AM

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 + \lambda \sum_i^n w_i^2$$

minimize regularizer
term, minima is

$$\lambda \sum_i^n w_i^2 = 0$$

$$\lambda \sum_i^n w_i^2 > 0$$

regularizer term

$$\lambda \sum_i^n w_i^2$$

error term

$$\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2$$

minimize error term,
minima is

$$\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 = 0$$

error $\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 > 0$

minimize combined error

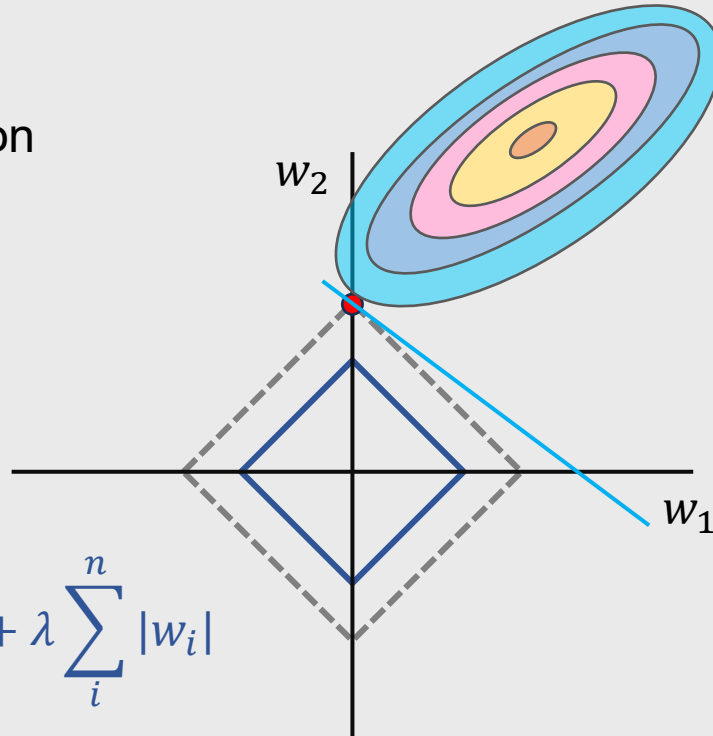
L_2 -norm regularization
Ridge regularization

Regularization to avoid overfitting

9:45 AM

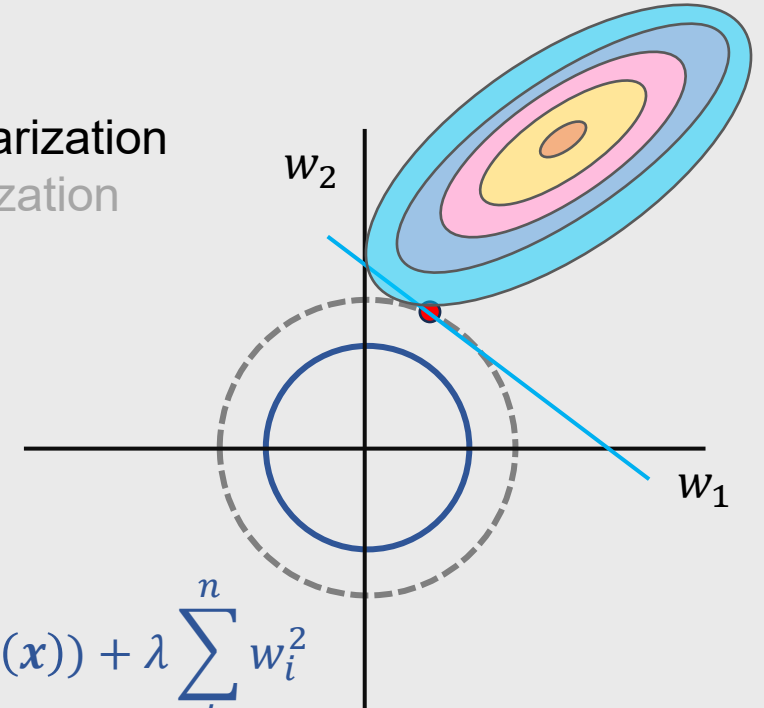
L_1 -norm regularization
Lasso regularization

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x})) + \lambda \sum_i^n |w_i|$$



L_2 -norm regularization
Ridge regularization

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x})) + \lambda \sum_i^n w_i^2$$



- Both Lasso and Ridge regularization help DNNs and CNNs avoid overfitting.
- As weights can get zero in Lasso regularization introduce sparsity in the networks and help feature selection because some weights goes to zero (although not necessary in CNN). Thus, eliminating effect of some input features, while in ridge regression weights can only get close to zeros and not exactly (see images).
- L2 penalizes large errors much more heavily than small errors (compared to L1), thus on optimization of network, it is safe to assume that all the errors are roughly of the same order of magnitude

Regularization: Dropouts Layer

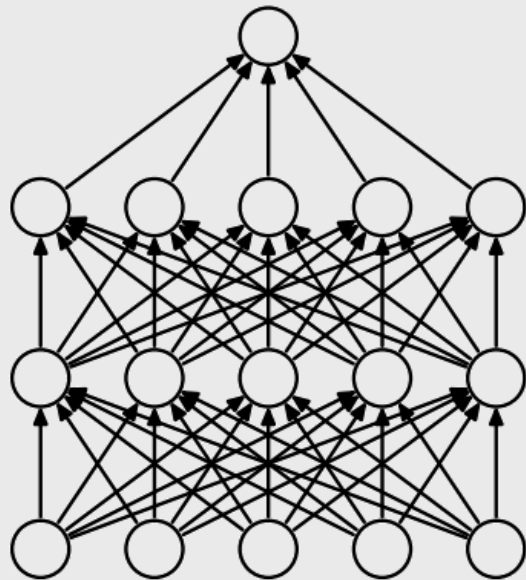
9:45 AM

Regularisation of DNNs and CNNs

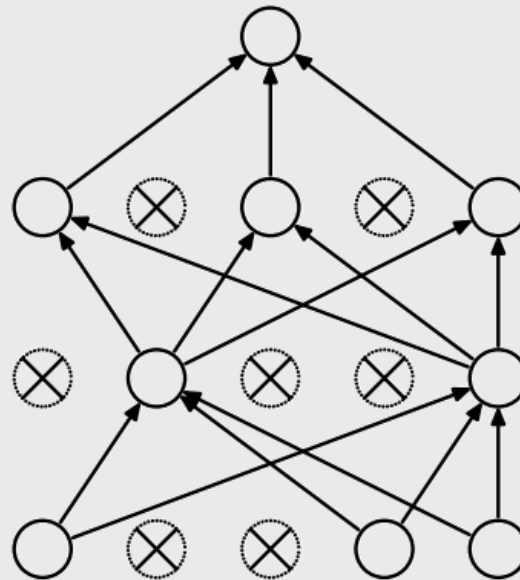
!!!

Slows down training in Convolutional Nets

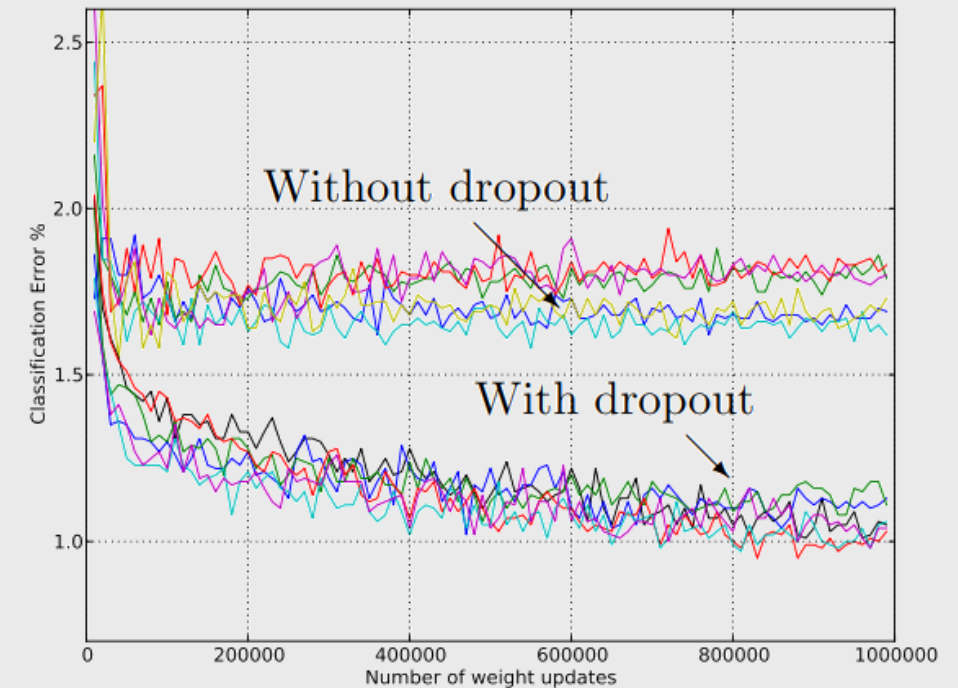
- It drop nodes with some probability
- It regularise Deep Neural Nets and Convolutional Neural Nets



(a) Standard Neural Net



(b) After applying dropout.



Sec 7.12, Goodfellow, Deep Learning, MIT Press

Srivastava et al (2014) Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR

Batch Normalization Layer

Ioffe and Szegedy (2017) Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift ICML.

Regularisation of Convolutional Neural Networks

- It eliminates the need of dropout
- Accelerates ConvNet training
- It reduces sensitivity to network weight initialisation
- Standardizing the activations to have a zero mean and unit variance

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i$$

Mean of activation

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

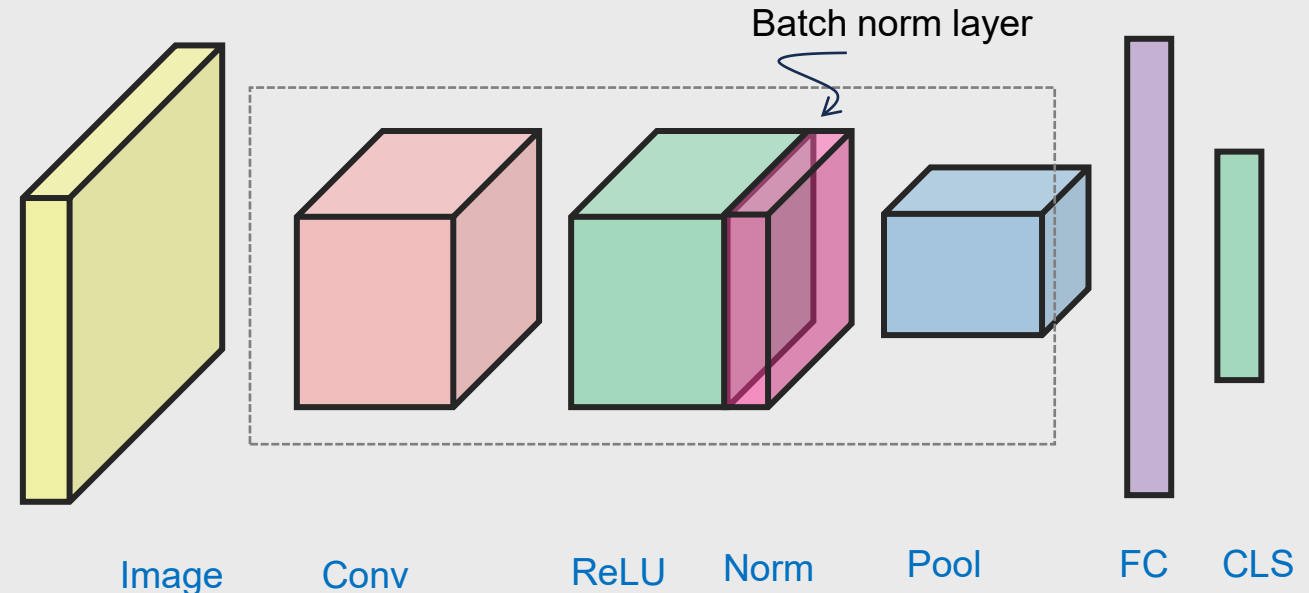
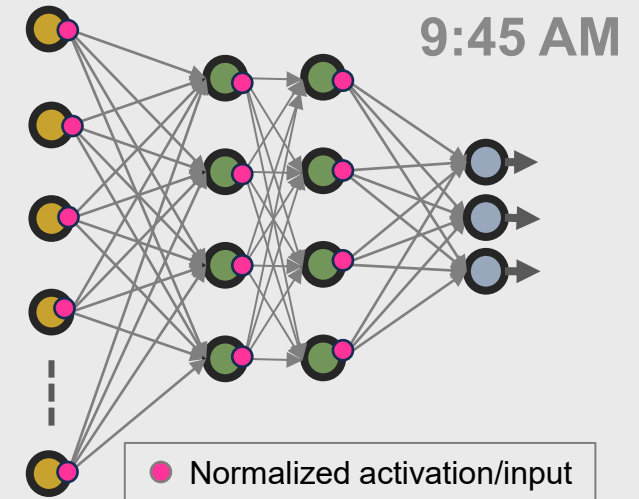
Variance of activation

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Standardization (Z-score) of activation

$$y_i = \gamma \hat{x}_i + \beta = \text{BN}_{\gamma, \beta}(x_i)$$

Batch norm regularization scale and shift with learnable $\gamma \rightarrow \text{mean}, \beta \rightarrow \text{variance}$

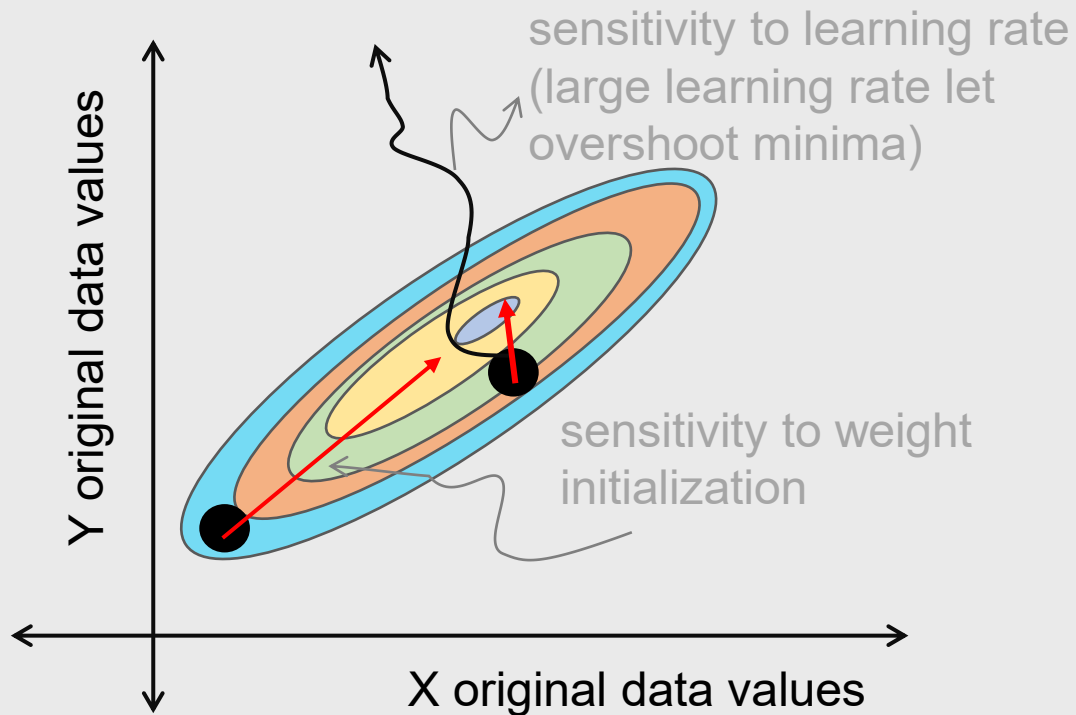


Batch Normalization Layer

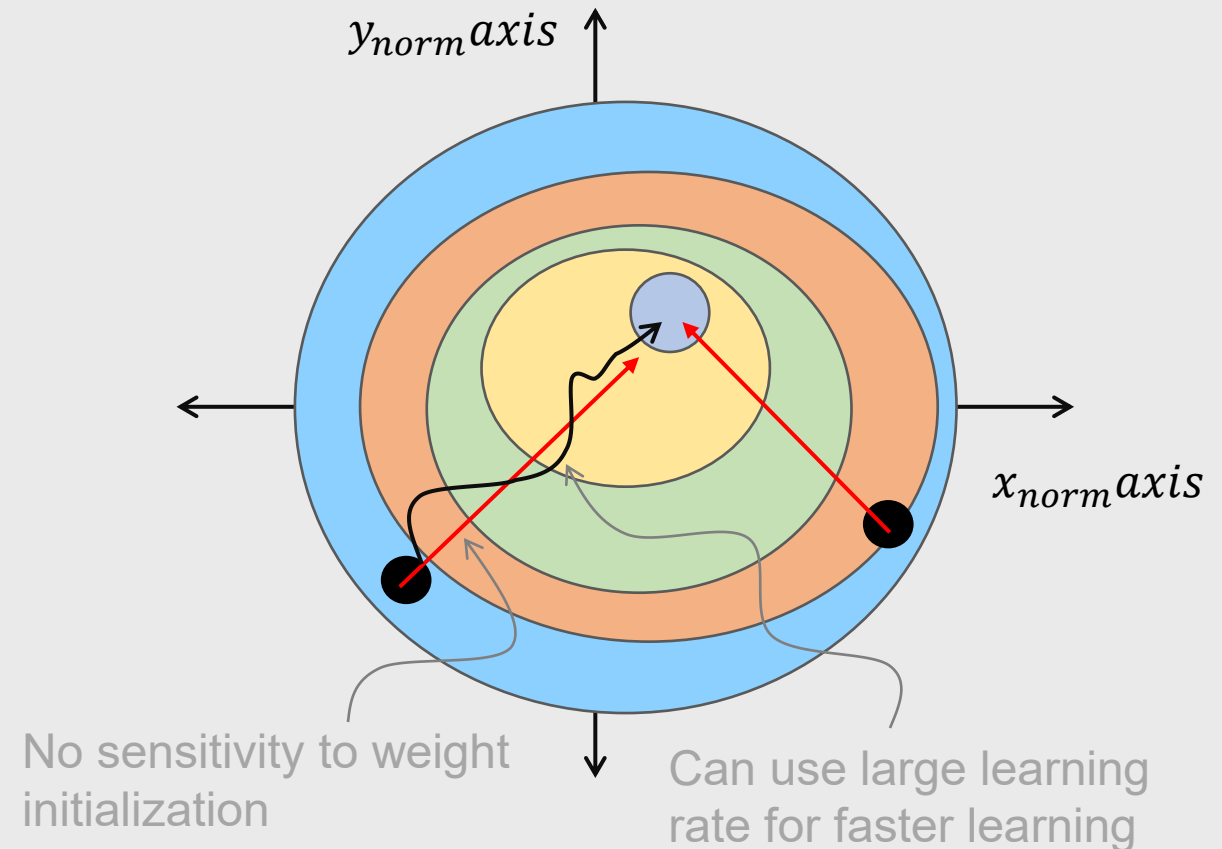
9:45 AM

Regularisation of Convolutional Neural Networks

Optimization/ cost function
space on unnormalized inputs

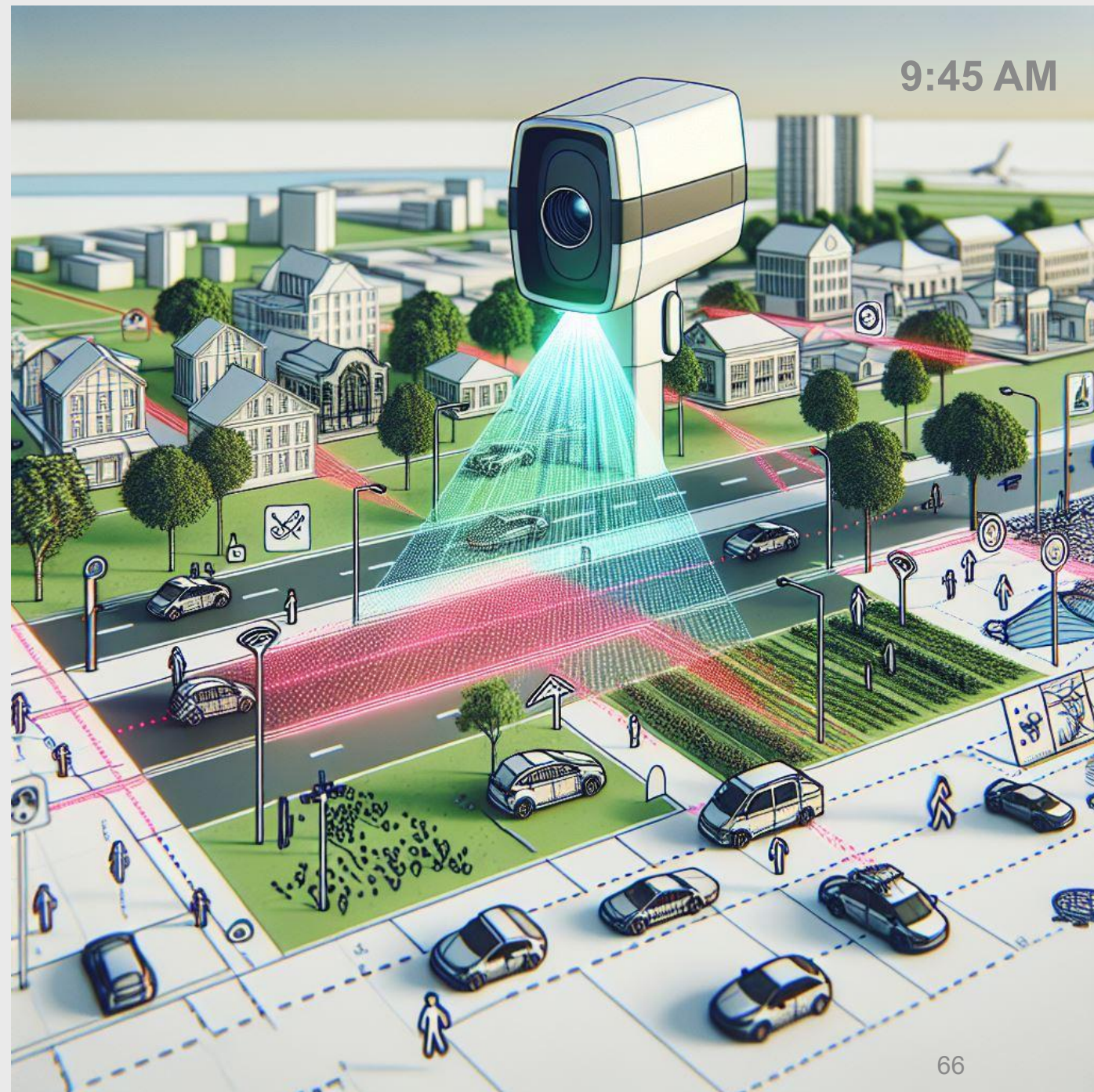


Optimization/ cost function
space on unnormalized inputs



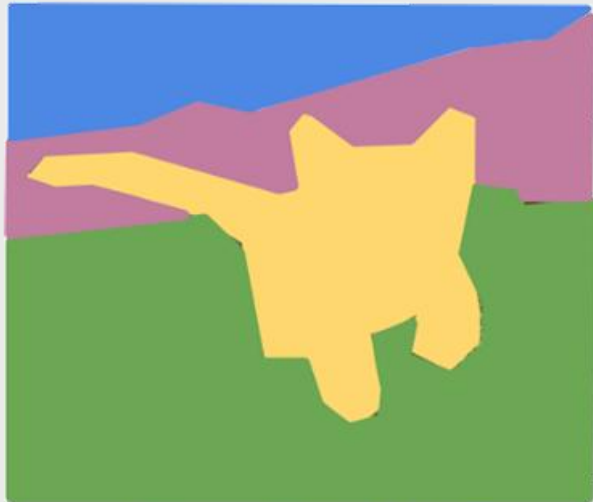
Part 3

Convolutional Neural Networks Models



Computer Vision Tasks

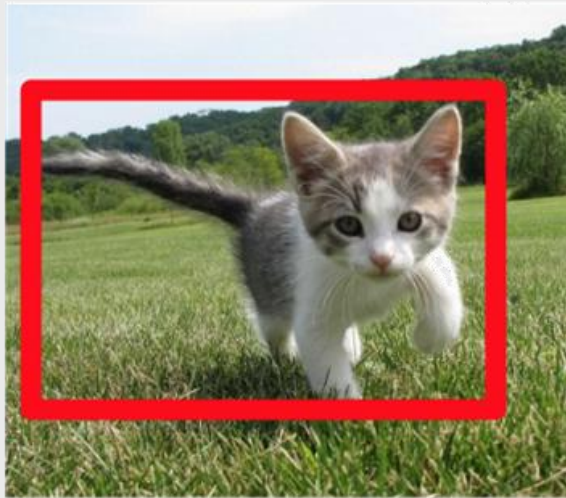
Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Classification + Localization



CAT

Single Object

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

This image is CC0 public domain

Image Classification Example

9:45 AM

Live demo <http://cs231n.stanford.edu/>

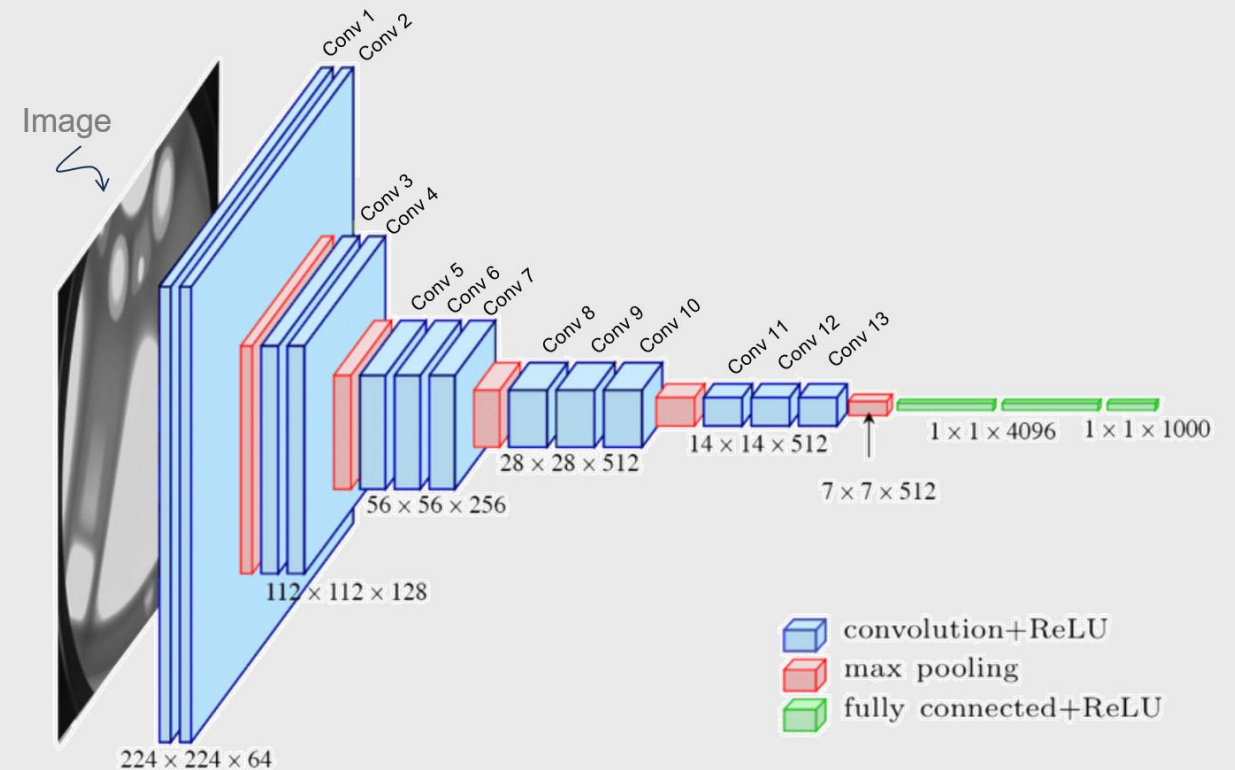


A very good source:
<http://cs231n.github.io/convolutional-networks/>

Image Classification Models

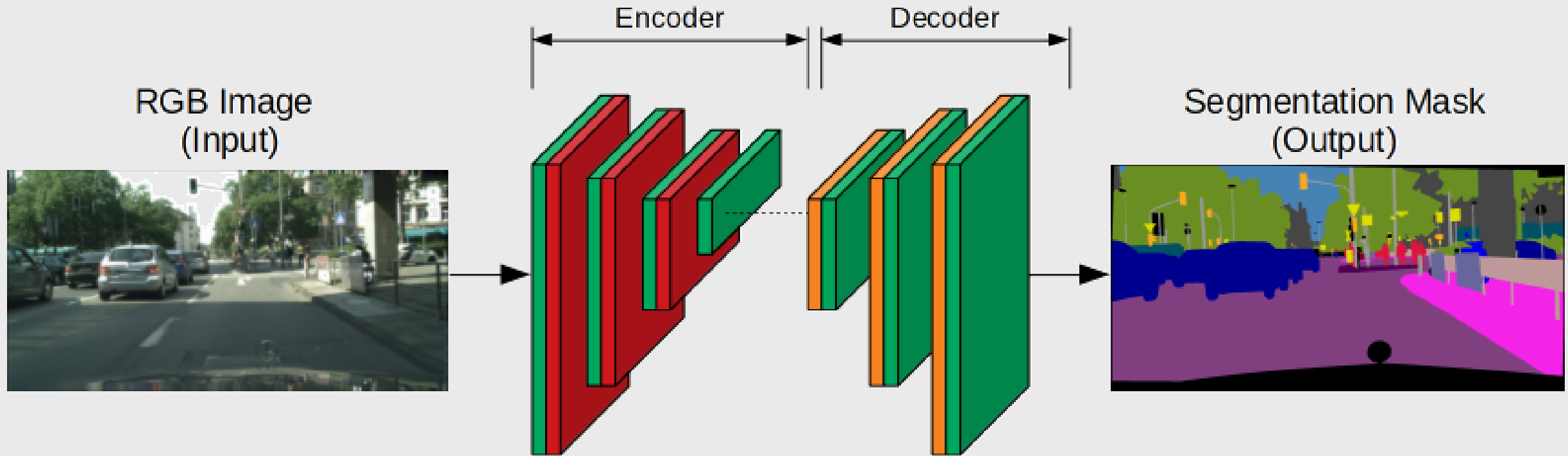
- Residual Networks (**ResNet**)
- Visual Geometry Group (VGG) Network (**VGG Net**)
- **DenseNet**
- **InceptionNet**
- Other pre-trained Image Classification Nets on pytorch library:

<https://pytorch.org/vision/main/models.html>



VGG Net 16

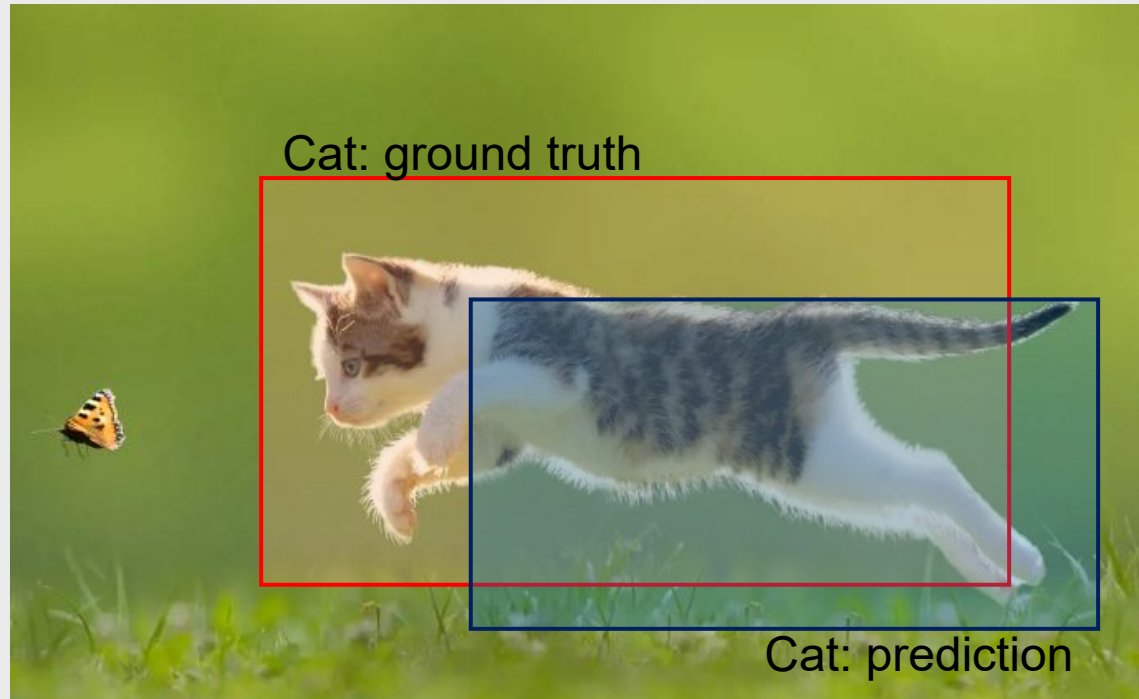
Image Segmentation (Concept)



Intersection over Union (IoU)

9:45 AM

IoU measure the performance of image segmentation and object detection algorithms performance



ground truth

prediction

Area of Intersection

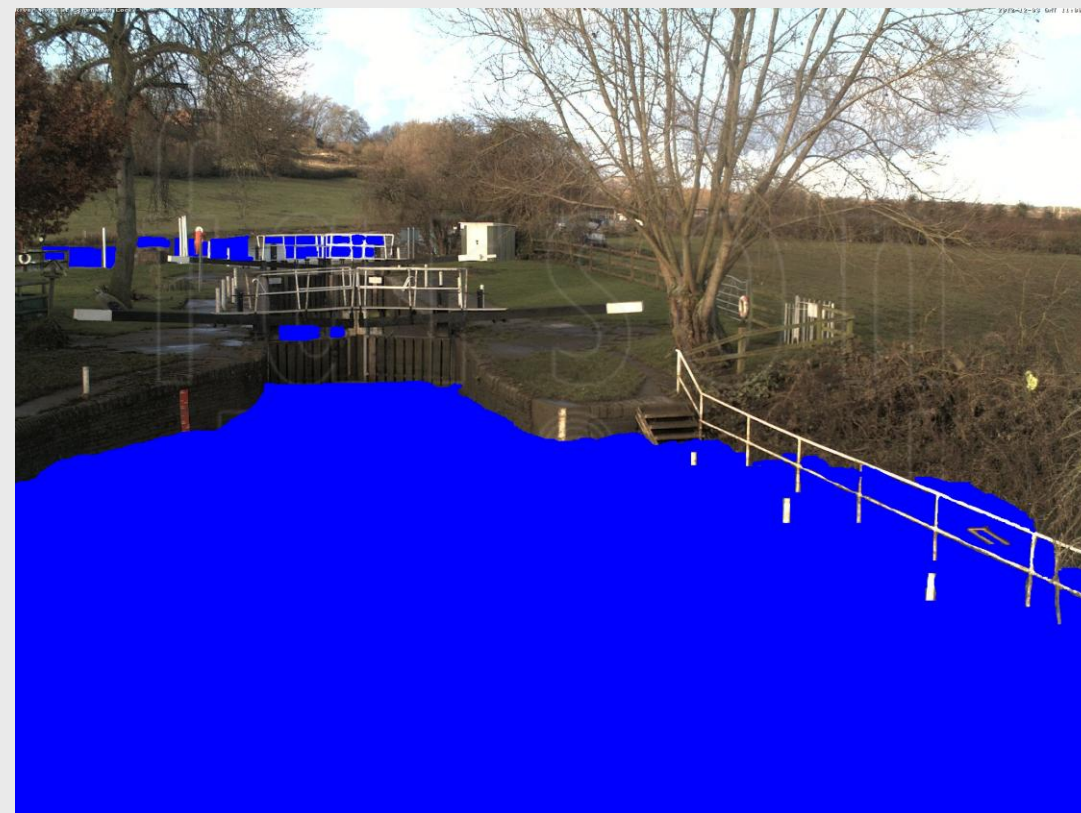
$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

ground truth

prediction

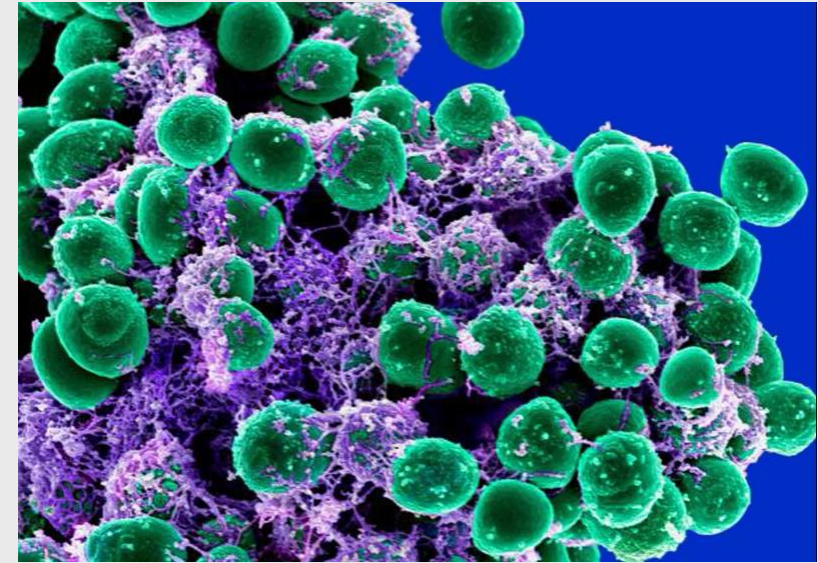
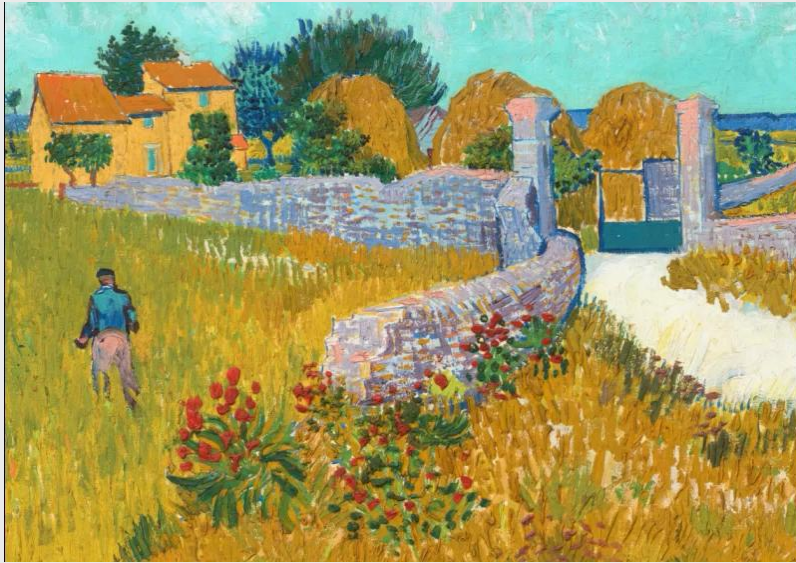
Image Segmentation used for water level estimation

9:45 AM



Pixel-wise water segmentation of RGB images for river water-level monitoring or flood monitoring

Image Segmentation (Concept)



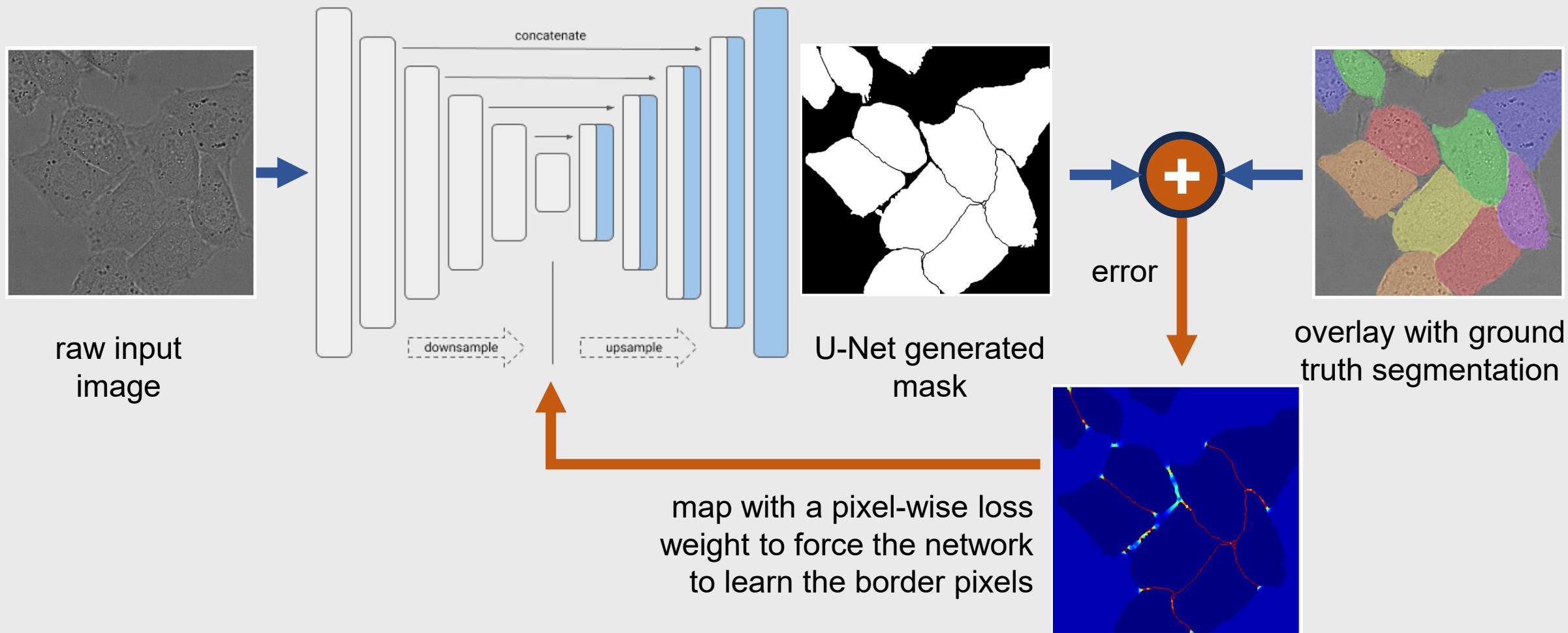
Segment Anything Model from Meta (2023)

<https://segment-anything.com/>

UNet

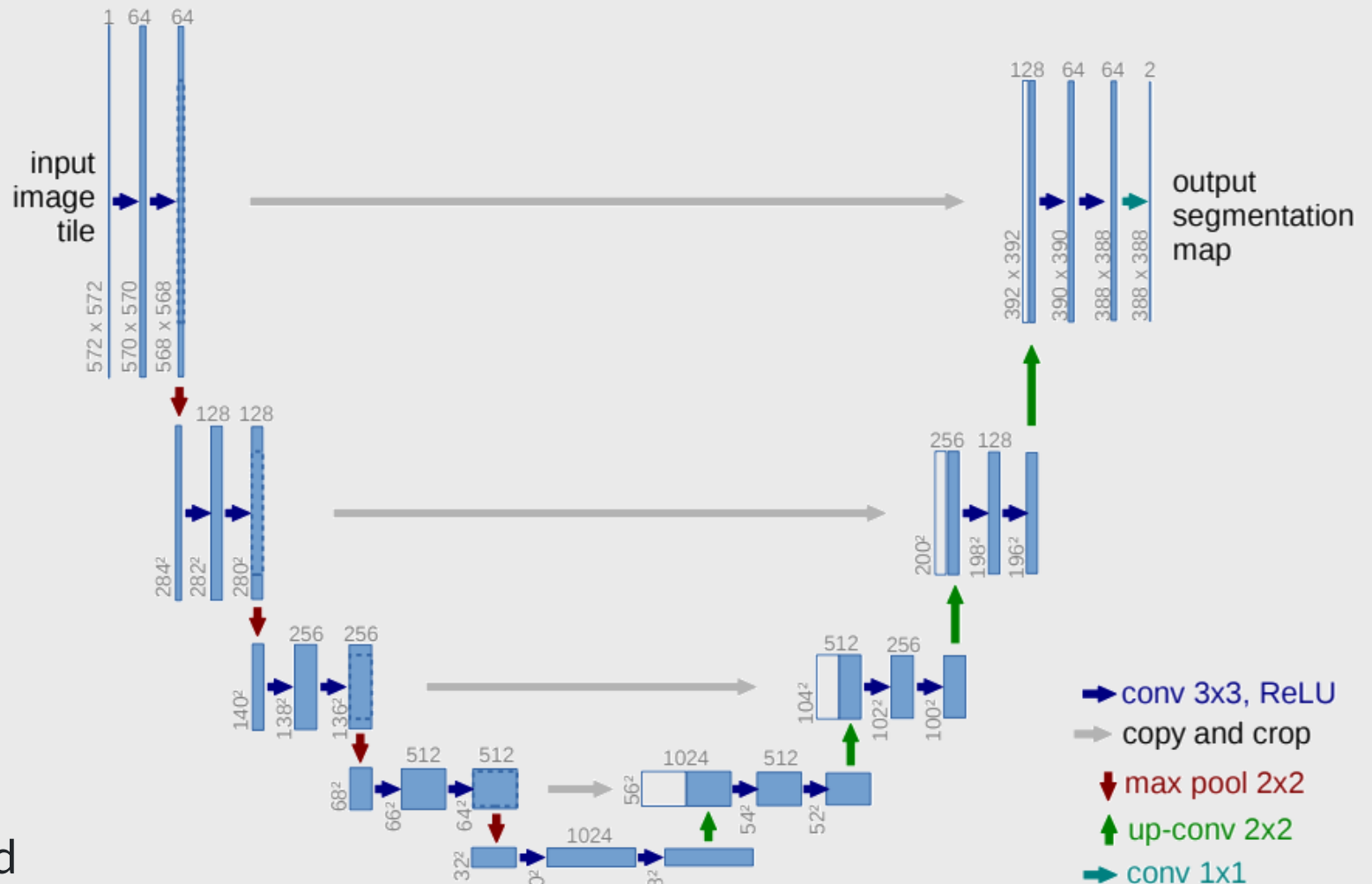
9:45 AM

U-Net is a typical CNN architecture for semantic segmentation. It has a contracting path (down sampling) and an expansive path (up sampling).



U-Net

- **Contracting path** is a repeated application of **two 3x3 convolutions** and a **ReLU** and a **2x2 Max Pooling** operation with stride 2
- **Expansive path** is an up sampling of the feature map followed by a **2x2 Convolution** (“up-convolution”) that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path, and **two 3x3 convolutions**, each followed by a **ReLU**.



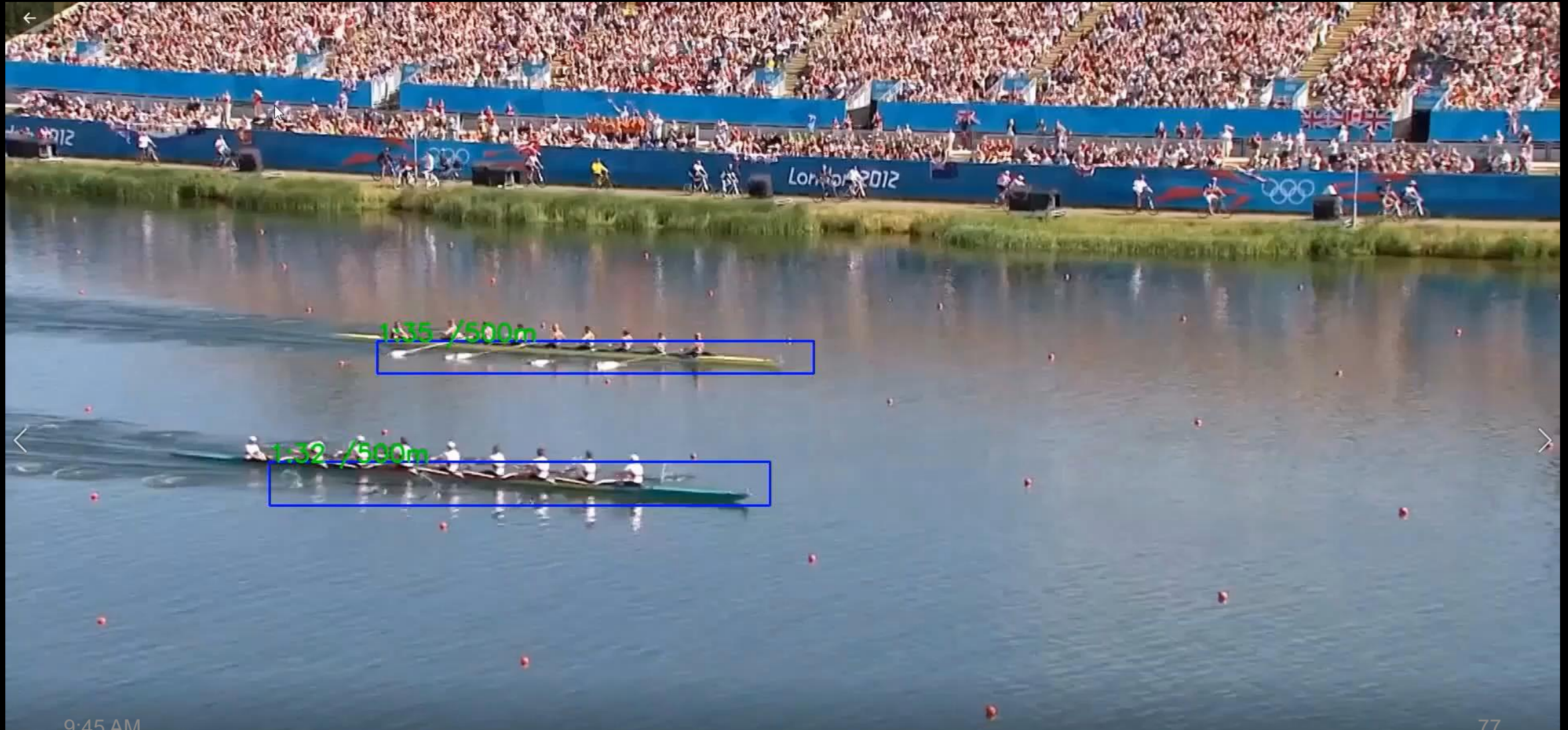
Object Detection

Drone based object detection
One of my previous student's project results



Application: Boat (Object) Detection and boat speed measurement

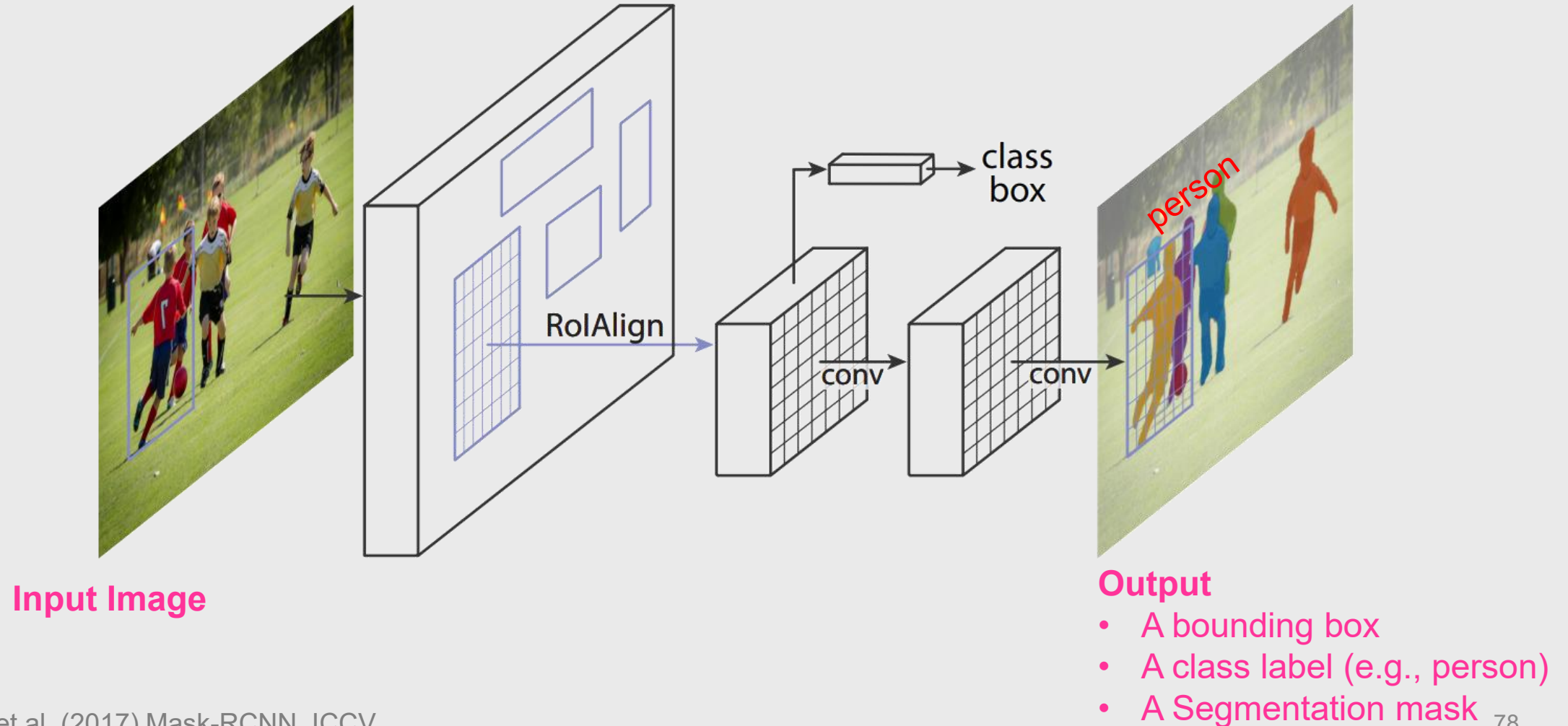
One of my Undergraduate student's project



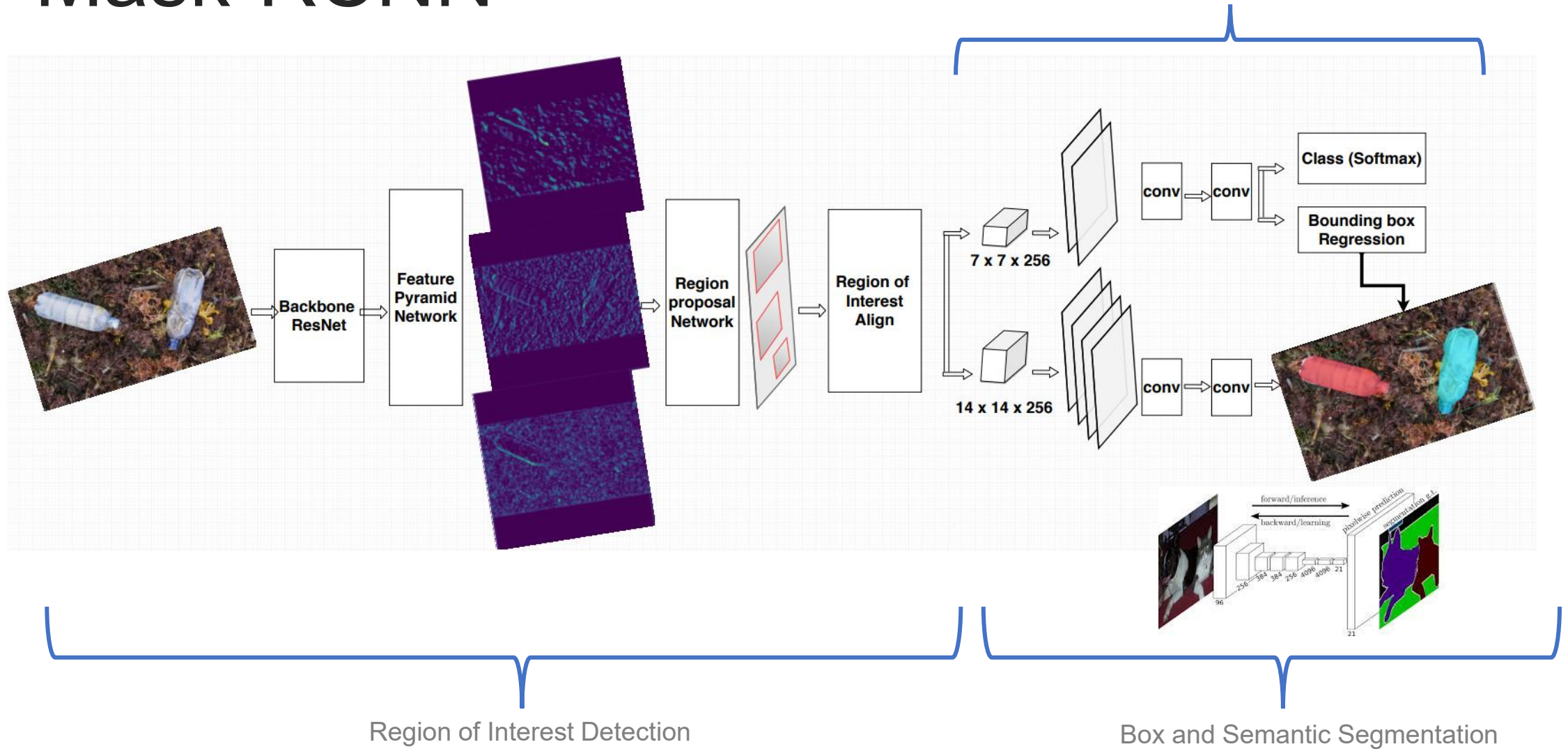
Masked RCNN

Masked Region Based Convolutional Neural Networks

Instance Segmentation (Object detection + Semantic Segmentation)



Mask-RCNN



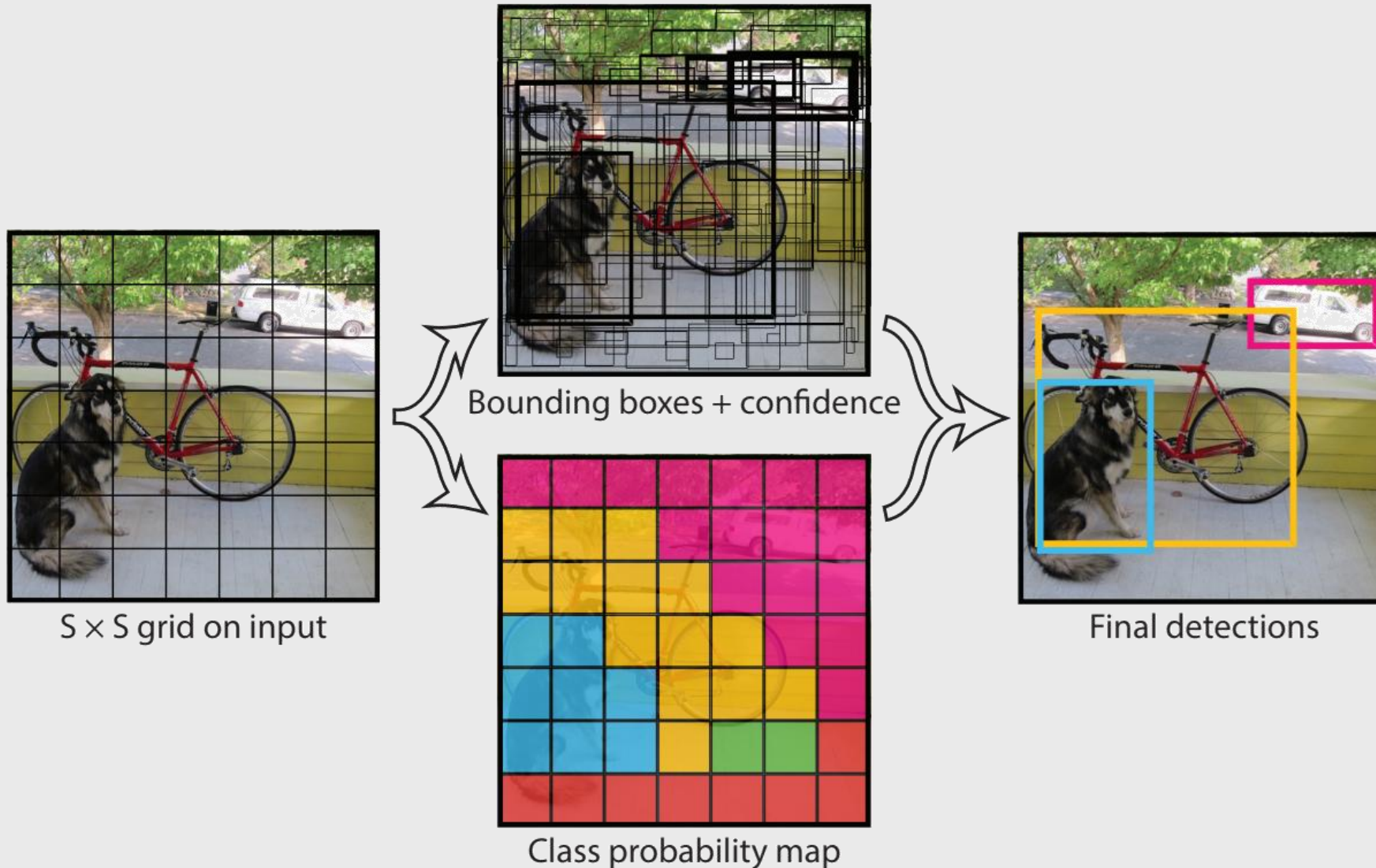
Instance Segmentation and Object Detection: Plastic Pollution Detection

9:45 AM



You Look Only Once (YOLO) Model

9:45 AM



Object detection in YOLO models are as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell it predicts:

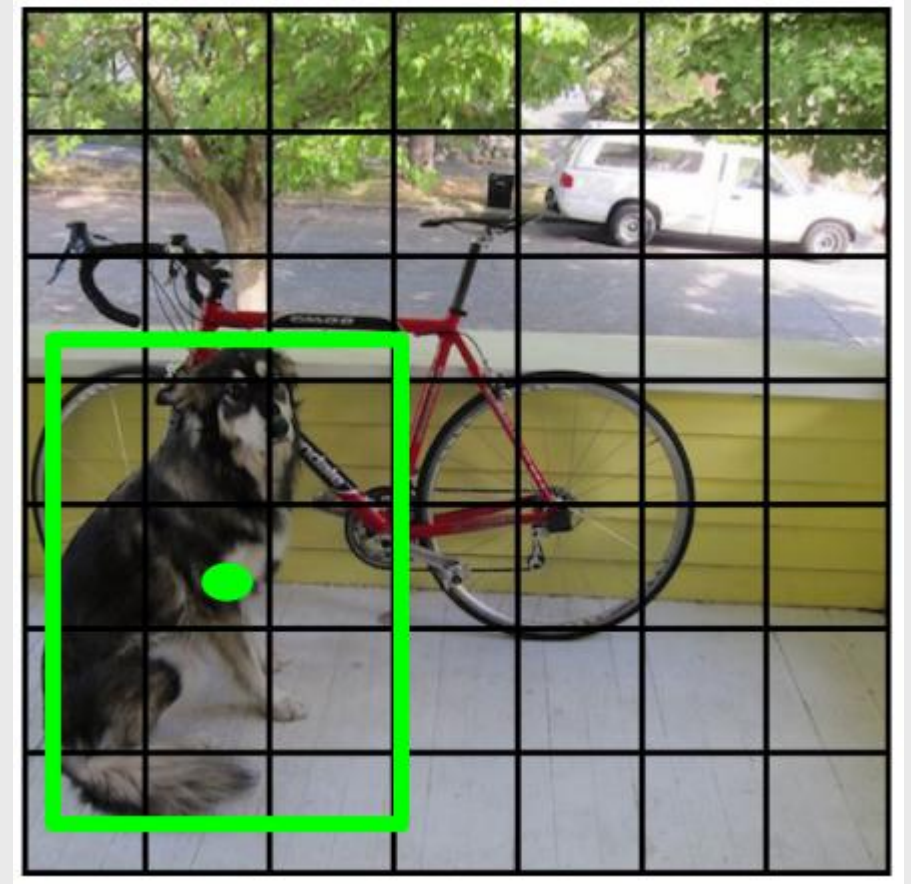
- B bounding boxes,
- confidence for those boxes, and
- C class probabilities.

You Look Only Once (YOLO) Model

9:45 AM



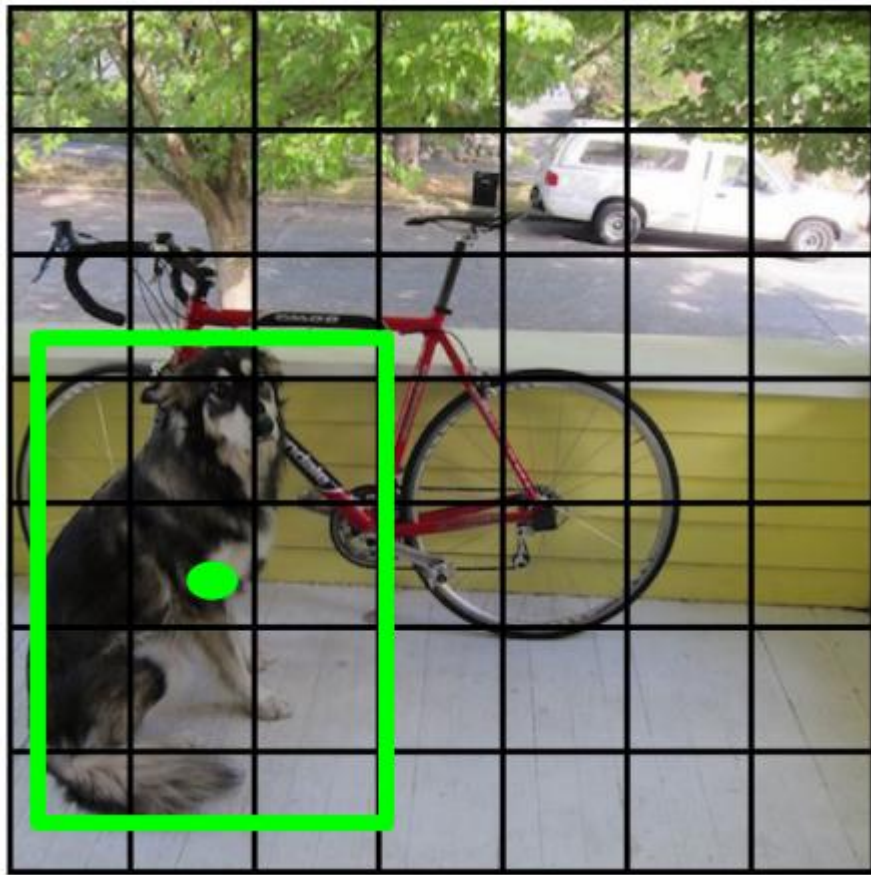
Original input



Ground Truth

You Look Only Once (YOLO) Model

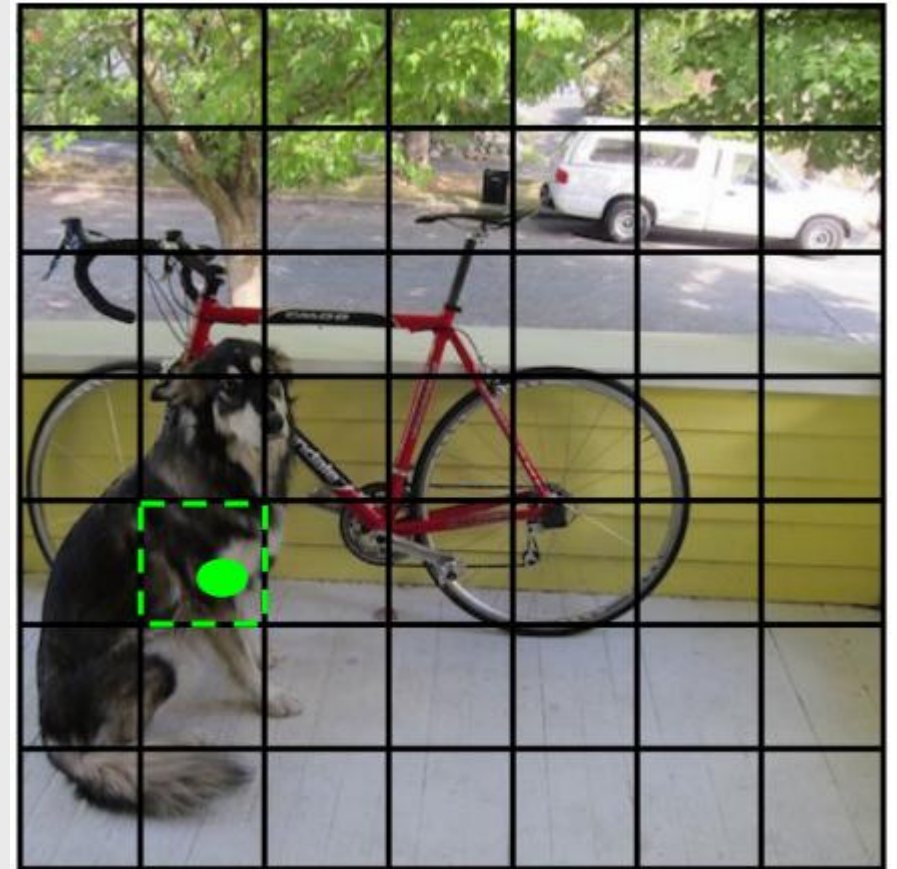
9:45 AM



Ground Truth



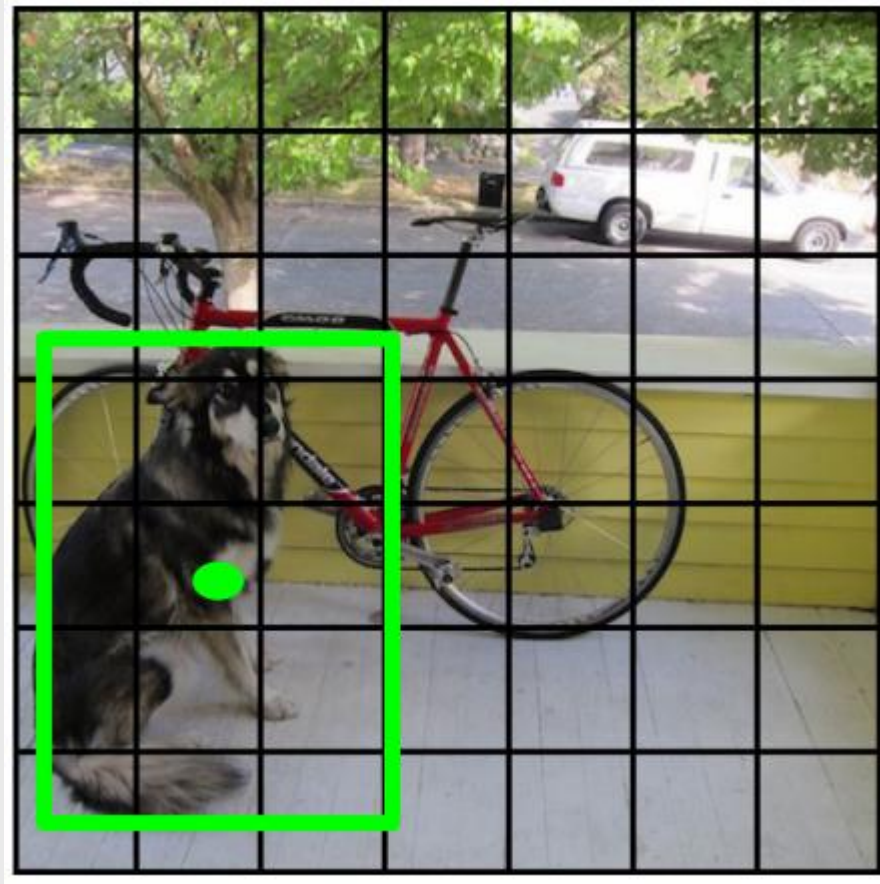
**Match
predicted
cell with
ground
through**



Cell Prediction

You Look Only Once (YOLO) Model

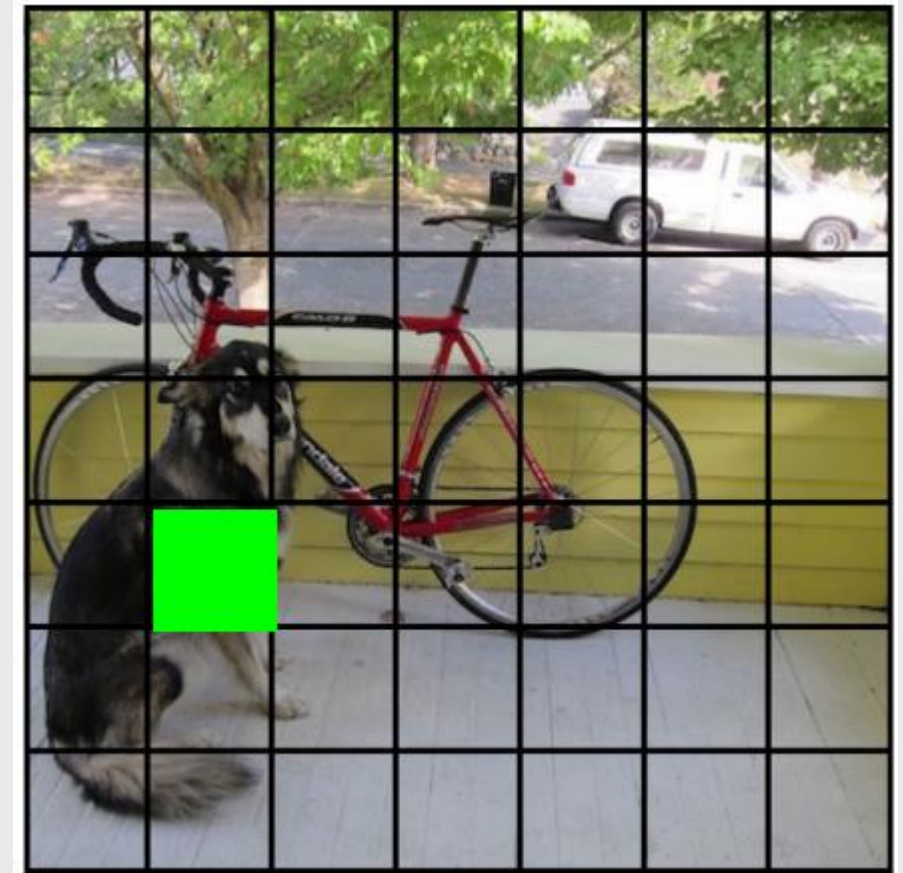
9:45 AM



Ground Truth



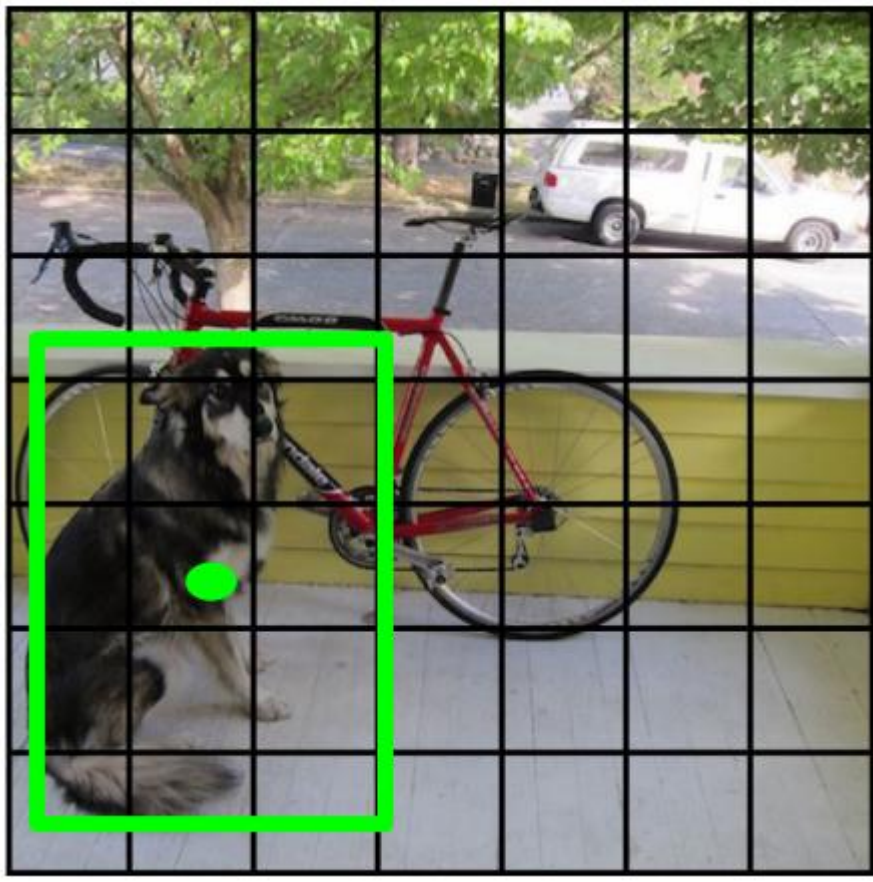
Check the
class
prediction
(class
probability,
i.e.,
SoftMax)



**Class
Prediction**

You Look Only Once (YOLO) Model

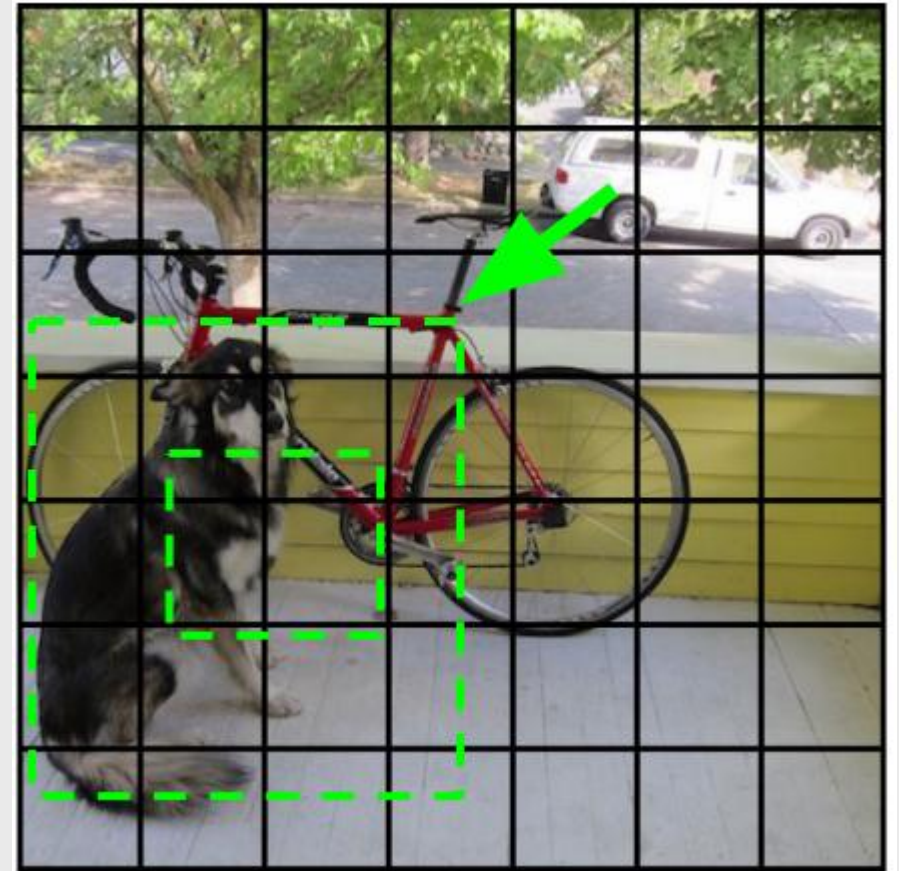
9:45 AM



Ground Truth



Check the
box
prediction
and its
confidence
(i.e., IoU)

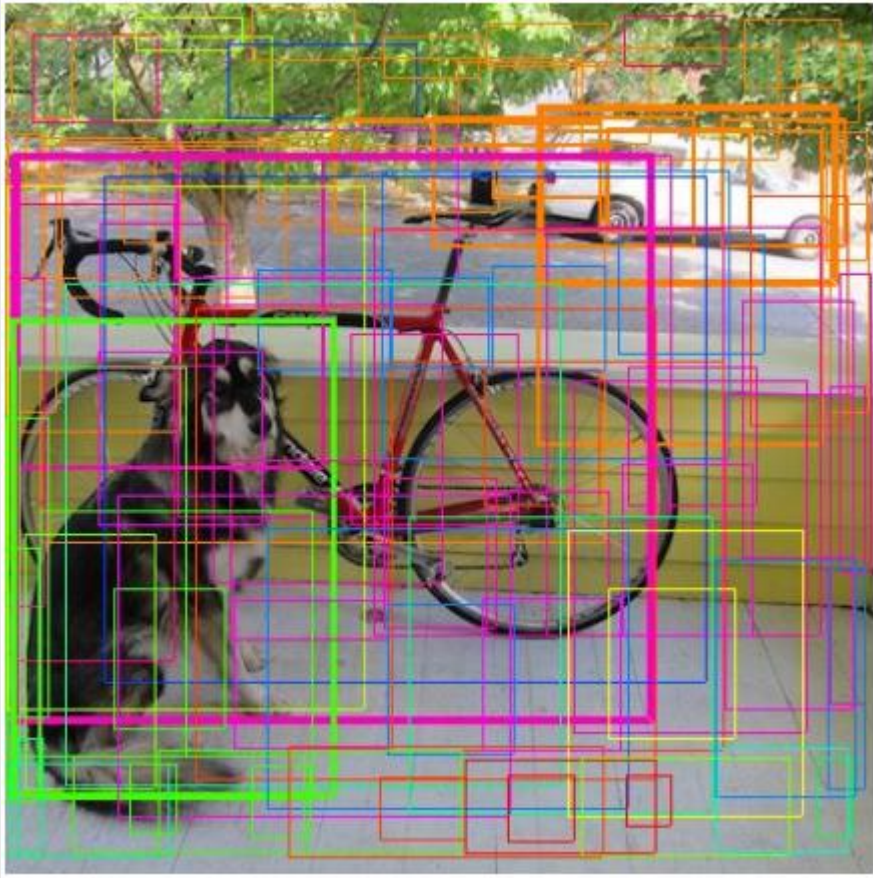


**Box
Prediction**

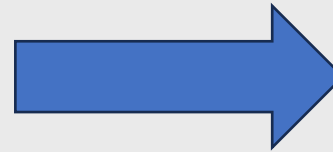
You Look Only Once (YOLO) Model

9:45 AM

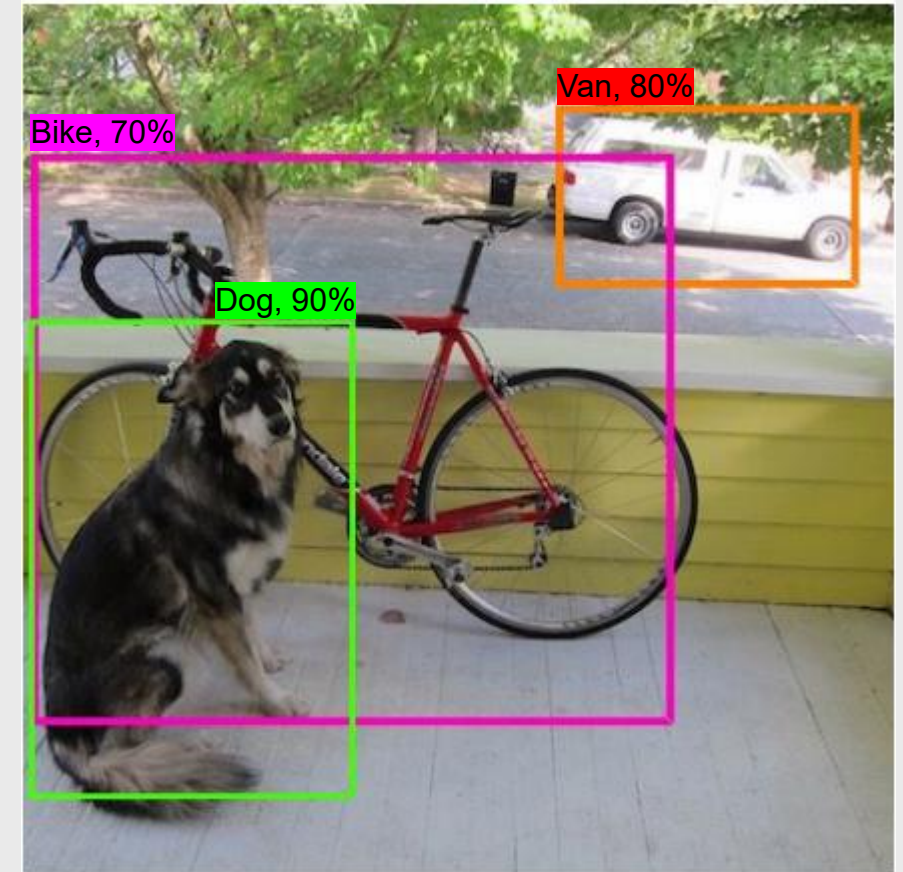
Non-maximal suppression (NMS) $Box = \operatorname{argmax}(C(P_1), C(P_2), \dots, C(P_n))$



**Box Prediction with
confidence scores**



**Perform NMS,
i.e., keep only
boxes with
maximal
confidence
score**



Final Detection